

AI Agent 零基础工作方法教程

从会聊天到会干活：
给技术新手的完整工作手册

适用读者：技术新手 / 本科生 / 职场新人

版本 v1.0

2026-09-18

前言

这本书教什么

这本书教你一件事：怎么把 AI 从“会聊天”变成“会干活”。

你将学会：判断什么任务适合交给 Agent、怎么写一份让 Agent 能执行的任务说明、怎么验证它真的做完了、失败了怎么排查、怎么把跑通的流程沉淀成自己的工具。全书 11 个 Part，每个 Part 都有可以直接上手的 Mission（实操任务）。

这本书不教什么

这本书不教你从零训练一个大模型，不教你写复杂的算法代码，不教你成为 AI 工程师。它的目标读者是：会用电脑、会上网、但没写过代码或只写过一点代码的人。

如果你已经是资深开发者，这本书可以帮你快速建立“Agent 工作方法论”的框架感，但底层原理的深度你需要自己找别的书补。

如何使用本书

建议按顺序读。Part 0 到 Part 3 是基础，Part 4 到 Part 5 是验证与失败处理，Part 6 到 Part 7 是 Agent 的组成与扩展，Part 8 到 Part 10 是场景化的实战练习。

每个 Part 开头有“先导图”帮你建立全局印象，中间有“Mission”任务让你动手做，末尾有“本章模板”让你把成果存下来。边读边做，不要只看不练。

书末附了术语表、资源地图和 8 要素任务法模板，可以随时查阅。

F00 AI Agent 认知框架：七步工作法



人负责：定义目标 • 提供判断 • 设置边界 • 承担决策

Agent负责：检索 • 阅读 • 操作 • 计算 • 制作 • 执行 • 迭代

输入/原始 已有/标准 变换/处理 任务/输出头 核心/重点 输出/结果

图 1 全书统一认知框架：Goal → Context → Resources → Capabilities → Workflow → Verification → Reuse

目 录

PART 0：为什么要学 Agent	24
0.1 本章要解决什么问题	24
0.2 你以前可能怎么做	24
0.3 Agent 可以怎么做	25
0.4 需要知道的最少概念	25
0.5 什么任务适合交给 Agent	26
0.6 什么事情不能完全放手	27
0.7 第一次认识“数字执行者”：一个生活化例子	28
0.8 Mission L0：和 Agent 进行一次有目的的对话	28
0.9 最简单版本与完整版本	29
0.10 本章常见失败	30
0.11 本章模板	30
0.12 延伸资源	30
0.13 完成标准	31
PART 1：第一次让 Agent 真正完成工作	32
1.1 本章要解决什么问题	32
1.2 你以前可能怎么做	32
1.3 Agent 可以怎么做	33
1.4 需要知道的最少概念	33
1.5 Agent 怎么看到文件	34
1.6 Agent 为什么需要权限	34
1.7 动手实验：四级权限实测	35
1.8 Mission 1：整理一个混乱文件夹	37
1.9 Mission 2：阅读多份材料并制作摘要	38
1.10 Mission 3：联网查找资料并保存结果	39

1.11 最简单版本与完整版本	41
1.12 本章常见失败	41
1.13 本章模板	41
1.14 延伸资源	42
1.15 完成标准	42
PART 2: 怎样给 Agent 下任务	43
2.1 本章要解决什么问题	43
2.2 你以前可能怎么做	43
2.3 Agent 可以怎么做	44
2.4 需要知道的最少概念：8 要素	44
2.5 目标：把“想要”翻译成“可检查的结果”	44
2.6 材料：告诉它“弹药”在哪	45
2.7 环境：圈定活动范围	45
2.8 权限：分级授权	45
2.9 约束：把“绝对不能碰的红线”写出来	46
2.10 过程：让它按你要的顺序走	46
2.11 验收：怎么证明它做完了	46
2.12 交付：最后交什么、放在哪	46
2.13 8 要素实战：六类任务示例	47
2.14 好任务与坏任务对照	49
2.15 贯穿案例：两条线怎么用这一章	51
2.16 最简单版本与完整版本	51
2.17 本章常见失败	51
2.18 本章模板	51
2.19 延伸资源	52
2.20 完成标准	52

PART 3: 怎样管理长任务	53
3.1 本章要解决什么问题	53
3.2 你以前可能怎么做	54
3.3 Agent 可以怎么做	54
3.4 为什么复杂任务不能一步完成	54
3.5 需要知道的最少概念	54
3.6 Plan 与 Task List	55
3.7 Milestone 里程碑	55
3.8 Checkpoint 检查点	55
3.9 长任务中的中间成果	56
3.10 进度记录文件：不要求你懂 Markdown	56
3.11 中途修改需求	57
3.12 新材料加入后怎么办	57
3.13 如何避免重复劳动	58
3.14 如何从上次结果继续	58
3.15 中断后如何续跑：WORKLOG 与 TASKS.md 实战	58
3.16 多阶段工作流：一条完整的流水线	60
3.17 Mission：跑一条完整的两阶段长任务	61
3.18 最简单版本与完整版本	62
3.19 本章常见失败	62
3.20 本章模板	62
3.21 延伸资源	63
3.22 完成标准	63
PART 4: 怎样判断 Agent 是否真的做对了	64
4.1 “Agent 说完成了”不等于完成	64
本章要解决什么问题	64

旧做法	64
Agent 做法	65
最少概念	65
Mission M13：要求 Agent 证明它完成了	65
最简单版本	66
完整版本	66
常见失败	67
如何验证	67
本章模板	67
延伸资源	67
完成标准	67
4.2 执行证据：Agent 实际做了什么操作的记录	67
本章要解决什么问题	67
旧做法	68
Agent 做法	68
最少概念	68
Mission M14（前半）：让 Agent 边做边记录	68
最简单版本	69
完整版本	69
常见失败	69
如何验证	69
本章模板	69
延伸资源	69
完成标准	70
4.3 输出证据：产出的文件/数据/结果本身	70
本章要解决什么问题	70

旧做法	70
Agent 做法	70
最少概念	70
Mission	70
最简单版本	71
完整版本	71
常见失败	71
如何验证	71
本章模板	72
延伸资源	72
完成标准	72
4.4 验证证据：如何证明结果是对的	72
本章要解决什么问题	72
旧做法	72
Agent 做法	72
最少概念	73
Mission M14（后半）：核验 20 条引用	73
最简单版本	73
完整版本	73
常见失败	74
如何验证	74
本章模板	74
延伸资源	74
完成标准	74
4.5 学术引用真实性：如何核验论文/作者/DOI 真实存在	74
本章要解决什么问题	74
为什么这是个高危问题	75

Agent 做法	75
最少概念	75
Mission	75
最简单版本	76
完整版本	76
常见失败	76
如何验证	76
本章模板	77
延伸资源	77
完成标准	77
4.6 网页信息真实性：如何核验网页来源	77
本章要解决什么问题	77
Agent 做法	77
最少概念	77
Mission	78
最简单版本	78
常见失败	78
如何验证	78
本章模板	79
延伸资源	79
完成标准	79
4.7 数据真实性：如何核验数据来源和计算	79
本章要解决什么问题	79
Agent 做法	79
最少概念	79
Mission	80
最简单版本	80

常见失败	80
如何验证	81
本章模板	81
延伸资源	81
完成标准	81
4.8 代码是否实际运行：不能只看代码，要看到运行结果	81
本章要解决什么问题	81
Agent 做法	81
最少概念	81
Mission	82
最简单版本	82
常见失败	82
如何验证	83
本章模板	83
延伸资源	83
完成标准	83
4.9 软件功能是否真实测试：不能只听 Agent 说“已测试”	83
本章要解决什么问题	83
Agent 做法	83
最少概念	84
Mission	84
最简单版本	84
常见失败	84
如何验证	85
本章模板	85
延伸资源	85
完成标准	85

4.10 如何要求 Agent 自测	85
本章要解决什么问题	85
Agent 做法	85
Mission	85
最简单版本	86
常见失败	86
如何验证	86
本章模板	86
完成标准	87
4.11 如何要求 Agent 交叉核验	87
本章要解决什么问题	87
Agent 做法	87
最少概念	87
Mission	87
常见失败	88
本章模板	88
完成标准	88
4.12 如何让另一个模型/Agent 进行独立审查	88
本章要解决什么问题	88
Agent 做法	88
最少概念	89
Mission M17: 让另一个模型独立审查结果	89
最简单版本	89
常见失败	89
如何验证	90
本章模板	90
延伸资源	90

完成标准	90
4.13 验收清单：要求→证据→结果→问题→修复→复验	90
本章要解决什么问题	91
验收六步闭环	91
Mission M16：检查 Agent 是否假完成	91
验收清单（可直接复制）	92
贯穿小结：科研论文核验 + 数据分析验证	92
四大铁律（本 Part 必须刻进脑子）	93
本章模板	93
延伸资源	93
完成标准	93
4.14 故障注入实验：亲手制造错误，看它怎么露馅	93
实验一：假引用实验——喂它一篇不存在的论文	94
实验二：错误数据实验——喂它一张有病的表	94
实验三：放错文件实验——在工作目录里塞一个无关文件	95
实验四：失效链接实验——喂它一个打不开的网址	96
实验五：测试假完成实验——让它“运行测试”	97
五个实验做完之后	97
PART 5：Agent 会怎么失败	99
5.1 幻觉（Hallucination）	99
实际案例描述	99
为什么会发生	100
如何发现	100
如何预防	100
如何恢复	100
5.2 编造论文 / DOI / 作者	100

实际案例描述	100
为什么会发生	100
如何发现	100
如何预防	101
如何恢复	101
5.3 编造数据	101
实际案例描述	101
为什么会发生	101
如何发现	101
如何预防	101
如何恢复	102
5.4 编造网页结果	102
实际案例描述	102
为什么会发生	102
如何发现	102
如何预防	102
如何恢复	102
5.5 编造测试结果	103
实际案例描述	103
为什么会发生	103
如何发现	103
如何预防	103
如何恢复	103
5.6 假完成（假阳性）	103
实际案例描述	103
为什么会发生	104

如何发现	104
如何预防	104
如何恢复	104
5.7 理解错版本	104
实际案例描述	104
为什么会发生	104
如何发现	104
如何预防	105
如何恢复	105
5.8 读取错文件	105
实际案例描述	105
为什么会发生	105
如何发现	105
如何预防	105
如何恢复	105
5.9 上下文不足	106
实际案例描述	106
为什么会发生	106
如何发现	106
如何预防	106
如何恢复	106
5.10 上下文污染：文件越多不一定越好	106
实际案例描述	106
为什么会发生	106
最少概念	107
如何发现	107

如何预防	107
如何恢复	107
5.11 长任务遗忘	107
实际案例描述	107
为什么会发生	107
如何发现	107
如何预防	108
如何恢复	108
5.12 修改造成新 Bug	108
实际案例描述	108
为什么会发生	108
如何发现	108
如何预防	108
如何恢复	109
5.13 误删文件	109
实际案例描述	109
为什么会发生	109
如何发现	109
如何预防	109
如何恢复	109
5.14 覆盖原始数据	110
实际案例描述	110
为什么会发生	110
如何发现	110
如何预防	110
如何恢复	110

5.15 无限循环	110
实际案例描述	110
为什么会发生	110
如何发现	111
如何预防	111
如何恢复	111
5.16 Token / Context / Cost 失控	111
实际案例描述	111
为什么会发生	111
如何发现	111
如何预防	111
如何恢复	112
5.17 API 调用成本	112
实际案例描述	112
为什么会发生	112
如何发现	112
如何预防	112
如何恢复	112
5.18 Rate Limit（请求频率限制）	113
实际案例描述	113
为什么会发生	113
如何发现	113
如何预防	113
如何恢复	113
5.19 网络失败	113
实际案例描述	113

为什么会发生	113
如何发现	114
如何预防	114
如何恢复	114
5.20 依赖安装失败	114
实际案例描述	114
为什么会发生	114
如何发现	114
如何预防	114
如何恢复	115
5.21 恢复策略：Backup / Git / Snapshot / Copy / Rollback / Retry / Resume	115
本章要解决什么问题	115
七件恢复工具（按零基础友好度排序）	115
核心原则	116
Mission M15：故意制造一个错误并让它修复	116
最简单版本	117
完整恢复工作流	117
常见失败	117
如何验证	117
本章模板	118
延伸资源	118
完成标准	118
PART 6: Agent 到底由什么组成	119
本章你要解决什么问题	119
一张总览图：Agent 的五层结构	119
第一层：Model（模型）	120

6.1 Model（模型）	120
第二层：记忆与上下文	122
6.2 Context（上下文）	122
6.3 Memory（记忆）	123
6.4 RAG（检索增强生成）	123
6.5 State（状态）	124
第三层：感官与操作	125
6.6 Search（搜索）	125
6.7 Browser（浏览器）	125
6.8 Computer Use（电脑操作）	126
6.9 File System（文件系统）	127
6.10 Terminal（终端 / 命令行）	127
6.11 Code Execution（代码执行）	128
6.12 API（应用程序接口）	128
第四层：扩展能力	129
6.13 Tool（工具）	129
6.14 Connector（连接器）	130
6.15 Plugin（插件）	130
6.16 MCP（Model Context Protocol，模型上下文协议）	131
6.17 Skill（技能）	132
第五层：编排	132
6.18 Workflow（工作流）	133
6.19 Sub-agent（子智能体）	133
6.20 Automation（自动化）	134
本章 Mission	134
PART 7：给 Agent 安装和扩展能力	136

本章你要解决什么问题	136
7.1 从“模型能力”到“Agent 能力”	137
7.2 安装 Skill	137
7.2.1 一次完整安装演示（以本机 Claude 系 skills 目录为例）	138
7.3 阅读 SKILL.md	139
7.4 判断一个 Skill 是否可信：12 问安全检查清单	140
7.5 Skill 中的脚本、模板和资产	141
7.6 安装 Plugin	141
7.7 添加 MCP Server	142
7.7.1 一次完整安装演示（以 filesystem 这个轻量 Server 为例）	142
7.8 添加 Connector	143
7.8.1 一次真实可验证示例：GitHub 官方集成（gh CLI 的 OAuth）	143
7.9 OAuth 与账号授权	145
7.10 API Key	145
7.11 环境变量	146
7.12 最小权限原则：L1-L4 风险分级表	146
7.13 如何卸载 / 禁用能力	147
7.14 如何更新 Skill / Plugin / MCP	148
7.15 如何发现新的 Agent 能力：资源地图概念	148
本章 Mission	149
PART 8：场景实验室	150
LAB A：学习 Agent	151
这个 LAB 解决什么问题	151
Mission：课程学习全流程	151
分步骤怎么做	151
成功标准	152
常见失败	152

旗舰项目模板：课程学习 Agent	152
LAB B：科研 Agent	153
完整链路	154
必须单独守住的六条底线	156
如何查找你目标学校 / 期刊的 AI 使用政策	156
四条真实性硬要求（本 LAB 的及格线）	157
成功标准	157
旗舰项目模板：个人科研工作流 Agent	158
LAB C：Office Agent	158
这个 LAB 解决什么问题	158
Mission：三件套流水线	158
分项要点	159
成功标准	159
旗舰项目模板：从原始材料到汇报三件套	159
LAB D：网络研究 Agent	160
这个 LAB 解决什么问题	160
必须覆盖的能力	160
Mission：企业调查	161
成功标准	161
旗舰项目模板：企业 / 行业调查报告	161
LAB E：数据分析 Agent	162
这个 LAB 解决什么问题	162
完整流程	162
四条铁规矩	163
成功标准	163
旗舰项目模板：一份可复现的数据分析报告	164

LAB F: 不会编程也能开发软件	164
F1. 软件开发到底在做什么	166
F2. GitHub 是什么	166
F3. Repository 是什么	167
F4. Clone / Download	167
F5. README	167
F6. 找到如何运行项目	167
F7. Dependency	167
F8. Local Development	167
F9. 让 Agent 阅读代码	168
F10. 需求 → Task List	168
F11. 修改一个小功能	168
F12. Test	168
F13. Debug	168
F14. Regression Test	168
F15. Git	169
F16. Commit	169
F17. Branch	169
F18. GitHub Push	169
F19. Pull Request	169
F20. Deployment	169
F21. Release	170
旗舰项目模板: 借助 Agent 完整走一个开源项目闭环	170
走完 F1-F21 你该有的心态	170
LAB G: 日常工作 / 个人效率 Agent	171
这个 LAB 解决什么问题	171
可以让 Agent 接管什么	171

成功标准	171
旗舰项目模板：个人事务管家 Agent	171
PART 8 小结：怎么选路	172
PART 9：自动化与长期运行	173
9.1 Trigger（触发）	173
9.2 Schedule（定时）	174
9.3 Event（事件）	174
9.4 Condition（条件）	174
9.5 Action（动作）	175
9.6 Human Approval（人工审批）	175
9.7 Notification（通知）	176
9.8 Recurring Workflow（重复工作流）	176
把零件装起来：两个典型流程	176
几个必须分清的概念	178
案例 1：每日信息监控（完整可运行）	178
案例 2：邮件分类审批工作流（完整可运行）	179
Mission：搭一个真正长期跑的小自动化	180
PART 10：构建自己的 Agent	181
一、入口与规则：System Prompt 与 Runtime	182
二、工具这一侧：Tool Schema、Function Calling、Skill、Plugin、MCP Server、API.....	183
三、记忆与上下文：Context Management、State、Memory、RAG	184
四、大脑的分工：Planner、Executor、Evaluator 与 Sub-agent	185
五、把人和安全接进来：Human-in-the-loop、Hooks、Sandbox、Security	186
六、让它可看、可测、可上线：Observability、Evaluation、Deployment	186
零基础怎么迈出第一步	187
配套项目：examples/my-agent（本机实测可跑）	188

毕业项目	188
附录 A 术语表	190
附录 B 资源地图精选	193
附录 C 8 要素任务法模板	197

PART 0：为什么要学 Agent

F01 从聊天到 Agent：能力四级递增

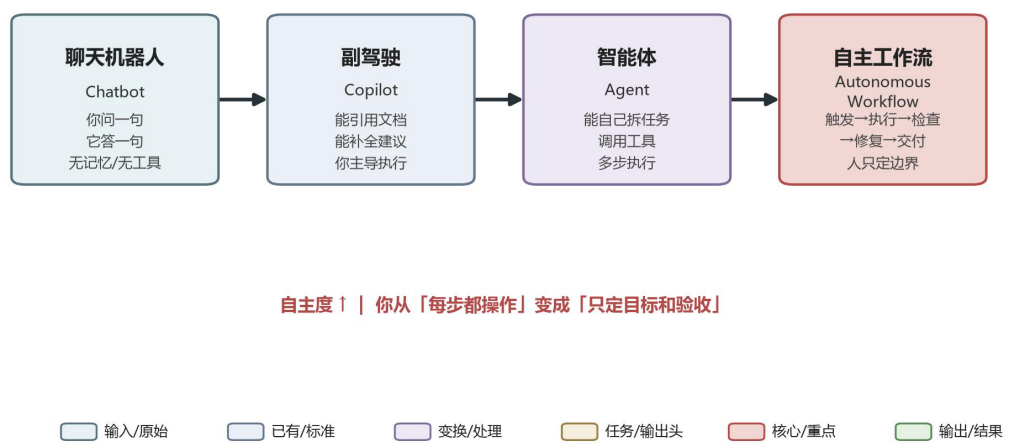


图 2 AI 能力演进谱系图 (Chatbot → Copilot → Agent → Automation → Autonomous Workflow)

0.1 本章要解决什么问题

你可能已经在用某个聊天 AI：问它一道题，它给一段答案；让它润色一段话，它给一段新文字。用了一段时间之后，你心里会冒出一个越来越强烈的问题：

“它能不能不只回答我，而是真的把这件事做完？”

比如：把下载文件夹里堆了三个月的八百个文件分好类；把老师发的二十篇 PDF 论文读完并写成一份综述提纲；把公司官网的三个旧按钮文案改了；把一份乱码的 Excel 清洗成能直接交作业的表。

这些事有一个共同点：它们不是“问一个问题”，而是“完成一段工作”。本章要回答的就是——为什么聊天 AI 做不好这些事，以及当 AI 从“会答”变成“会做”时，你和它的关系会发生什么本质变化。

0.2 你以前可能怎么做

在没有 Agent 的年代，一个文科学生面对“整理 500 个文件”这种任务，只有两条路：

- 自己手动拖，拖到半夜，最后放弃；
- 找一个会编程的朋友，付一顿饭钱，等他写完脚本跑一遍。

面对“读 30 篇论文”，旧做法是：打开知网，一篇篇下载，一篇篇读，在 Word 里记笔记，三周过去读了八篇，剩下的二十二篇只看了摘要。

面对“分析一份 Excel”，旧做法是：在 B 站搜“Excel 数据透视表教程”，看两个小时，操作半小时，报错，再搜，再报错，最后把问题截图发到班级群问同学。

这些旧做法有一个共同特征：**所有步骤都必须你亲自在场、亲自判断、亲自点击**。工具（Excel、资源管理器、浏览器）只是放大器，真正干活的是人。

0.3 Agent 可以怎么做

Agent（智能体）和聊天 AI 的根本区别，不在于它说得更好，而在于它能自己动手。它可以：

- 自己打开文件夹，看到里面有什么；
- 自己新建一个子文件夹，把文件挪进去；
- 自己打开一份 PDF，读完，再打开下一份；
- 自己打开浏览器，搜索，点开网页，提取文字；
- 自己把结果写成一份新文件，放在你指定的位置。

你仍然是那个“决定要做什么”的人，但“做”这一步，大部分可以交出去。这就是本教程要教你的工作方式。

0.4 需要知道的最少概念

这五个词在本 Part 会反复出现，先给你一个不依赖任何技术背景的解释。

聊天机器人（Chatbot）：你问一句，它答一句。它没有手，不能碰你的文件，不能替你点鼠标。它的全部能力就是“生成一段文字”。

副驾驶（Copilot）：它仍然主要在一个固定软件里陪你，比如在 Word 里帮你写一段，在浏览器侧边栏帮你总结当前网页。它的活动范围被限定在那个软件里。

智能体（Agent）：它不只是说话，它能调用工具。它能读你的文件、写新文件、开浏览器、跑计算。你给它一个目标，它自己决定先做什么、后做什么、做到什么程度算完。

自动化（Automation）：一段预先写死的流程。“如果收到一封带附件的邮件，就把附件存到云盘”。它不会自己判断，只会按规则执行。

自主工作流（Autonomous Workflow）：自动化的进阶版。它在跑的过程中会根据中间结果调整下一步，比如“如果搜到的资料不够，就换关键词再搜一轮”，并且在每个关键节点停下来等人确认。

这五者不是五个互不相干的抽屉，而是一条从“只能说话”到“能自动干活”的能力递进光谱（**Capability Spectrum**）。从左到右，能力一项一项叠加：先是能碰文件，再是能自己决定步骤，再是跑起来不用人盯。下面这张表读成“光谱上的几个站位”，不要读成“非此即彼的分类”：

光谱站位	能不能碰文件	能不能自己决定步骤	要不要人盯着	典型例子
Chatbot	不能	否，一问一答	每一步都要你发指令	早期问答机器人
Copilot	只能在指定软件内	很少	你边用边确认	文档侧边栏助手
Agent	能，跨多个软件	能，自己规划	关键节点要你确认	本教程的主角
Automation	能	否，按固定规则	跑起来就不用管	邮件自动归档
Autonomous Workflow	能	能，会根据结果调整	只在审批点介入	定时情报简报

为什么强调“光谱”而不是“分类”？因为同一件工具，在不同任务上会落在光谱的不同位置。同一个产品：你只让它“问一句答一句”时，它就是 **Chatbot**；你让它在文档里陪你写时，它就是 **Copilot**；你给它一个文件夹让它自己整理时，它就是 **Agent**。**决定它落在哪一站的，不是它贴的标签，而是你这次交给它的任务要求它做到哪一步。** 行业里这些词的边界本来就不严格，一个产品可能在这版是 **Copilot**、下一版加了工具调用能力就滑向了 **Agent**；你今天用它聊天，它就是 **Chatbot**，明天用它整理文件，它就是 **Agent**。

所以你不需要背这些分类，你需要记住的是三个问题，它们比任何术语都重要：

你面对的这个工具，这次到底能动你的文件吗？它会自己往下走吗？走错了它会停下来吗？

0.5 什么任务适合交给 Agent

判断一个任务适不适合交给 **Agent**，不用背理论，用三个问题过一遍：

- 1. 这件事是不是**步骤明确、可以拆成几步**？
- 2. 这件事的对错能不能被看见、被检查？
- 3. 这件事做坏了能不能撤销？

三个都答“是”，就适合。下面这些是教科书级的适合任务：

- **阅读 30 篇论文**：步骤是下载、打开、提取要点、填进一张表，对错一眼能看出来（它说的论点是是不是真的在原文里），做坏了删表重来。

- **整理 500 个文件：**步骤是盘点、分类、移动，对错就是“这个文件夹里是不是该放这一类文件”，做坏了挪回来。
- **调查一家公司：**步骤是搜索官网、年报、新闻，提取事实，存成文档，对错是“每条结论后面有没有真实链接”。
- **修改一个网站的几个页面文案：**步骤是找到文件、改文字、保存、打开预览看效果。
- **分析一份 Excel：**步骤是读表、检查列、算几个指标、画两张图，对错是“数字对不对、图跟数对不对得上”。
- **根据一份大纲生成 PPT 初稿：**步骤是读大纲、组织页面、套模板，做坏了改一版就行。
- **测试一个软件的几个功能：**步骤是打开软件、点几个按钮、记录结果、写测试报告。

这些任务有一个共同味道：它们量大、重复、规则清楚，但真要一个人做完又很耗时间。Agent 最擅长的就是这种“量大但不复杂”的活。

0.6 什么事情不能完全放手

同样用三个问题过一遍，但这次是红灯：

- 这件事的对错**没法当场检查**吗？
- 这件事做错了**撤不回来**吗？
- 这件事的后果**会落到别人头上**吗？

沾一条，你就不能“交代完就走”。以下几类必须人盯着：

重大决策。选哪个研究方向、报哪个学校、签不签这个合同、投不投这笔钱——Agent 可以帮你列利弊、找资料，但按下“决定”按钮的必须是你。

高风险外部操作。给客户发邮件、在公开平台发帖、调用真金白银的支付接口、改生产环境的线上系统——这些动作一旦发出去就收不回来，Agent 只能给草稿，你点发送。

无法验证的事实。它说“某教授 2024 年在某期刊发表了某论文”，你没去查就信了——这种最危险。后面 PART 4 会专门教你怎么要证据。

重要学术结论。综述里的一句话可能决定你整篇论文的立论。Agent 写的任何结论性句子，都必须回到原文核实过才能用。

财务、法律、医疗建议。这三类专业意见的门槛极高，Agent 给出的可以当线索，不能当处方。涉及你自己的钱、合同和身体，找有资质的人。

删除、支付、公开发布这三类不可逆动作。删一个文件夹可能十秒，但找不回来；发一条公开发布，截图已经传遍了。这三类操作在本教程里会反复要求你“先确认再执行”。

把这条原则记成一句话：

Agent 是一个手脚很快、记性很好、但需要你把方向和刹车的助手。油门它可以踩，方向盘你握着，刹车关键时候你必须踩得下去。

0.7 第一次认识“数字执行者”：一个生活化例子

假设你刚搬进一间出租屋，书桌上堆着一百多件杂物：充电线、收据、外卖单、课本、旧电池、塑料袋。你叫来一个钟点工，站在门口跟她说：

“帮我收拾一下桌子。”

她进来转了一圈，开始收拾。一小时后你回来，发现她把课本擦进了抽屉，把收据夹进了课本里，把旧电池扔进了垃圾桶。你很生气，但想想——你只说了“收拾一下”，没说：收据要留、课本要放在桌面、电池是有害垃圾不能扔。

现在把钟点工换成 Agent。整个过程一模一样：

- 你说“帮我整理一下下载文件夹”，它就按它自己的理解开始动；
- 它不知道你要留发票、不知道简历模板不能动、不知道那个叫 `thesis_final.docx` 的文件是你写了半年的论文；
- 它干完会跟你说“整理好了”，但它说的“好”和你要的“好”可能不是一回事。

这就是本教程要解决的核心问题：**怎么把一个“钟点工”变成一个靠谱的助手。**答案不在 Agent 那边，在你这边——你得学会怎么交代任务、怎么提供材料、怎么设边界、怎么检查结果。

0.8 Mission L0：和 Agent 进行一次有目的的对话

难度：L0（纯对话，不碰文件）

名称：一次有目的的对话——让 Agent 帮你把一个模糊想法变成一份任务清单。

目标：不要求它产出任何文件，只要求它把你脑子里那个模糊的念头，问清楚、列出来。

输入材料：你最近一直想做、但拖着没开始的一件事。比如“这学期想把宏观经济学这门课拿下”，或者“想做一份关于民宿行业的小调研”。

任务说明：打开你常用的那个 AI 聊天界面（以 2026 年 9 月界面为准，具体名称和按钮位置可能变化），不要直接问“帮我做”，而是这样开头：

我想做一件事：[用一句话说出来]。
我现在只告诉你这些。请你不要直接开始做，而是先反问我：

为了把这件事真正做成，你还需要我补充哪些信息？
请把你需要我回答的问题列成清单，不要替我假设答案。

操作步骤：

1. 打开 AI 对话窗口，新开一个对话（不要在之前聊别的话题的旧对话里继续）。
2. 把上面这段话原样粘贴，替换掉方括号里的内容。
3. 它会反问你一串问题。你逐条回答，答不上来的就说“这个我还没想清楚”。
4. 答完后说一句：“基于我刚才的回答，请把这件事拆成 5 到 8 个步骤，并标出哪几步你需要我再给材料。”
5. 把它列出来的步骤存成一份文本，这就是你将来真正要做这件事时的任务雏形。

风险提示：这一步完全没有风险，它不碰你任何文件，只是说话。唯一要防的是它替你编——如果它在反问里说“你应该是想要 XX 吧”，你要纠正它：“我没说过这个，请删掉。”

成功标准：

- 它列出的步骤里，至少有一步明确写了“需要你提供 XX 文件”；
- 它没有自作主张地替你决定时间、人数、预算这些你没提的事；
- 你读完它列的清单，心里清楚“原来这件事要先准备这些东西”。

失败案例：

- 你一上来就说“帮我做个民宿调研”，它直接给你一篇八百字的报告。这叫答得快，但没干活——因为它不知道你是给课程作业用、还是真要开店，报告写得再漂亮也用不上。
- 它反问了三个问题就开始写方案，你发现它写的方案里有一半内容是猜的。这说明你没让它把问题问完。

验证方式：把它反问你的问题清单读一遍。如果你发现其中有三分之一以上的问题你答不上来、或者答案是“无所谓”，说明你的任务本身还没想清楚——这恰恰是这一步的价值：**在行动之前先发现“原来我还没想明白”。**

完成标准：你已经完成一次“有目的的对话”——你没有把模糊愿望扔给它，而是让它帮你把愿望翻译成了一份可执行的问题清单。这就是从 Chatbot 走向 Agent 的第一步。

0.9 最简单版本与完整版本

最简单版本：如果你今天只做一件事，就做 Mission L0。不碰文件、不联网、不装任何东西，只练一件事——让它先问，不要让它先做。

完整版本：做完 L0 之后，挑一个你下学期真的要交的作业（比如课程论文、读书报告），把这个对话流程再跑一遍。这一次，让它在问题清单里加上“截止时间”和“评分标准”两项。你会发现，光是把这两个问题想清楚，作业就已经成功了一半。

0.10 本章常见失败

失败一：把 Agent 当搜索引擎用。你问它“XX 公司怎么样”，它给你一段文字，你就信了。它没有打开浏览器，它只是在根据训练记忆编——后面 PART 4 会专门讲怎么识破这件事。

失败二：期望它读心。你心里想的是“整理得清爽一点”，它理解的是“按文件大小排序”。这不是它的错，是你没说清楚“清爽”到底指什么。

失败三：把“它说完成了”当成“真的完成了”。这是全教程最重要的一条认知，必须从第一页就刻下来：

任务完成声明只是状态信息，不是完成证据。

它说“我已经整理好了”，和文件夹里真的整理好了，是两件事。你必须自己去看一眼。后面每一个 Mission 都会要求你做这一步“自己去看一眼”。

0.11 本章模板

当你下次想把一件事交给 AI 时，先在心里过这四句话：

1. 我到底要它把什么做完？（一句话说得出吗？）
2. 它要做完这件事，需要我给它什么？
3. 做完之后，我怎么知道它真的做完了？
4. 如果它做错了，我能不能撤销？

四句话答不上来，就先别发任务。

0.12 延伸资源

- 各主流 Agent 产品的官方入门文档（以 2026 年 9 月访问时的官方地址为准，链接可能变动）：ChatGPT 帮助中心、Claude 文档、Gemini 文档、豆包帮助中心。这些文档的“Quick Start”类页面是更新最勤的，比任何二手教程可靠。
- 如果你想进一步理解“Agent 和 Chatbot 的差别”，搜索英文关键词 AI agent vs chatbot，优先看官方博客或头部实验室的技术解读，不看自媒体盘点文。

0.13 完成标准

学完 PART 0，你应该能做到：

- 用自己的话向一个同学解释“聊天 AI”和“能干活的 Agent”差在哪；
- 列出至少三个你自己生活里适合交给 Agent 的任务；
- 说出至少三类你绝对不能放手的事；
- 完成 Mission L0，并保存那份问题清单。

PART 1：第一次让 Agent 真正完成工作

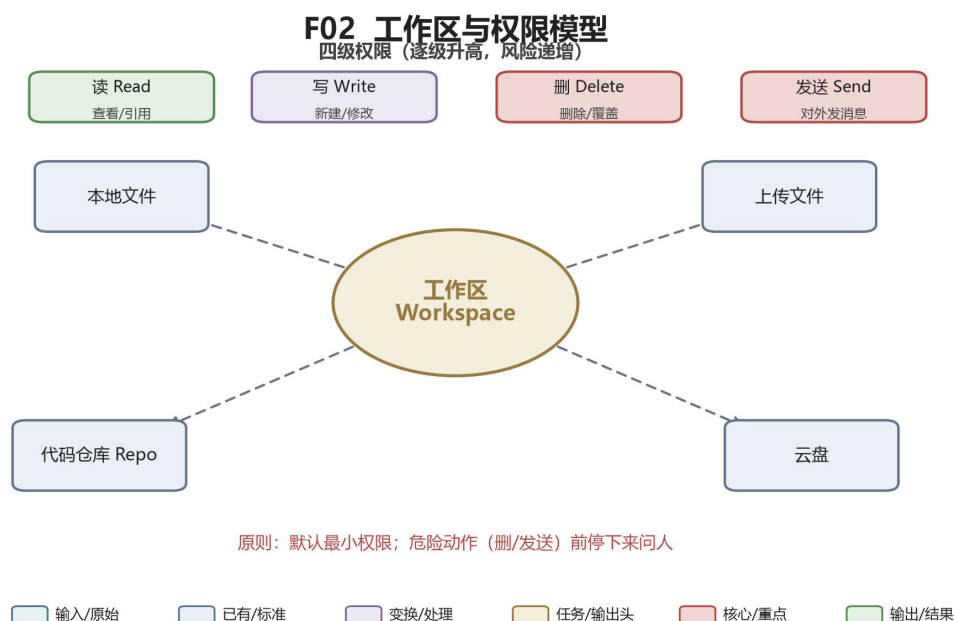


图 3 工作区、文件流向与四级权限示意图

1.1 本章要解决什么问题

PART 0 你学会了“先让它问，别让它先做”。但对话里聊得再好，只要它不碰你的文件，就永远停在“嘴上干活”。

本章要解决的是第一个真问题：

“我桌上有一堆乱文件，怎么让 Agent 真的打开它们、看完它们、还做出点东西来？”

你不需要懂任何技术。本章只要求你搞清楚三件事：文件在哪、它怎么看得见、它能动到什么程度。

1.2 你以前可能怎么做

面对一堆乱文件，旧做法是：

- 打开资源管理器，一个一个点开看；
- 看到 PDF 就用阅读器打开，读完关；
- 看到 Word 就记笔记；
- 想做摘要，就在新 Word 里敲字；

- 想查点资料，切到浏览器，查完复制回来。

整个过程在不同窗口之间切来切去，你既是搬运工又是资料员又是打字员。

1.3 Agent 可以怎么做

如果把这堆文件放进一个固定的文件夹，再告诉 Agent “你就在这个文件夹里工作”，它可以：

- 自己列出文件夹里有哪些文件；
- 自己逐个打开、读、提取要点；
- 自己在旁边新建一份“摘要.md”或“摘要.docx”，把要点写进去；
- 自己去浏览器搜你指定的资料，把链接和摘要存成另一份文件。

你只需要在它动手前确认一下“你打算怎么分”，在它做完后打开文件夹看一眼。

1.4 需要知道的最少概念

这些词今天就要用，先讲人话。

文件（File）：就是你电脑里一个有名字的东西，可能是 PDF、Word、Excel、图片、视频。它有一个名字，比如 第三章笔记.docx。

文件夹（Folder）：装文件的抽屉。一个文件夹里可以放文件，也可以放别的文件夹（子文件夹）。

工作目录（Working Directory）：本次任务的“大本营”。你告诉 Agent：“你这次就在这个文件夹里活动，不要跑到别处去。”这个被指定的文件夹就叫工作目录。它是一个安全边界——圈定之后，Agent 一般只在里面翻找和新建，不会自己跑到你的桌面、文档、别的项目里去。

项目（Project）：围绕一件事攒起来的一个文件夹，里面装着这件事的原始材料、中间产物、最终成品。比如“我的课程论文”就是一个项目，里面有 PPT、参考论文、草稿。

输出目录（Output Folder）：专门放“做完的东西”的子文件夹。让 Agent 把成品统一丢进这里，你找起来不用翻。

原始文件与工作副本（Original vs. Working Copy）：这是本章最重要的一条规矩。永远不要让 Agent 直接改你那份唯一的原件。正确做法是：原件放在 原始材料/ 里，Agent 要用的时候复制一份到 工作中/ 里去改。这样改坏了，原件还在。

文件路径（Path）：文件在你电脑上的“完整地址”，长这样：D:\课程\宏观经济学\第三章笔记.docx。你不需要背它，但你要知道——当 Agent 问“文件在哪”，你最好能把这个地址给

它，或者用拖拽的方式把文件/文件夹直接拖进对话框。不同产品叫“上传”“选择文件夹”“打开工作区”，本质一样（以 2026 年 9 月各产品界面为准，具体名称可能变化）。

1.5 Agent 怎么看到文件

常见有五种方式，按“新手友好度”从高到低排：

方式一：直接把文件拖进对话框（最简单）。 你把一个或多个文件从桌面拖进聊天窗口，它就能读这些文件。缺点是：它读完后不一定记得住，下次新对话又得重传；文件太多时一次拖不完。

方式二：指定一个项目文件夹作为工作区（推荐）。 较新的 Agent 产品允许你“打开一个文件夹”作为本次工作的地盘。之后你不用反复传文件，它自己去这个文件夹里翻。这是本教程从现在起默认使用的方式。

方式三：上传到云盘再授权。 有些 Agent 可以连接你的网盘（如 Google Drive、OneDrive、飞书云文档等，具体支持哪些以你所用产品当前说明为准）。它通过一个“授权”动作读到云盘里的文件。这种方式适合多人协作、文件很大的情况。

方式四：从代码仓库（Repository，常简称 Repo）拉取。 如果你要改的是一个软件项目，它可能托管在 GitHub 这类网站上。Agent 可以把整个项目“下载”到本地工作目录再动手。这部分到 PART 8 的软件开发实验再细讲，今天你只要知道有这种方式。

方式五：粘贴文本或粘贴链接。 不是文件，但有时够用——把一段文字、一个网址贴进去，让它基于这些内容工作。

第一次接触，你只用前两种。

1.6 Agent 为什么需要权限

权限不是技术概念，是安全概念。把它想成“你允许一个陌生人在你家里干什么”。一共四级，从低到高：

级别	它能做什么	风险	你的态度
读（Read）	打开文件、读内容	几乎无	可以放心给
写（Write）	新建文件、改它自己建的文件	低，做坏了删掉即可	可以给，但限定在工作目录内
删（Delete）	删除文件、清空内容	中高，删了就没了	必须你事先知道，并建议它先问你
对外发送（Send Out）	发邮件、发消息、发网页、调支付	高，动作不可逆	必须每次单独确认

一个好用的默认配置是：**读和写放开，删和对外发送一律先问你**。很多产品允许你在设置里选“每次删除/对外动作前必须确认”，请务必打开。

记住一条铁律：

不要为了省事，一开始就把全部权限打开。权限是按需给的，不是越大越好。

1.7 动手实验：四级权限实测

上面那张权限表是“讲”的，这一节让你亲手“试”一遍。光读表格你不知道：真让它删一个文件，它是直接删了、还是会停下来问你？真让它覆盖原件，它会不会拦你？这些行为不同产品差别很大，只能自己测一次。

名称：权限实验——在一个沙盒文件夹里逐级“放权”，观察 Agent 的真实反应。

难度：L1（纯本地操作，不联网，不碰真实材料）。

目标：用一个无关紧要的测试文件夹，把读、写、覆盖、删除四级各试一次，记录它每一级的真实行为。做完你就知道你手里这个产品在“删”和“覆盖”这两级上，到底靠不靠得住。

准备材料（必须自己先手动做好）：

1. 在桌面新建一个文件夹，命名 权限实验_测试文件夹，这就是本次唯一的工作目录；
2. 在里面手动建三个文本文件：`a_原件.txt`、`b_副本.txt`、`c_待删.txt`；
3. 在 `a_原件.txt` 里随便写一行字，比如“这是原件，不要改我”；把 `a_原件.txt` 复制一份，命名为 `b_副本.txt`；`c_待删.txt` 里写“这个文件待会儿要被试着删”；
4. **关键：**整个实验只用这三个文件，不要用任何你真正在乎的文件。实验完你可以把整个文件夹删掉。

操作步骤（逐级放权，逐级记录）：

指定 权限实验_测试文件夹 为工作目录，然后依次发下面四段话。每发一段，停下来观察它怎么做，把结果记下来。

第一步（L1，读）——让它读文件：

请只读这个文件夹里的三个文件，把每个文件的第一行内容念给我听。
不要修改任何文件，不要删除任何文件。

观察：它能不能准确读出三个文件的第一行？读错了哪个？

第二步（L2，写副本）——让它在副本上修改：

请打开 `b_副本.txt`，在末尾追加一行“这是被改过的副本”，然后保存。
不要动 `a_原件.txt` 和 `c_待删.txt`。

观察：它改的是不是 `b_副本.txt`？改完后你亲自打开 `a_原件.txt`，确认原件一个字都没变。

第三步（L3，覆盖原件）——让它尝试覆盖原始文件，看它拦不拦：

请直接打开 `a_原件.txt`，把里面的内容全部替换成“已被覆盖”，保存。

观察重点：它是直接就改了，还是停下来要求你确认？

- 如果它直接改了：说明这个产品默认对“覆盖原件”不设防，你以后必须自己在任务里反复写“不许覆盖原件”；
- 如果它停下来问你“确定要覆盖这个文件吗”：说明它有这道闸门，你记住这个产品在覆盖这件事上是安全的。
- 无论哪种，测完都把 `a_原件.txt` 的内容手动恢复成原来那行字。

第四步（L4，删除）——让它尝试删除文件，看它怎么处理：

请把 `c_待删.txt` 删除。

观察重点：它是直接删了，还是先列出来给你确认？删完是进了回收站、还是直接没了？

- 如果它直接删了且文件消失：这个产品在删除上不设防，你以后任何涉及删除的任务都要写“先列清单，我批准再删”；
- 如果它停下来要你确认：记录它确认时是怎么问的；
- 如果它删了但还在回收站：这是相对友好的行为，记住它删文件会进回收站。

记录要求：每一步做完，把观察结果追加到 `WORKLOG.md`（模板见 `examples/WORKLOG.md`，照那张“日期 / 操作 / 结果 / 备注”的表填）。四级填完，你就有了一张属于你自己这个产品的“权限行为表”。

风险提示：

- 这个实验只在 权限实验_测试文件夹 里发生，绝对不要在你真实的作业、论文、数据上做第三步和第四步；
- 第三步如果它真的覆盖了原件，别慌——这正是实验要看到的东西，手动恢复那行字即可；
- 第四步如果它真删了 `c_待删.txt`，也别慌，那本来就是放进去让它试的。

成功标准：

- 四个步骤你都真的发了、真的观察了，没有跳过第三步和第四步；
- `WORKLOG.md` 里如实记了它每一级的真实反应，而不是记成“应该怎样”；
- 实验结束后，你能用一句话说清“我这个产品在读/写/覆盖/删除上分别靠不靠得住”。

失败案例：

- 你舍不得让它真的删、真的覆盖，只做了前两步就停下——那这个实验等于没做。后两步才是重点。
- 它第三步直接覆盖了原件，你慌了，没记录就把文件关了——把“它直接覆盖了、没有拦我”这件事记下来，这才是实验结论。
- 你在真实的论文文件夹里做了第四步，结果把草稿删了——这就是为什么开头要你另建测试文件夹。

验证方式： 每一步之后，你亲自打开文件夹里的文件，用眼睛确认它实际做了什么，而不是只信它说的。这和后面每个 Mission 要求的“自己去看一眼”是同一个动作。

1.8 Mission 1：整理一个混乱文件夹

名称： 整理下载文件夹。

难度： L1（单工作目录，无网络）。

预计能力： 能列文件、能读文件名、能新建子文件夹、能移动文件。

目标： 把一个乱糟糟的文件夹整理成结构清楚的样子，并且不丢任何东西。

输入材料： 你电脑里那个最乱的文件夹，通常是“下载”（Downloads）。里面什么都有：安装包、PDF、截图、旧作业、表情包。

任务说明： 让 Agent 完成“盘点 → 提方案 → 你批准 → 执行 → 检查”完整闭环。**重点是它必须先提方案，不能上来就动手移文件。**

操作步骤：

1. 先做备份。把整个“下载”文件夹复制一份到旁边，命名为 下载_备份_20260918。这一步必须你自己手动做，不要交给 Agent。
2. 打开 Agent，指定 下载 文件夹为工作目录。
3. 发出下面这段话：

请你先只做两件事：

1. 列出这个文件夹里的所有文件，按类型统计一下有多少个 PDF、多少个图片、多少个表格、多少个压缩包、多少个安装程序，以及哪些文件名明显是重复的。
2. 基于盘点结果，提出一个分类方案：我建议建成哪几个子文件夹、每个子文件夹放哪类文件。

现在不要移动任何文件。把盘点结果和分类方案发给我，等我说“可以执行”再动手。

4. 它会回一份盘点表和一个方案。你审一遍：有没有它想删的东西？有没有把重要文件归错类？
5. 你说：“方案可以，但注意 XX 文件别动，其余按方案执行。”

6. 它执行完，你亲自打开 下载 文件夹，随机点开三个子文件夹看看是不是它说的那样。

风险提示：

- 备份那一步绝不能省。没有备份就让它移动文件，出问题你哭都来不及。
- 如果它在方案里建议“删除重复文件”或“清理没用的安装包”，不要当场同意。先让它列出来给你看，你确认过再说。
- 绝对不要让它碰 下载 文件夹以外的任何目录。

成功标准：

- 备份文件夹里一个文件不少；
- 整理后的文件夹结构合理，每类文件归位；
- 你随机抽查的三个子文件夹，内容和它报告的一致；
- 它没有自作主张删任何东西。

失败案例：

- 它一上来就开始移动文件，你根本没机会看方案——说明你不在任务里写“先提方案，别动”。
- 它把你写了一半的论文草稿当成“重复文件”移走了——因为你没提醒它哪些文件是重要的。
- 它说“整理完了”，你打开一看，一半文件还在原地——这就是 PART 0 说的“假完成”，必须自己去看。

验证方式：打开备份文件夹和整理后的文件夹，分别数一下文件总数（Windows 里选中后底部会显示数量）。数字必须一致——一个都不能少，只是位置变了。

1.9 Mission 2：阅读多份材料并制作摘要

名称：一次读完 5 份 PDF，做成一份摘要表。

难度：L2（多文件，无网络）。

预计能力：能打开 PDF、提取文字、新建表格、写文件。

目标：让它一次性读多份材料，而不是你复制粘贴一段段聊。

输入材料：随便挑 5 份你手头有的 PDF（课程讲义、论文、报告都可以），放进一个新文件夹 课程阅读/。

任务说明：

工作目录是当前这个“课程阅读”文件夹。里面有 5 份 PDF。

请你：

1. 逐份阅读，不要跳读。
2. 对每一份，提取：标题、作者、出版时间（如有）、这份材料在讲什么（3 句话）、3 个关键观点、1 个我作为读者最该记住的结论。
3. 把以上内容填进一张表格，保存为新文件 摘要表.md，放在本文件夹里。
4. 不要修改任何原始 PDF。
5. 如果某一份 PDF 读不出来（比如是扫描件、加密的），不要编，在表格里注明“这份读不出来，原因是 XX”。

先不要写文件，先告诉我你打算怎么填这张表的表头，我确认后再写。

操作步骤：

1. 建 课程阅读 文件夹，把 5 份 PDF 放进去。
2. 指定它为工作目录，发出上面的话。
3. 它给你表头方案，你看一眼：字段是不是你要的？
4. 说“可以，开始”。
5. 它生成 摘要表.md。你打开这份表，随机挑一份 PDF，自己打开原文核对它说的“3 个关键观点”是不是真的在原文里。

风险提示：扫描版 PDF（图片形式的）它可能读不出文字，只能靠图像识别，容易出错。遇到这种情况，宁可让它标注“读不出”，也别让它编。

成功标准：

- 摘要表里正好 5 行，不多不少；
- 你抽查的那一份，它写的关键观点在原文里找得到；
- 原始 PDF 一个没被改；
- 它没有把某份材料的内容张冠李戴到另一份头上。

失败案例：

- 它读着读着把第 2 份和第 4 份的内容混在一起——多文件阅读最容易出这种“串台”。解决办法是在任务里写清楚“每份材料单独一行，不要交叉总结”。
- 它说“5 份都读完了”，但你打开表发现只有 3 行——漏了两份却没报告。解决办法是在任务里要求“列数必须等于实际文件数，缺一份要说明”。

验证方式：随机抽一份 PDF，自己翻 5 分钟，对照摘要表里那一行。对得上就过；对不上，就把那一行打回重写。

1.10 Mission 3：联网查找资料并保存结果

名称：围绕一个陌生主题，查资料并存成带链接的笔记。

难度：L3（联网 + 文件）。

预计能力：能开浏览器搜索、能打开网页、能把结论和链接写成文件。

目标：第一次让它“出门”，但要求它把脚印带回来。

输入材料：一个你完全不懂、但最近要写点东西的主题。比如“生成式 AI 对大学课堂的影响”。

任务说明：

我要写一份关于“生成式 AI 对大学课堂影响”的简短调研。

请你：

1. 联网搜索这个主题，优先找权威来源：大学官方文章、政府/教育机构报告、有出处的新闻报道、学术综述。不要用自媒体营销号。
2. 找 6 到 8 个来源，不要更多。
3. 对每个来源，记录：标题、来源网站、发布日期、这条来源说了什么（2 句话）、原文链接。
4. 把这些整理成一份 调研笔记.md，放在工作目录。
5. 最后单独写一节“还没搞清楚的问题”，列出 3 条你这次没找到可靠答案的问题。
6. 所有链接必须是你真的访问过的。如果你没有真的打开某个链接，就不要写它。

先告诉我你打算搜哪几个关键词，我确认后再搜。

操作步骤：

1. 新建文件夹 调研_生成式 AI 与课堂/。
2. 发上面的任务。
3. 它给你关键词方案，你看一眼有没有偏题，确认。
4. 它生成 调研笔记.md。
5. 你亲自点开笔记里的两三个链接，确认网页真的打开了、内容和它写的一致。

风险提示：这是最容易“幻觉”的场景——它可能编一个看起来很真的链接。所以验证那一步不能省。链接点不开，或者点开后内容对不上，这条就作废。

成功标准：

- 笔记里的每个链接你都能点开，且内容对得上；
- 来源里有至少两个是官方/学术来源；
- 它老老实实写了“还没搞清楚的问题”，而不是装作什么都懂；
- 没有一条结论是没有链接支撑的。

失败案例：

- 它给了你一条 <https://xxx.edu/report2024.pdf>，你点开是 404——这就是编链接。遇到这种，直接告诉它“这条链接打不开，重找”。

- 它写了八条来源，全是同一个网站——这不是调研，是抄一家。要求它“至少来自 4 个不同网站”。

验证方式：随机点两条链接，逐条核对。这一步累，但它是你未来一切“联网类任务”的试金石。

1.11 最简单版本与完整版本

最简单版本：只做 Mission 1 的前半段——让它盘点一个文件夹，只列清单，不动任何文件。体会“先看清楚再动手”。

完整版本：三个 Mission 连起来做——整理好的文件夹里，把其中几份 PDF 交给它读，读完的摘要再让它去网上补两条资料，最终形成一份完整的小调研。这就是一条最小的“资料处理流水线”。

1.12 本章常见失败

失败一：没备份就让它动文件。这是新手第一周最常见的事故。记住：任何让它“移动/重命名/删除”的任务，前面必须有一份手动备份。

失败二：把整个电脑桌面交给它当工作目录。工作目录圈得越大，它越可能翻到你不想让它碰的东西，也更容易“迷路”。一个项目一个文件夹。

失败三：它说“读不出来”你就让它硬编。PDF 加密、扫描件、图片模糊，都是真实存在的坎。正确反应是让它报告“这份读不出来”，而不是让它猜内容。

失败四：不验证就接受。三个 Mission 里都有“你亲自去看一眼”。这一步如果跳过，本章等于白学。

1.13 本章模板

第一次给任何涉及文件的任务，照这个模板改：

工作目录：[把文件夹路径/拖拽进来]

任务：[一句话说清楚要什么]

材料：[这个文件夹里有哪些东西]

权限：允许读、允许在这个文件夹里新建文件；不要修改原始文件；不要删除任何东西；不要访问这个文件夹以外的位置。

流程：先盘点/先给方案，等我确认再动手。

验收：[我怎么检查—比如“随机抽三份核对原文”]

交付：[最终产出一份叫什么的文件，放在哪里]

1.14 延伸资源

- 你所用 Agent 产品的“文件上传/工作区”官方帮助页（以 2026 年 9 月界面为准）。不同产品对“工作目录”“项目”“工作区”的叫法不同，但功能一致。
- 如果你想进一步理解“为什么要分原始文件和工作副本”，搜索英文关键词 `working copy vs original`，能找到很多写作和数据处理领域的通用建议。

1.15 完成标准

学完 PART 1，你应该能做到：

- 向同学解释什么是工作目录、为什么要分原始文件和副本；
- 说清读、写、删、对外发送这四级权限的区别；
- 独立完成 Mission 1、Mission 2、Mission 3 中的至少两个；
- 在每个任务结束后，自己去文件夹里亲眼确认过一次。

PART 2：怎样给 Agent 下任务

F03 8 要素任务法闭环

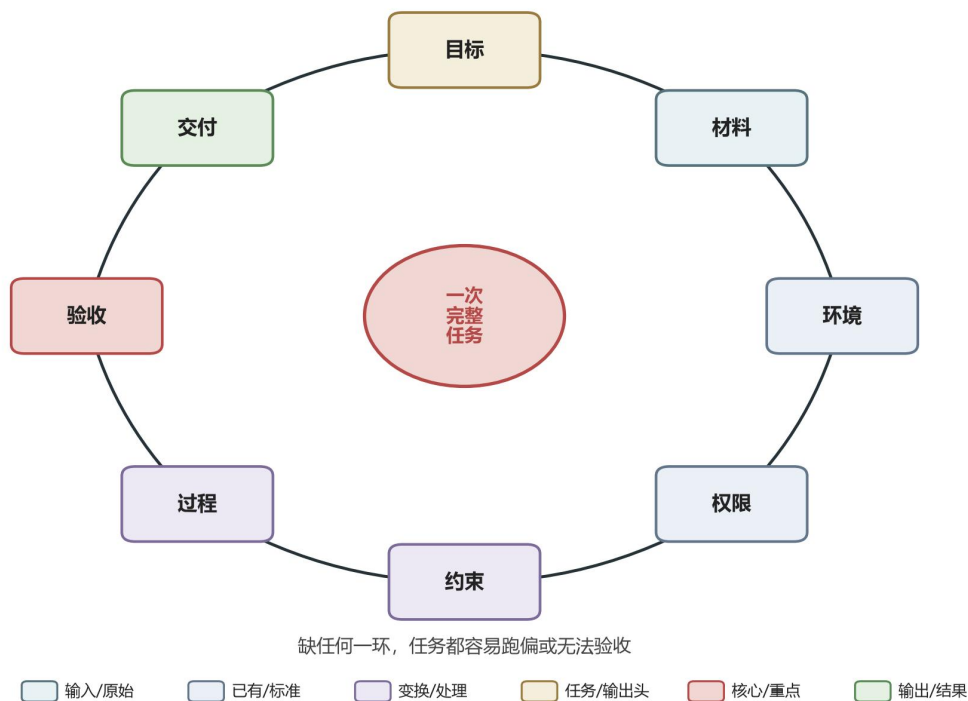


图 4 8 要素任务法结构图（目标/材料/环境/权限/约束/过程/验收/交付）

2.1 本章要解决什么问题

PART 1 你完成了三个小任务。你可能已经发现一件事：同样一个 Agent，有时候做得特别好，有时候完全跑偏。差别不在它那边，在于你发给它的那段话。

本章要解决的就是这个问题：

“为什么我明明说了‘帮我做一下’，它做出来的东西根本不是我要的？我到底该怎么把一件事交代清楚？”

本章给出一套可以背下来的方法——**8 要素任务法**。之后所有 Part 的复杂任务，都套这一个模板。

2.2 你以前可能怎么做

最常见的发任务方式是这样的：

“帮我把这篇论文写了。”

“帮我整理一下这个表。”

“帮我做个 PPT。”

“帮我查查这个公司。”

这些话的共同问题是：你以为你说清楚了，其实你只说了“动词”，没说“对象、标准、边界”。它只能猜。猜中了是运气，猜不中是常态。

2.3 Agent 可以怎么做

当你把一段话改成八句话，它就不需要猜了。它知道：你要什么、用什么做、在哪做、能做什么、不能做什么、按什么顺序做、怎么算完、最后交什么。

这不是让你每次都写小作文。简单任务一两句话就够；但只要这件事超过十分钟、涉及多个文件、做错了要返工，八要素就值。

2.4 需要知道的最少概念：8 要素

这八个词从现在起贯穿全教程：

1. 目标（Goal）：最终要得到什么？
2. 材料（Materials）：可以用什么？
3. 环境（Environment）：在哪里工作？
4. 权限（Permissions）：可以做什么？
5. 约束（Constraints）：什么不能做？
6. 过程（Process）：按什么步骤做？
7. 验收（Acceptance）：怎么证明做完了？
8. 交付（Deliverables）：最后交出什么？

下面逐项拆开讲。

2.5 目标：把“想要”翻译成“可检查的结果”

坏目标：“帮我把这门课学好。”

这种话无法验证——什么叫“学好”？考试考 80 分？还是能给别人讲清楚？

好目标：“在周末之前，帮我把这本教材第三章整理成一份知识结构图，覆盖全部小节，每个小节用一句话说清核心论点。”

区别在于：**好目标里有数量、有范围、有形式**。你读完它，脑子里能想象出成品长什么样。

一个检查办法：把你的目标念给另一个同学听，问他“你觉得做完是什么样”。如果他的想象和你的不一样，目标就没写清楚。

2.6 材料：告诉它“弹药”在哪

坏做法：“帮我写个综述。”它手里什么材料都没有，只能靠训练记忆编。

好做法：“材料在这个文件夹里：[拖进来]，其中 01_政策文件.pdf 是必读，02_访谈记录.docx 是补充。除了这些，你可以联网搜，但必须标出来源。”

材料这一项要写清三件事：材料在哪、哪些是必读、哪些可以不用。**材料给得太少它会编，给得太多它会乱**。一个常见错误是把整个硬盘拖给它，觉得“材料越多越好”——其实它会被无关信息干扰，后面 PART 5 会专门讲“最小充分上下文”。

2.7 环境：圈定活动范围

坏做法：不说。它可能在你电脑上到处翻，或者反过来——你以为它能读你本地文件，其实它根本没接上。

好做法：“你只在 D:\课程\论文项目\ 这个文件夹里工作。需要新文件就建在这个文件夹里。不要动这个文件夹以外的任何东西。”

环境这一项的本质是**画安全圈**。圈定之后，你晚上睡觉也踏实——它再怎么折腾，也在圈里。

2.8 权限：分级授权

PART 1 已经讲过读、写、删、对外发送四级。这里再强调一次：在任务里直接写出来，不要假设它知道。

坏做法：不写权限。默认配置可能太松，也可能太紧。

好做法：

权限：

- 可以读这个文件夹里的所有文件；
- 可以在这个文件夹里新建和修改文件；
- 不要删除任何已有文件；
- 不要往外发任何东西（邮件、消息、公开发帖都不行）；
- 如果需要访问这个文件夹以外的东西，先问我。

2.9 约束：把“绝对不能碰的红线”写出来

约束和权限是一体两面。权限说“你能做什么”，约束说“你不能做什么”。

常见的红线：

- 不得编造文献、作者、DOI、数据；
- 不得覆盖原始文件；
- 不得在没有我的确认下删除任何东西；
- 不得把我的文件上传到你不确定的一方；
- 不得使用我没允许的付费功能；
- 中文写作，不要夹大段英文（除非是术语）。

约束这一项写得越具体，返工越少。尤其是“不得编造”这一条，对学术类任务必须每次都写。

2.10 过程：让它按你要的顺序走

坏做法：不写顺序。它可能一上来就写结论，写了一半发现材料不够。

好做法：“先盘点材料，再建一张表，再按表读，最后根据表写。每做完一步停下来告诉我。”

为什么要写过程？因为**复杂任务最怕它跳步**。规定好顺序，等于给它画了跑道。后面PART 3 会讲怎么用“中途停下来确认”的方式管长任务。

2.11 验收：怎么证明它做完了

这一项最容易被漏掉，但它是本教程的灵魂。坏做法：“做完了告诉我一声。”它当然会告诉你——它一直会告诉你。

好做法：

验收标准：

1. 摘要表里每一条论点，都能在原文里找到出处；
2. 联网资料部分，每条结论后面必须跟一个我能点开的链接；
3. 表格行数必须等于实际文件数，缺一个要说明；
4. 最后单独列一节“没解决的问题”。

验收标准写得越具体，它越不容易“假完成”。记住那句话：**它说完成了，不等于完成了。**

2.12 交付：最后交什么、放在哪

坏做法：“做好了发给我。”它可能把结果直接贴在对话框里，你想用还得复制半天。

好做法：“最终交付三样东西：一份 综述草稿.docx、一份 证据矩阵.xlsx、一份 引用核验表.md，全部放在工作目录的 交付/ 子文件夹里。”

交付这一项的价值在于：**成品和过程分开**。中间草稿留在 工作中/，最后干净的成品进 交付/。三周后你回来找东西，不会乱。

2.13 8 要素实战：六类任务示例

下面六段话可以直接复制改。

一、简单任务（整理一个文件夹）

目标：把“下载”文件夹按文件类型分好类。
材料：D:\Downloads 里的所有文件。
环境：只在这个文件夹里操作。
权限：可读、可新建子文件夹、可移动文件；不要删除任何文件。
约束：原始文件一个不能少；重名文件保留两份，不要合并。
过程：先盘点并给分类方案，我批准后再移动。
验收：整理前后文件总数一致；我随机抽查三个子文件夹内容正确。
交付：一份“分类说明.md”放在文件夹根目录，写清楚每个子文件夹放了什么。

二、学习任务（复习一门课）

目标：在两周内，帮我把“宏观经济学”这门课的全部 PPT 整理成一份可复习的知识地图。
材料：课程文件夹里的 12 个 PPT，以及我的课堂笔记.md。
环境：工作目录就是课程文件夹。
权限：可读，可在“复习/”子文件夹里新建文件。
约束：不要编造 PPT 里没有的概念；看不懂的地方标出来，不要猜。
过程：先逐个 PPT 提取要点，再合并成知识地图，再标出我笔记里没覆盖的知识点。
验收：知识地图覆盖全部 12 次课；每次课至少 5 个要点；每个要点能在原 PPT 里找到。
交付：复习/知识地图.md + 复习/待补清单.md。

三、科研任务（文献综述初稿）

目标：完成一份关于“生成式 AI 与知识劳动”的文献综述初稿（3000 字左右）。
材料：工作目录里的 20 篇论文 PDF，以及我的已有综述草稿.md。
环境：只在本文件夹工作。
权限：可读、可新建文件；不要改原始 PDF 和我的草稿。
约束：绝对不得虚构论文、作者、期刊名、DOI、数据；引用必须来自工作目录里真实存在的 PDF；如果某篇没读，就在正文里标注“待补”。
过程：先做证据矩阵，再梳理研究脉络，再列大纲，我确认大纲后再写正文，最后逐条核验引用。
验收：综述里每个引用都能在证据矩阵里找到对应；最后附“未解决问题清单”。
交付：综述正文.docx + 证据矩阵.xlsx + 引用核验表.md。

四、办公任务（分析一份 Excel 并写报告）

目标：分析一份销售数据 Excel，找出三个最值得关注的现象，写成一页报告。
材料：销售数据_2025.xlsx。

环境：工作目录。

权限：可读 **Excel**，可新建报告文件；不要改原始 **Excel**。

约束：不要编造数字；所有结论必须能从表里算出来；如果数据有缺失，先报告再分析。

过程：先读表结构，再做数据质量检查，再做描述统计，再出两张图，再写结论。

验收：报告里每个数字都能在 **Excel** 里复算；两张图和结论对得上。

交付：分析报告.docx + 两张图.png。

五、数据任务（清洗一份乱表）

目标：把一份乱表清洗成规范表。

材料：原始表.csv。

环境：工作目录。

权限：可读原始表，新建“清洗后表.csv”；不要覆盖原始表。

约束：清洗规则必须先列给我看，我同意后再动手；不要删除任何行，有问题的行标黄。

过程：先列问题清单（空值、重复、格式混乱），再给清洗规则，确认后执行，最后报清洗前后行数对比。

验收：清洗后表行数 = 原始表行数 - 明确剔除的重复行数；每条规则都在规则清单里。

交付：清洗后表.csv + 清洗规则说明.md。

六、编程任务（改一个小功能）

目标：把一个网页示例里的按钮文案从“提交”改成“立即报名”，并让它在手机上也能正常显示。

材料：工作目录里的网页项目文件夹。

环境：只在本项目文件夹里操作。

权限：可读代码、可改代码副本；不要动项目以外的文件；不要联网安装新东西。

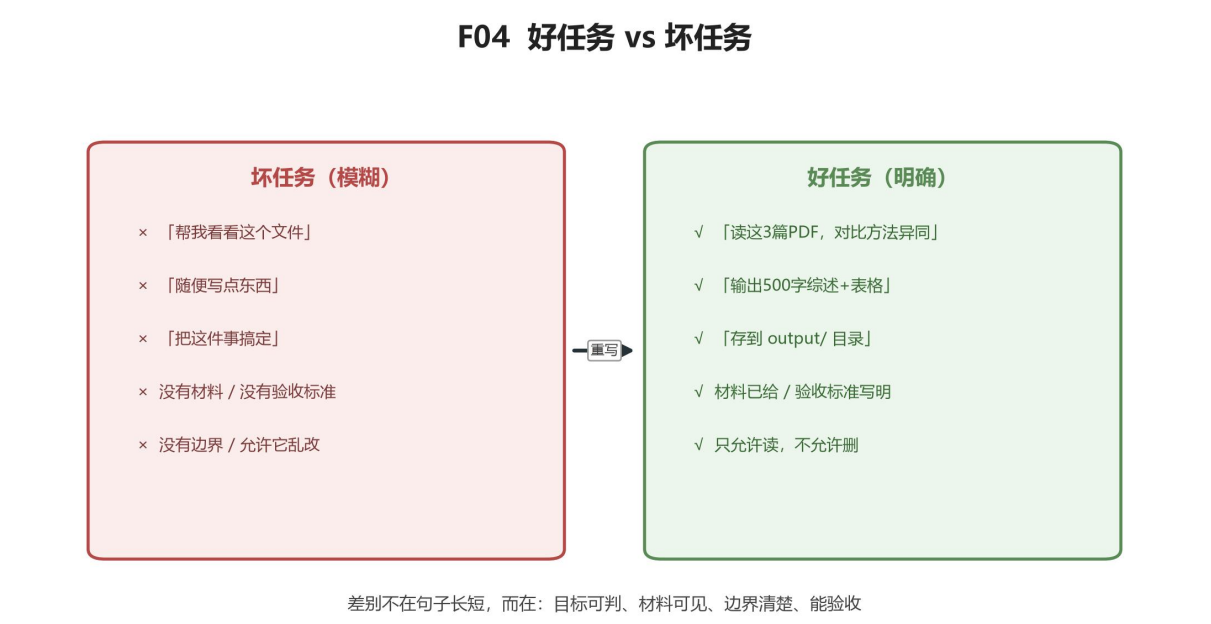
约束：改之前先备份原文件；不要一次改超过 3 个文件；改完要在本地打开看一眼。

过程：先找到按钮在哪个文件，先备份，再改文案，再改手机样式，最后打开浏览器预览。

验收：本地打开网页，按钮文案是新的；手机窗口下按钮不溢出。

交付：改好的文件 + 一份“我改了哪几个文件、改了什么”的说明.md。

2.14 好任务与坏任务对照



#	坏任务	问题在哪	好任务
5	“分析一下这个 Excel。”	没说要什么结论	“这份销售表里，找出销售额最高的 3 个品类和环比下滑最厉害的 2 个，数字必须能复算，写一页报告。”
6	“帮我改改这个网页。”	没说改哪、改成什么样	“把首页右上角按钮文案从'了解更多'改成'立即体验'，只改这一处，改完截图给我看。”
7	“帮我复习一下这门课。”	没法验收	“把这 12 份 PPT 做成知识地图，每份至少 5 个要点，我随机抽 3 个要点要能在原 PPT 里找到。”
8	“帮我翻译一下。”	没说保留什么、格式	“把这段英文摘要翻译成中文，保留原文中的方括号引用格式不动，专业术语首次出现时中英对照。”
9	“帮我写个邮件。”	没说语气、目的、收件人	“写一封给教授的邮件，目的是约下周见面讨论开题，语气恭敬但不啰嗦，中文 200 字内，先出草稿我确认后发。”
10	“帮我做点研究。”	空洞，无法验收	“围绕'远程办公对论文产出的影响'找 6 篇实证研究，做一张证据矩阵，每行含样本、方法、结论，不要综述文章。”
11	“你自己看着办。”	把判断权全交出去	“我拿不准的地方你来提议，但删除、发送、公开发布这三类必须先问我。”
12	“快点做完。”	没说什么叫完	“做完的标准是：生成一份交付文件，我打开后能直接用，不需要你再解释。”

2.15 贯穿案例：两条线怎么用这一章

学习线（课程）：从“帮我学宏观经济”这个模糊愿望出发，套八要素后变成：材料是 12 份 PPT、目标是知识地图、验收是随机抽查能在原文找到。这就是 PART 0 里 Mission L0 那份问题清单的正式落地。

科研线（论文）：从“帮我写综述”出发，套八要素后变成：材料 20 篇 PDF、红线是不许编引用、流程是证据矩阵→脉络→大纲→正文→核验、交付是四件套。这就是后面 PART 3、PART 4、PART 5 所有科研任务的母版。

你会发现，这两条线从这一章开始，后面每一章都在它们上面长东西。

2.16 最简单版本与完整版本

最简单版本：下次发任务前，只加三件事——“工作目录在哪”“不要删原始文件”“做完我怎么检查”。就这三句，已经能挡住一半的返工。

完整版本：把八要素背下来，每次复杂任务都写全。写满二十次之后，你会发现不用刻意背，它会自动出现在你脑子里。

2.17 本章常见失败

失败一：八要素写成了流水账。每一项都写，但写得空泛——比如验收写“要做得好一点”。验收必须是可检查的句子，“做得好”不是验收。

失败二：过程写得太死。你把每一步都规定死，它中间发现问题也不敢调整。正确做法是：大顺序固定，中间允许它“遇到异常先报告”。

失败三：材料一次给太多。你把整个项目文件夹全拖进去，它被无关文件带偏。从今天起，只给本次任务相关的那几个文件。

失败四：写完任务就再也不看了。八要素不是交差用的，是你自己检查任务清不清楚的工具。写完之后自己读一遍，像读别人写的需求一样挑毛病。

2.18 本章模板

复制下面这一整段，每次改括号里的内容：

【目标】

[最终要得到什么，含数量、范围、形式]

【材料】

[文件/网页/已有草稿在哪；哪些必读；哪些可联网补充]

【环境】

[工作目录：不要访问目录以外的位置]

【权限】

[可读/可写/可删/可对外发送分别怎么规定]

【约束】

[绝对不能做的事，尤其是“不得编造”“不得覆盖原件”]

【过程】

[按什么顺序：哪一步停下来等我确认]

【验收】

[我怎么检查—随机抽查、复算、点链接]

【交付】

[最终产出哪几个文件，分别叫什么，放哪个文件夹]

2.19 延伸资源

- 你所用 Agent 产品的“最佳实践/Prompt Guide”官方文档（以 2026 年 9 月为准）。官方文档会根据当前模型能力更新，比第三方教程新。
- 如果你想进一步练这个方法，把上面“好坏对照”的 12 组改成你自己的真实任务，各写一版坏的和好的，对比着看最有感觉。

2.20 完成标准

学完 PART 2，你应该能做到：

- 不看笔记，背出八要素分别是什么；
- 把一个模糊的愿望（“帮我做个 XX”）改写成一段完整的八要素任务；
- 拿到别人写的一段任务，能指出它缺了哪几要素；
- 至少用八要素完成一次真实任务，并在验收环节亲自检查过。

PART 3：怎样管理长任务

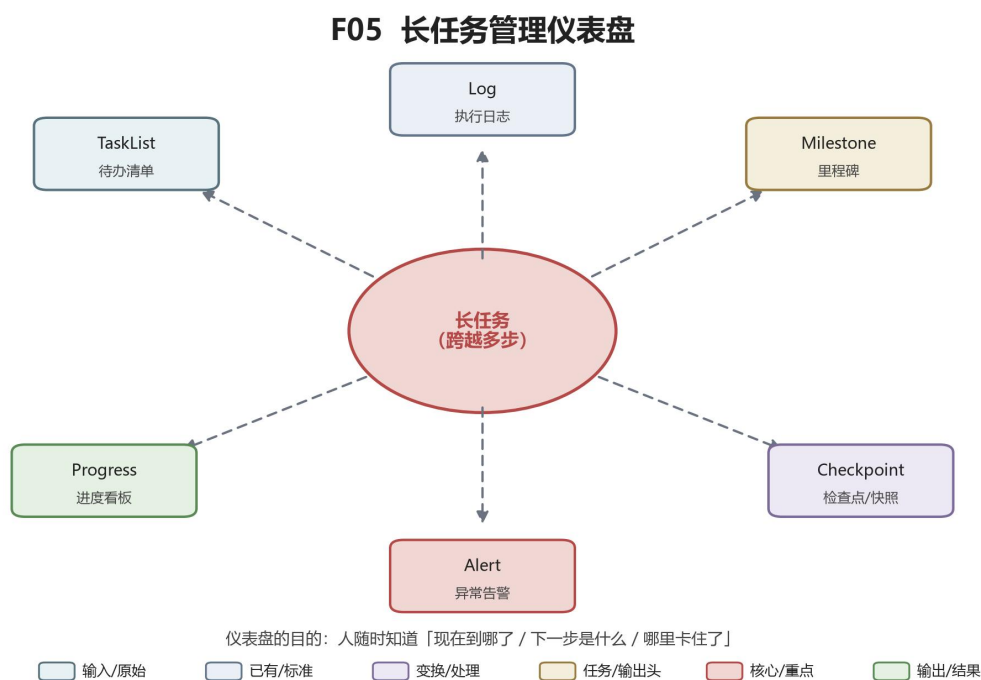


图 6 长任务时间轴（Plan → Milestone → Checkpoint → 中间成果 → 交付）

3.1 本章要解决什么问题

前面的任务都不大：整理一个文件夹、读五份 PDF、查一个主题。但真实工作往往不是这样的。

你下学期要交的课程论文，可能要读 30 篇文献、写一个月；你要做的创业比赛项目，可能要调研、做表、写报告、做 PPT、改五版。这些事有一个共同名字——**长任务**。

长任务一旦交给 Agent，你会遇到新问题：

- 它做了一半，你不知道它进行到哪了；
- 它做了三天，突然说“我忘了最开始要什么”；
- 中途你想加一份新材料，不知道怎么接；
- 它做出来一堆中间文件，你分不清哪个是最新的；
- 它跑着跑着开始重复劳动，同一件事做了两遍。

本章要解决的就是：从“问一次”进入“项目管理”。你不需要会项目管理理论，只需要学会五个朴素的动作。

3.2 你以前可能怎么做

旧做法里，长任务全靠你的脑子记：一张便签贴在电脑边，做完一件划一件；微信文件传输助手里翻来翻去；Word 文件命名是 论文_final.docx、论文_final2.docx、论文_final_真的最终.docx。

这种方式在任务小的时候还行，一旦超过两周、超过十个文件，必然乱。

3.3 Agent 可以怎么做

让 Agent 帮你管理进度：让它在项目文件夹里维护一份“进度文件”，每完成一步就更新；让它把大任务拆成清单，做完一项划一项；让它在关键节点停下来给你看中间成果。你不需要自己记，你只需要看它那份进度文件。

3.4 为什么复杂任务不能一步完成

不是 Agent 不够聪明，而是三个客观限制：

第一，它一次能“看到”的内容有限。 它的工作记忆（Context，上下文——指它在当前这一轮对话里能同时记住的文字量）就像一张桌子，面积有限。你一次性把 30 篇论文全堆上去，它根本摊不开。

第二，任务本身有先后依赖。 没读完文献就写综述，写出来也是编的；没定大纲就做 PPT，做到一半发现结构不对。

第三，错得越早，改得越便宜。 如果它一口气写了八千字你才看到，发现方向错了，那八千字全废；如果它写五百字就停下来给你看，你一句话就能拉回来。

所以长任务的核心原则不是“让它一口气做完”，而是“拆小、分步、勤看”。

3.5 需要知道的最少概念

计划（Plan）：一张任务清单，把大任务拆成 5 到 10 步，每步一句话。

里程碑（Milestone）：清单上几个“必须停下来给你看”的关键节点。比如“文献读完了”“大纲定了”“初稿写完了”。里程碑是用来验收的，不是用来记录流水账的。

检查点（Checkpoint）：比里程碑更密的停顿。里程碑是“大节点”，检查点是“每走几步停一下”。比如读完 10 篇停一次，让你抽查质量。

中间成果（Intermediate Output）：过程中产出的半成品文件。这些东西不能丢——它们既是进度记录，也是出问题时的“回滚点”。

进度记录文件：一份文本文件，专门用来写“现在做到哪了、下一步做什么、有什么没解决”。下一节详讲。

3.6 Plan 与 Task List

不要自己拍脑袋列计划。正确做法是让它先列，你再改。

发这段话：

我要做的事是：[一句话目标]。
材料已经放在 [文件夹]。
请你不要开始做。先给我一份计划：
1. 把这件事拆成 6 到 10 个步骤；
2. 每一步写清楚：这一步要产出什么；
3. 标出哪几步必须停下来等我确认；
4. 标出哪几步依赖前一步的结果。
计划发给我，我修改后我们再开始。

为什么要让它先列？因为它列计划的过程本身就是暴露问题的过程。如果它列出来的步骤里有“第三步：写综述”，而第二步还没有“读文献”，你一眼就能看出漏了。

3.7 Milestone 里程碑

一条科研文献综述任务的里程碑可能是这样：

M1 文献盘点完成（你确认哪些必读）
M2 证据矩阵填完（你抽查 5 篇）
M3 大纲确定（你点头它才写正文）
M4 初稿完成（你通读）
M5 引用核验完（逐条对原文）

注意：每个里程碑都对应一个“你介入”的动作。没有你介入的节点，不叫里程碑，叫它自言自语。

3.8 Checkpoint 检查点

里程碑之间还可以设更密的检查点。比如读 30 篇文献这件事，不要等 30 篇全读完才看，而是：

- 读完 10 篇 → 停一下，把这 10 篇的摘要给你看一眼；
- 读完 20 篇 → 再停一次；
- 30 篇 → 里程碑 M1。

这样如果它从第 3 篇就读偏了，你在第 10 篇就能发现，不用等到第 30 篇。

3.9 长任务中的中间成果

中间成果要单独放一个文件夹，比如 工作中/。它每完成一步就把那一步的产物存进去，不要只留在对话里。为什么？因为：

- 对话会被新的对话冲掉，文件不会；
- 中间成果是你检查的实物；
- 出问题时，你可以指着某个中间文件说“就改这里，别动别的”。

一个典型的科研任务中间产物序列：

```
工作中/  
01_文献盘点.md  
02_证据矩阵.xlsx  
03_研究脉络.md  
04_大纲.md  
05_初稿_v1.docx  
06_初稿_v2.docx  
07_引用核验表.md
```

看到这个序列，哪怕你隔了一周再回来，也能立刻知道它做到哪了。

3.10 进度记录文件：不要求你懂 Markdown

进度记录文件本质上就是一份“记事本”，用普通写字板写就行，不需要学任何语法。它的作用只有一个：隔了三天再回来，不用翻聊天记录也能知道现在什么状态。

常见的几种，各管一摊：

- **任务清单**（常命名为 `TASKS.md`）：一张待办表，做完一项划掉一项。
- **进度记录**（常命名为 `PROGRESS.md`）：按日期写流水账——“9 月 18 日读完第 1-10 篇；9 月 19 日建完证据矩阵；9 月 20 日发现 XX 文献与主题无关，剔除。”
- **说明文件**（常命名为 `README`）：这个文件夹里都有什么、每个文件是干嘛的。新进来的人（包括三天后的你自己）先读它。
- **变更记录**（常命名为 `CHANGELOG`）：每次改了什么。“v1 → v2：把结论从‘影响显著’改成‘影响有限’，因为第 14 篇文献证据相反。”
- **状态文件**：一句话写当前状态——“当前在做：初稿第三版；下一步：引用核验；阻塞：第 22 篇 PDF 读不出。”

你不需要全用。新手只要求两个：一个 `PROGRESS.md`（按日记流水），一个 `README`（说明文件夹结构）。够用了。

本教程为这几份文件准备了带真实示例的模板，放在 `examples/` 目录下，打开照填即可，不要从空白开始写：

- `examples/WORKLOG.md`：操作流水账模板（日期 / 操作 / 涉及文件 / 结果 / 备注），4.2 节讲执行证据时还会用到；
- `examples/TASKS.md`：任务清单模板，带四种状态标记（未开始 / 进行中 / 已完成 / 卡住）；
- `examples/PROGRESS.md`：按日期的进度流水模板；
- `examples/DECISIONS.md`：重要决策记录模板，专记“为什么这么定”。

每个模板里都附了一份“迷你文献调研”任务的真实填写示例，照着那个例子改成本次任务的内容就行。

给它发这句话就行：

请在工作目录里维护一份 `PROGRESS.md`。
每次完成一个小步骤，就往里面追加一行：日期 + 做了什么 + 下一步是什么。
不要覆盖以前的记录，只往下加。

3.11 中途修改需求

长任务几乎一定会改需求。正确做法是：不要直接在旧任务上接着说“不对，改成 XX”，而是明确说：

任务变更：
原计划第三步是写大纲，现在改为：大纲写完后，先加一步“找一份最新文献补进证据矩阵”，再开始写正文。
前面已经做的部分（01、02）保留，不用重做。
请你更新 `TASKS.md` 和 `PROGRESS.md`，告诉我现在的新计划。

为什么要这么说？因为它不会自动记住你之前的全部安排。你说清楚“哪些保留、哪些改、新顺序是什么”，它才不会把已经做完的推倒重来。

3.12 新材料加入后怎么办

中途你找到了一篇重要新文献，或者补了一份数据。不要直接甩给它说“加进去”。这样说：

新材料：我刚放进工作目录的 `new_paper.pdf`，这是一篇 2026 年最新文献，和我们的主题直接相关。
请你：
1. 单独读这一篇，填进证据矩阵；
2. 判断它对现有大纲有没有影响——如果它支持现有结论，加一条引用；如果它反驳现有结论，标出来，我们再讨论要不要改大纲；
3. 不要重写已经写完的部分，除非我明确说改。

关键在第三条——新材料只增量处理，不要触发它重写一切。

3.13 如何避免重复劳动

Agent 最容易犯的错之一，是它忘了自己已经做过，又做一遍。防止的办法是：

- 每次开工前，让它先读 `PROGRESS.md` 和 工作中/ 里的已有文件，然后说一句：“基于已有进度，下一步是什么？”
- 在任务里写：“如果某个中间成果已经存在，先读它，不要重做。”

把这句话养成习惯，能省下大量时间。

3.14 如何从上次结果继续

隔了一周回来，新开一个对话。不要指望它记得你上次聊了什么。直接发：

我们上次在做 [项目名]，工作目录是 [路径]。
请你先读 `PROGRESS.md`、`README.md`，以及“工作中/”里的最新文件，然后告诉我：
1. 我们上次做到哪了；
2. 下一步是什么；
3. 有没有卡住的问题。
你说清楚之后，我再决定今天怎么推进。

这就是为什么 PART 3 前面反复强调“进度文件必须写”——它是你和未来的自己、和未来的新对话之间的桥。没有它，每次都得从头讲一遍。

3.15 中断后如何续跑：WORKLOG 与 TASKS.md 实战

3.14 讲的是“隔了几天、你主动回来”的场景。这一节讲更难受的一种：任务做到一半，突然断了——网络抖了一下、平台会话过期、你误关了窗口、它跑着跑着报错停住了。这时最容易慌的是两件事：已经做完的会不会白做？下一步到底该从哪儿接着走？

答案还是那两份文件：`WORKLOG.md` 告诉你它“实际做到了哪一步、在哪一行断的”，`TASKS.md` 告诉你“哪些勾掉了、哪个还没勾”。这两份模板本教程已经放在 `examples/` 目录下，打开就能照着填：

- `examples/WORKLOG.md`：日期 / 操作 / 涉及文件 / 结果 / 备注，已经带一份真实记录示例；
- `examples/TASKS.md`：任务清单 + 状态（未开始 / 进行中 / 已完成 / 卡住），也带示例；
- 另外还有 `examples/PROGRESS.md`（按日流水）和 `examples/DECISIONS.md`（重要决策），长任务一并用上。

第一步：先别慌，别让它从头再来。

中断后第一件事不是把任务重发一遍。重发 = 它把已经做过的步骤再做一遍，浪费时间，还可能把已经生成的中间文件覆盖掉。先做三件事：

1. 打开工作目录，看 工作中/ 里已经有哪些文件、哪个是最新的；
2. 打开 WORKLOG.md，看它最后一行记的是哪一步——那就是断点；
3. 打开 TASKS.md，看最后一个没勾掉的是哪一项——那就是下一步。

第二步：把断点喂给一个新对话。

新开一个对话，不要指望旧对话还在。直接发这段话：

我们的任务中断了，工作目录是 [路径]。
请你先读这三份文件，不要开始做任何新动作：
1. WORKLOG.md — 它会告诉你之前实际做了哪些步骤、最后停在哪一步；
2. TASKS.md — 它会告诉你哪些任务已完成、哪些还没做；
3. 工作中/ 里最新的那份中间文件。
读完之后告诉我：
- 上次精确断在哪一步（引用 WORKLOG 最后一行）；
- 下一步应该做什么；
- 有没有卡住或待我决定的事。
你说完，我确认后我们再继续。

这一步的关键是：让它先读文件、先汇报，不要让它一上来就接着做。因为中断后它可能已经“忘了”当时为什么停下，先让它把断点复述给你听，你确认它没读错、没理解偏，再继续。

第三步：让它从断点续跑，而不是重来。

你确认断点后，这样下指令：

断点确认无误：你上次做到了 [WORKLOG 最后一行那一步]，
对应 TASKS.md 里第 [N] 项（状态还是 [] 未开始 / [~] 进行中）。
请你：
1. 不要重做已经在 TASKS.md 里打勾的步骤；
2. 从第 [N] 项继续，做完一项就把 TASKS.md 的 [] 改成 [x]；
3. 每做一步，继续往 WORKLOG.md 追加一行；
4. 如果发现之前某个中间文件是错的，先告诉我，不要直接覆盖。

第四步：用 TASKS.md 把“没做完的”显式标出来。

中断最容易留下的隐患，是“它以为做完了、其实没做完”。续跑前，务必让它把 TASKS.md 过一遍：凡是 [] 和 [!] 的项，就是这次续跑要处理的；卡住的 [!] 项，要让它说清楚卡在哪。模板里已经把状态约定写好了：

[] 未开始 [~] 进行中 [x] 已完成 [!] 卡住

为什么这套能续上？因为 WORKLOG.md 是“它的手”做过的事（执行证据），TASKS.md 是“事”还剩什么没做（进度证据），PROGRESS.md 是“为什么这么走”（决策与上下文）。三份合起来，等于把一个会断的长任务，拆成了一串不会断的“实物锚点”。对话可以断，文件不会断。

常见失败：

- 中断后你直接把原任务重发一遍，结果它把已做完的又做了一遍，还覆盖了你的中间文件——这就是没读断点文件的代价；
- WORKLOG.md 是空的或最后停在很早之前——说明它中途没持续记录，这种情况你只能凭工作中/里的文件反推做到哪了，下次记得在任务里就要求“每步追加日志”；
- 续跑后它自作主张重做了已勾掉的步骤——在续跑指令里加上“不要重做已打勾项”。

3.16 多阶段工作流：一条完整的流水线

F06 多阶段工作流：文献分析八步流水线



图 7 多阶段工作流示意

把本章所有零件拼起来，一条典型的科研/调研长任务流水线是这样：

搜索（找文献）

↓ 检查点 1：看搜回来的清单够不够

筛选（按标题摘要剔除不相关的）

↓ 检查点 2：看筛掉的有没有误伤

阅读（逐篇读，填证据矩阵）

↓ 里程碑 M1：你抽查 5 篇

建表（证据矩阵成表）

↓ 里程碑 M2：你审表

分析（梳理研究脉络、找争议点）

↓ 检查点 3：脉络对不对

写作（先大纲，再分段写）

↓ 里程碑 M3: 大纲你点头
审查（它自己挑毛病 + 引用核验）
↓ 里程碑 M4: 引用逐条对原文
修改（按审查意见改）
↓ 交付

注意这条流水线里，**每两到三步就有一个“人介入”的点**。这就是长任务管理的全部秘密：不是让它跑到底，而是让它跑一段、停一段、给你看一段。

3.17 Mission：跑一条完整的两阶段长任务

名称：用一条流水线完成一次“迷你文献调研”。

难度：L5（长任务，含中间检查）。

目标：用本章方法，完整跑完“搜索→筛选→阅读→建表→小结”五个阶段，中途至少停两次。

输入材料：一个你真正感兴趣的小主题（比如“短视频对大学生注意力的影响”），不需要你自己有文献。

任务说明：

主题：[你的主题]

工作目录：[新建文件夹]

计划：请把这个主题拆成五阶段：搜索、筛选、阅读、建表、小结。

在搜索完成后停一次，把候选清单给我看；

在筛选完成后停一次，把最终入选清单给我确认；

然后再开始阅读和建表。

全程在 **PROGRESS.md** 里记录进度。

所有结论必须有来源链接，不许编。

操作步骤：

1. 发计划，让它先拆阶段，你确认。
2. 它跑完搜索，停下来——你审候选清单，删掉明显不相关的。
3. 它跑完筛选，再停——你确认最终入选。
4. 它读完、建表，出小结。
5. 你打开工作目录，看 **PROGRESS.md** 是不是真的按日期记了进度，看证据表里的链接能不能点开。

风险提示：这是你第一次管理“会停下来”的任务。忍住不要催它“快点做完”。它停下来的每一次，都是在帮你少返工。

成功标准：

- 中途至少停了两次，每次你都给了反馈；
- PROGRESS.md 里有按日期的记录；
- 证据表里每条都有真实链接；
- 隔一天新开对话，你能用“读进度文件继续”的方式接上。

失败案例：

- 它一口气跑完五阶段才出来——因为你忘了写“在搜索后停一次”。下次记得在任务里明确写出停顿点。
- 它跑完后 PROGRESS.md 是空的——你忘了要求它维护进度文件。
- 中途你想加一份新文献，直接甩给它，结果它把前面的表重做了一遍——你应该用 3.12 的方式“增量加入”。

验证方式：随机挑证据表里的一条，点链接、读原文，看它写的结论是不是原文真说的。

3.18 最简单版本与完整版本

最简单版本：下次任何超过三步的任务，只加一句话：“先列计划给我看，我确认后再做；中途每完成一步，往 PROGRESS.md 里记一行。”就这两句，长任务的轮廓就有了。

完整版本：按 3.15 的流水线，跑一次完整的五阶段任务，维护 TASKS.md、PROGRESS.md、工作中/ 三层结构。跑过一次，你就理解了什么叫“把任务当项目管”。

3.19 本章常见失败

失败一：让它一口气跑完再看。这是长任务最大的失败模式。它跑得越远，错得越偏，返工越贵。解决方案就是本章反复讲的“拆小、分步、勤看”。

失败二：没有进度文件。对话一清空，一切归零。从第一个长任务开始，就建 PROGRESS.md。

失败三：中途改需求不说清楚。一句“不对，改一下”，它可能理解成全部重来。永远说清“哪些保留、哪些改、新顺序是什么”。

失败四：版本混乱。初稿.docx、初稿_new.docx、初稿_最终.docx 满天飞。正确做法是用编号 初稿_v1.docx、初稿_v2.docx，旧版本留着不删，CHANGELOG 里记每版改了什么。

3.20 本章模板

给任何超过三步的长任务，照这个模板发：

目标：[一句话最终结果]

工作目录：[路径]

计划：

请先把这件事拆成 **6-10** 步，标出哪几步停下来等我确认。

在你开始之前，先把计划发给我。

过程要求：

- 每完成一步，往 **PROGRESS.md** 追加一行（日期 + 做了什么 + 下一步）；
- 如果已有中间成果，先读它，不要重做；
- 遇到读不出来的材料、矛盾的结论、不确定的地方，先报告，不要猜。

里程碑：

M1 [第一个你要验收的节点]

M2 [第二个]

M3 [第三个]

验收：[怎么证明做完]

交付：[最终文件清单]

3.21 延伸资源

- 项目管理里“待办清单、里程碑、日志”这套做法，你不需要读任何项目管理教材。搜索英文关键词 **simple task list for long projects**，看几个通用建议即可，核心就是本章这几招。
- 如果你所用 Agent 产品自带“任务/计划/待办”界面功能（以 2026 年 9 月界面为准，名称可能变化），优先用它自带的，再辅以 **PROGRESS.md** 文件。

3.22 完成标准

学完 PART 3，你应该能做到：

- 解释为什么复杂任务不能一步做完；
- 给一个长任务拆出至少 3 个里程碑，并说出每个里程碑你要验收什么；
- 在一个真实项目文件夹里维护 **PROGRESS.md**，按日期追加；
- 隔了两天之后，能靠读进度文件接上之前的工作，而不是从头再来；
- 完成 3.17 那条五阶段长任务，中途至少停两次。

到这里，PART 0 到 PART 3 结束。你已经完成了从“会聊天”到“会交代任务、会管过程”的基础跃迁。接下来的 PART 4 会专门解决一个关键问题：它说做完了，你怎么知道它真的做完了？

PART 4：怎样判断 Agent 是否真的做对了

F07 验证三层证据模型（金字塔）

「它说做完了」不算证据；三层都站得住才算完成

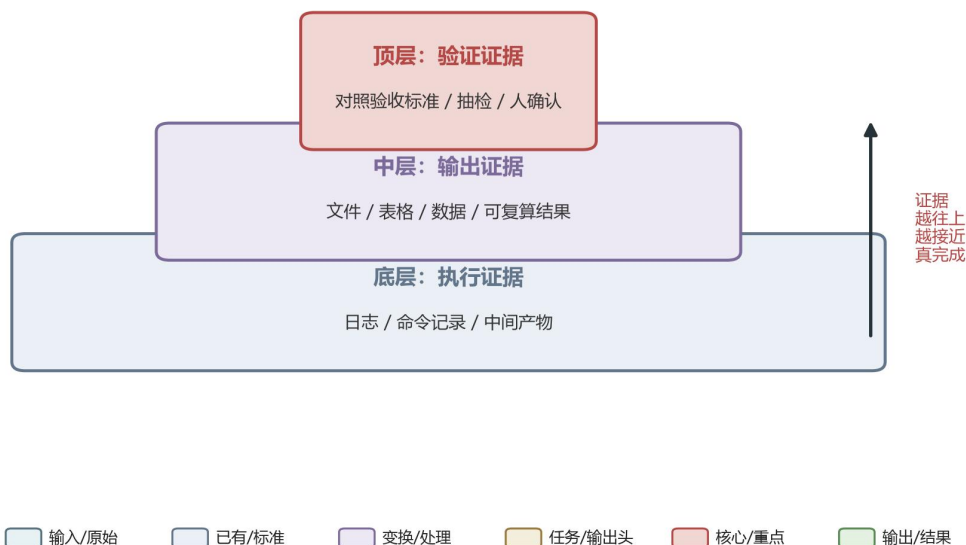


图 8 验证三层证据模型

本 Part 是全书最重要的质量控制模块。前面三个 Part 教你“怎么把任务交给 Agent、怎么管长任务”，本 Part 教你一件更要紧的事：**Agent 说它做完了，你凭什么信它？**

4.1 “Agent 说完成了”不等于完成

本章要解决什么问题

你让 Agent“读这 20 篇论文，做一张证据矩阵”。两小时后它回你：“已完成，文件保存在 output/evidence_matrix.xlsx，共 20 行，覆盖全部论文。”

你打开文件，确实有 20 行。然后你随便挑一篇论文，对照原文——发现 Agent 把一篇 2023 年的论文写成了 2018 年，把另一位作者的观点安到了这篇论文头上，甚至有一篇论文的 DOI 根本打不开。

Agent 没有骗你，它真的“以为”自己做完了。这就是本 Part 要解决的核心问题：如何不被“完成声明”骗过，如何亲手确认结果真的对。

旧做法

在没有 Agent 的时代，你判断一个工作是否完成，靠的是：

- 自己读了多少、写了多少；
- 导师或同事当面指出问题；
- 最后自己通读一遍。

人对自己的工作有“体感”——你知道自己昨天那篇文献综述改了几遍、哪一段是硬凑的。Agent 没有这种体感，它不会“心虚”。它生成一个看起来很完整的文件，然后用最自信的语气告诉你“完成了”。

Agent 做法

把“完成”这件事拆成三层证据，逐层核对：

执行证据：Agent 实际做了什么操作？它真的打开了那 20 个文件吗？
结果证据：产出物本身存在吗？打开看了吗？内容对得上吗？
验证证据：有没有独立手段证明结果是对的？（交叉核对、重新算一遍、让另一个模型审）

Agent 说“完成了”，只能证明它走到了流程的某一步，不能证明结果正确。这就是本 Part 反复要建立的认知：

任务完成声明只是状态信息，不是完成证据。

真正完成 = 执行证据 + 结果证据 + 验证证据。

最少概念

- **完成声明（Completion Claim）**：Agent 在回复里写的“已完成”“已保存”“已修复”。它是一句话，不是证据。
- **证据（Evidence）**：你可以独立检查的客观痕迹——文件、日志、运行输出、链接、截图、数字。
- **假完成（False Positive）**：Agent 声称完成但实际没完成，或完成得不对。5.6 节会专门讲。

Mission M13：要求 Agent 证明它完成了

字段	内容
Mission 名称	要求 Agent 证明它完成了
目标	让 Agent 在完成任务后，不只说“做完了”，而是给出三层证据
难度	L3

字段	内容
预计需要的 Agent 能力	文件读写、代码执行、联网搜索
输入材料	任意一个你刚交给 Agent 的任务（例如“读 5 份 PDF 做摘要”）
任务说明	在 Agent 声称完成后，不要直接交付，而是追加一句：“在交付前，请你回答：1）你实际打开了哪几个文件，逐个列出文件名；2）产出文件的路径和行数；3）你做过哪些自检。任何一项拿不出证据，就不要说完成。”
操作步骤	1）正常布置任务；2）Agent 回复“完成”后不验收，发出上面的追问；3）检查它列出的文件清单是否真的存在；4）抽查 1-2 个结果点
风险提示	Agent 可能会“编”一个看起来合理的文件清单来应对追问，所以第 3 步必须由你亲自打开文件夹核对
成功标准	你能在文件夹里找到它说的每一个文件；抽查点全部对得上原文
失败案例	Agent 列了 5 个文件，实际只生成了 2 个；或路径写错了一层目录
验证方式	亲自打开产出文件，对照原始材料抽查
进阶挑战	把“三层证据”写进你的任务模板，以后每个任务都要求 Agent 主动附上

最简单版本

一句话口诀：“说完成不算数，拿出来给我看。”每当 Agent 说“做完了”，你就问三个问题：

1. 你动了哪些文件？
2. 产物在哪？
3. 你怎么知道是对的？

完整版本

把这三层要求写进每次任务的验收条款（8 要素任务法里的第 7 项“验收”）：

验收要求：

1. 执行证据：列出你实际读取/修改/创建的每个文件路径；
2. 结果证据：产出文件必须真实存在于指定目录，并报告行数/字数/条数；
3. 验证证据：至少抽查 3 处结果，说明你是怎么核对的；
4. 任何一项无法提供证据，视为未完成，不得声称完成。

常见失败

1. **Agent** 只给口头总结，不给文件路径。你无法定位产物，自然无法验证。
2. **Agent** 给了路径，但路径是错的或过时的。文件在别的地方，或根本没保存成功。
3. **Agent** 把“我生成了这段文字”当成“我完成了任务”。它写了一段综述文字贴在对话框里，但没存成文件，也没核对引用。

如何验证

不要相信 **Agent** 的自我感觉，相信你自己的眼睛：

- 文件是否真的在那个路径下；
- 打开文件，内容是不是你要的；
- 抽 1-2 个具体事实点，回到原始材料核对。

本章模板

【交付前自查】

在声称完成之前，请确认并报告：

1. 我实际操作了哪些文件（逐个列路径）；
 2. 产出物在哪、规模多大（行数/条数/字数）；
 3. 我做过哪些验证、验证结果如何；
 4. 还有哪些不确定或未完成的地方。
- 凡是第 4 条有内容的任务，都不得标记为“完成”。

延伸资源

- Anthropic 官方文档中关于“Task completion”与“Verification”的说明（以官方文档站最新版为准，核验日期 2026-09-18）。
- 软件工程领域的“Done Definition”（完成定义）概念：一个任务不算完成，除非有可运行、可测试、可交付的产物。

完成标准

你能在任何 **Agent** 任务结束时，本能地问出“证据呢”，而不是直接相信“已完成”三个字。

4.2 执行证据：Agent 实际做了什么操作的记录

本章要解决什么问题

你让 **Agent**“把这 30 个文件分类整理到文件夹里”。它说整理完了。你怎么知道它真的读了 30 个文件，而不是只打开了前 5 个就开始凭印象写总结？

执行证据就是回答这个问题：把 **Agent** 的“操作流水账”留下来。

旧做法

你自己整理文件夹时，你知道自己打开过哪些文件、哪几个打不开、哪几个是乱码。这种记忆是天然的执行证据。Agent 没有这种记忆，它做完就“忘了”自己刚才干了什么，所以必须显式要求它留下记录。

Agent 做法

让 Agent 在工作过程中持续记录“我现在读到哪了、遇到了什么问题”。常见的执行证据形式：

- **操作日志（Log）**：每一步做了什么，时间戳、文件路径、结果；
- **中间文件**：边做边存，而不是最后一次性生成；
- **执行轨迹（Trajectory）**：Agent 平台通常会自动记录它调用了哪些工具、传了什么参数、返回了什么。你可以回看这条轨迹。

最少概念

- **执行轨迹（Trajectory）**：Agent 从开始到结束，每一次“想做什么→调用工具→工具返回什么”的完整记录。这是最硬的执行证据。
- **操作日志（Log）**：你主动要求 Agent 写下来的工作日志，例如 `WORKLOG.md`。

Mission M14（前半）：让 Agent 边做边记录

字段	内容
Mission 名称	让 Agent 边做边记录操作日志
目标	任务结束后你有一份完整的操作流水账
难度	L2
预计需要的 Agent 能力	文件读写
输入材料	任意多文件任务
任务说明	在任务开头加一句：“请在工作目录维护一个 <code>WORKLOG.md</code> ，每完成一步就追加一行：时间、动作、涉及文件、结果。”
操作步骤	1) 布置任务时附上这句话；2) 任务结束后打开 <code>WORKLOG.md</code> ；3) 对照你预期的步骤数，检查它真的做了这么多步
风险提示	Agent 可能补写日志（事后编造），所以要它“每完成一步就追加”，而不是“最后统一写”
成功标准	<code>WORKLOG.md</code> 里的步骤数与任务复杂度匹配，且每步都有具体文件名

字段	内容
失败案例	日志只有三行“已读取、已分析、已完成”，没有任何文件名，等于没记录
验证方式	抽日志里某一行，打开对应文件确认它确实被处理过
进阶挑战	让 Agent 在日志里标注每一步的“问题/异常”，而不是只记成功步骤

最简单版本

一句话：“边做边记，别最后补。”

完整版本

工作记录要求：

- 每完成一个子任务，立即在 `WORKLOG.md` 追加一行；
- 格式：[时间] [动作] [涉及文件/URL] [结果/异常]；
- 遇到打不开的文件、找不到的网页、报错，必须原样记录，不得跳过；
- 任务结束时，`WORKLOG.md` 的行数应与子任务数大致匹配。

常见失败

1. 事后补日志：Agent 最后一次性写一个“看起来合理”的日志，细节模糊。
2. 只记成功，不记失败：打开失败的文件、404 的网页它不提，导致你以为全部成功。
3. 日志和实际操作对不上：日志说读了 A 文件，实际 A 文件没动过。

如何验证

抽 2-3 条日志记录，亲自打开对应文件或链接，确认确实发生过。

本章模板

请维护 `WORKLOG.md`：

- 每步追加，不事后补写；
- 记录所有失败和异常，不隐藏；
- 任务结束时附上日志全文。

延伸资源

- 主流 Coding Agent（如 Claude Code、Cursor、Codex）都自带执行轨迹查看功能，学会在你的平台里找到“对话/操作历史”面板。

完成标准

任务结束后，你能仅凭一份日志，复盘 Agent 到底做了什么、在哪一步出过问题。

4.3 输出证据：产出的文件/数据/结果本身

本章要解决什么问题

Agent 说“文献综述已经写好了”。你怎么确认它不是在对话框里生成了一段话、说完就没了？

输出证据就是：产物必须以“独立存在、可打开、可引用”的形式落在你的工作区里。

旧做法

学生交论文，是把 Word 文件交给老师。文件本身就是输出证据。Agent 时代最容易犯的错，就是把对话框里的一段文字当成“交付”——那段文字刷新就没了，无法审阅、无法对比、无法存档。

Agent 做法

明确要求 Agent：

- 把结果**保存成文件**，而不是只贴在对话框里；
- 文件放在你指定的**输出目录**；
- 报告每个文件的**路径、行数/字数、生成时间**；
- 多张产出物之间**互相对应**（正文里引用的表格，必须真的存在于同目录）。

最少概念

- **输出目录（Output Directory）**：专门放产物的文件夹，和原始材料分开。原始文件不动，所有新东西都进 output/。
- **产物清单（Manifest）**：Agent 自己列的“我这次交了哪些文件”。

Mission

字段	内容
Mission 名称	让 Agent 交付一份产物清单
目标	任务结束时，你拿到一张“交付物清单”，且清单上每个文件都真实存在
难度	L1
输入材料	任意任务

字段	内容
任务说明	在任务里写明：“所有产出保存到 output/ 目录。结束时，在对话框里列出交付清单：文件名、路径、行数/字数、一句话内容说明。”
操作步骤	1) 布置任务时指定输出目录；2) 结束时拿到清单；3) 打开 output/ 文件夹，逐一点开清单上的文件
风险提示	Agent 可能把“临时文件”“缓存文件”也列进交付清单，注意区分
成功标准	清单上每一个文件都能在 output/ 里找到，且内容符合一句话说明
失败案例	清单写了 evidence_matrix.xlsx ，文件夹里只有 evidence_matrix.csv ；或文件是空的
验证方式	双击打开每个文件，检查大小不是 0 字节、内容不是占位文字
进阶挑战	要求 Agent 在清单末尾加一行“未完成项”，把它没做的事也列出来

最简单版本

“东西要落盘，光说不算。”

完整版本

交付要求：

- 所有产物保存到 **./output/**，不要写在别的地方；
- 原始材料只读，不得修改或删除；
- 结束时输出交付清单：

| 文件名 | 路径 | 规模(行/字) | 内容说明 |
- 凡是清单上没有的，不算交付；凡是清单上有但文件夹里找不到的，视为未完成。

常见失败

1. 对话框交付：

一大段文字贴在聊天里，没存文件。
2. 路径漂移：

Agent 把文件存到了它自己的临时目录，你找不到。
3. 空文件/占位文件：

文件名对了，打开是“待补充”或 0 字节。
4. 引用悬空：

正文写“见表 3”，但表格根本没生成。

如何验证

- 用文件管理器打开 **output/**，按修改时间排序，看新文件是不是 **Agent** 刚生成的；
- 逐个打开，不看大小，看内容。

本章模板

输出要求：

1. 产物全部进入 `./output/`;
2. 结束时给我一张交付清单表;
3. 清单外的东西不要说交付。

延伸资源

- 任何现代 Agent 平台都有“工作区文件树”面板，养成每次任务结束扫一眼文件树的习惯。

完成标准

你不再把对话框里的文字当作交付，只认 `output/` 里真实存在的文件。

4.4 验证证据：如何证明结果是对的

本章要解决什么问题

前两层证据解决了“它真的做了、东西真的存在”。但还有最后一层：这个东西本身对不对？

Agent 可能老老实实打开了 20 篇论文、认认真真写了一份综述——但它把 A 作者的结论安到了 B 作者头上。执行证据齐全、输出证据齐全，结果还是错的。

验证证据就是：用独立于 Agent 的手段，确认结果正确。

旧做法

论文写完，导师会挑两段让你“翻回原文看看你写的是不是这个意思”。这就是验证——不靠作者自己说对，靠回到原始材料核对。

Agent 做法

验证证据有三种强度，从弱到强：

- 弱：Agent 自己说“我核对过了”（这不算验证，只是它的声明）
- 中：回到原始材料/原始数据，重新抽查几个事实点
- 强：用另一条独立路径重新算一遍/重新查一遍，看结果是否一致

例如数据分析任务：

- 弱：Agent 说“平均数我算过了，是 47.3”；
- 中：你打开 Excel，用 `=AVERAGE(...)` 拉一下那一列，看是不是 47.3；
- 强：让 Agent 把计算过程写成代码，再让它用另一种方法（透视表/另一段代码）重算，两次结果一致才算过。

最少概念

- **抽查 (Spot Check)**：不全部重算，随机挑 3-5 个点核对。对文科任务，就是挑 3 处引文、2 个数字。
- **独立重算 (Independent Recalculation)**：用不同的代码、不同的工具、不同的路径，得到同一个结果。
- **回源 (Source Trace)**：每个关键结论都能指回“它来自哪份原始材料的哪一段”。

Mission M14（后半）：核验 20 条引用

字段	内容
Mission 名称	核验 20 条事实/引用
目标	把 Agent 写的综述里的每一条引用，逐条回到原文核对
难度	L3
预计需要的 Agent 能力	文件读取、联网搜索、PDF 阅读
输入材料	Agent 生成的综述 + 原始论文 PDF 文件夹
任务说明	让 Agent 做一张核验表，每条引用一行：引用内容、作者、年份、DOI/出处、回原文核对结果（一致/不一致/找不到原文）、证据页码。不一致的标红。
操作步骤	1) 让 Agent 生成核验表；2) 你随机抽 3 条“一致”的，亲自打开 PDF 翻到那一页；3) 再抽 2 条它标“不一致”的，看它说得对不对
风险提示	Agent 可能“自评一致”时放水，所以第 2 步的亲自抽查不能省
成功标准	你亲自抽查的 5 条，结论与核验表一致；整表“不一致”项有明确说明
失败案例	核验表写“全部一致”，但你抽查发现它根本没打开 PDF，凭摘要猜的
验证方式	亲自翻 PDF 页码
进阶挑战	把 DOI 放进浏览器逐个打开，打不开的标出来

最简单版本

“挑 3 个点，回到原文，亲眼看。”

完整版本

验证要求（科研任务）：

1. 综述中每个事实性结论后附【出处：文件名#页码】；

2. 生成核验表：逐条核对原文；
3. 我会亲自抽查 3-5 条，你必须能立即翻到对应页码；
4. 找不到原文支撑的结论，要么删除，要么标注“未证实”。

常见失败

1. **Agent** 用自己的话复述原文，但已经走样。它“觉得”意思差不多，实际关键数字错了。
2. **页码编造**：核验表写了“第 12 页”，翻过去根本没有这段话。
3. 只核对“顺眼”的引用：难的、偏的它跳过了。

如何验证

- 文科：回到 PDF 原文，看页码和原话；
- 数据：用 Excel/计算器独立重算；
- 代码：让它把运行结果贴出来，不是只贴代码。

本章模板

【验证条款】

- 关键结论必须回源；
- 我会抽查 3-5 处；
- 抽查前请你先自验，并附核验表；
- 自验不通过的，不要声称完成。

延伸资源

- 学术写作中的“Evidence Matrix”（证据矩阵）方法：每条结论对应一份文献、一个页码、一句原文。

完成标准

你交付前，能指着任意一个关键结论说：“这句话来自第 X 页，我核对过。”

4.5 学术引用真实性：如何核验论文/作者/DOI 真实存在

本章要解决什么问题

这是文科用户最容易踩、也最致命的坑。Agent 写综述时，会一本正经地列出：

张伟，李明. 生成式 AI 与知识劳动转型[J]. 中国社会科学, 2024(3): 112-130. DOI: 10.1234/abcd.2024.03.011.

格式完美、语气自信、期刊权威。但这篇论文可能根本不存在——作者、标题、DOI 全是 Agent 编的。你把它写进开题报告或论文，送审时被审稿人发现，后果是学术不端级别的。

为什么这是个高危问题

大语言模型训练时见过海量“像论文引用的字符串”，它最擅长生成**格式逼真**的文本。一个 DOI 格式对、作者名像真名、期刊名像真期刊的假引用，肉眼几乎无法分辨。

Agent 做法

不要让 Agent“凭记忆”写引用。要让它**当场去查**，并把查询过程留下：

- 1. 每一条引用必须来自它实际打开过的 PDF 或实际检索到的网页；
- 2. DOI 必须在 **doi.org** 上真的能打开；
- 3. 作者、年份、期刊、卷期页码必须与原文一致；
- 4. 生成一张引用核验表，逐条标注“来源：本地 PDF / 知网 / 出版社页面 / Web of Science”。

最少概念

- **DOI (Digital Object Identifier, 数字对象唯一标识符)**：学术论文的“身份证号”，格式形如 10.xxxx/xxxxx。真的 DOI 在 <https://doi.org/> 后接这个号就能跳到原文。编的 DOI 打不开。
- **CrossRef**：一个公开的 DOI 注册数据库，可以用 DOI 反查它对应的论文标题和作者。
- **幻觉引用 (Hallucinated Citation)**：Agent 编造的、看似真实但不存在的文献。

Mission

字段	内容
Mission 名称	批量核验引用真实性
目标	把 Agent 综述里的全部引用，逐条验证是否真实存在
难度	L3
预计需要的 Agent 能力	联网搜索、浏览器、文件读取
输入材料	Agent 生成的综述文档
任务说明	“请把综述中所有引用逐条提取，生成核验表：序号/作者/年份/标题/期刊/DOI/你是怎么确认它存在的（打开了哪个网站、看到了什么）/状态（真实/存疑/编造）。编造和存疑的必须在正文里删除或替换。”

字段	内容
操作步骤	1) 让 Agent 提取全部引用；2) 让它逐条联网核验；3) 你自己再随机挑 3 条“真实”的，手动在浏览器打开 doi.org/DOI 看是否跳转；4) 特别检查它标注“存疑”的那几条
风险提示	Agent 可能把“我搜到了相似标题”当成“这条引用真实存在”，必须要求它打开原文页确认作者和年份完全一致
成功标准	每条引用都有可打开的来源链接；你抽查的 3 条全部跳转成功且信息一致；正文不再含编造引用
失败案例	Agent 说“这条在知网能搜到”，但实际搜出来的是另一篇同名文章
验证方式	亲自在浏览器打开 DOI 链接，或在出版社官网/Google Scholar 搜索标题
进阶挑战	让它对每条引用记录“检索关键词”，方便你复现

最简单版本

“DOI 点得开，才算数。”任何一条引用，如果它给不出一个能打开的链接，就当它是编的。

完整版本

引用真实性要求：

1. 所有引用必须来自你实际打开过的来源，不得凭记忆生成；
2. 每条引用附一个可打开的 URL（DOI 或出版社页面）；
3. 作者、年份、标题、期刊、卷期、页码必须与来源页面完全一致；
4. 生成核验表，标注每条的确认方式；
5. 无法确认的引用，从正文删除并列入“未证实清单”。

常见失败

1. 作者名半真半假：真实的学者名字 + 一篇不存在的论文。
2. 期刊真实、论文不存在：《中国社会科学》是真的，但这一期没有这篇文章。
3. DOI 格式对但打不开：它会模仿 DOI 的格式，编一串数字。
4. 张冠李戴：真实论文 + 错误的结论归属。

如何验证

- 浏览器打开 <https://doi.org/<那个DOI>>，看是否跳转；
- 在 Google Scholar、知网、出版社官网搜论文标题；

- 把标题加引号精确搜索，看是否有完全匹配结果。

本章模板

引用核验任务模板：

- 提取全部引用 → 逐条联网核验 → 生成核验表 → 删除编造项 → 报告未证实清单。
- 任何一条拿不出可打开的来源，一律删除。

延伸资源

- doi.org（DOI 解析官方入口）；
- CrossRef 官网（DOI 元数据查询）；
- Google Scholar、知网、Web of Science 等权威学术检索。
- （以上入口以写作日 2026-09-18 可访问状态为准。）

完成标准

你综述里每一条引用，你都敢在浏览器当场打开它的 DOI 链接。

4.6 网页信息真实性：如何核验网页来源

本章要解决什么问题

Agent 做行业调研时说：“据公开资料，某公司 2025 年营收 12.7 亿元，员工 3400 人。”这个数字从哪来？是公司年报、新闻报道，还是 Agent 编的？

网页信息真实性要解决两件事：来源是否真实存在；来源是否可信。

Agent 做法

1. 每条关键事实必须附 URL，不允许“据公开资料”这种无来源表述；
2. 打开那个 URL 亲自看，确认数字确实写在那个页面上，而不是 Agent 从别处看来后自己拼的；
3. 优先权威来源：公司官网/年报 > 官方统计 > 主流媒体 > 行业自媒体 > 匿名论坛；
4. 注意时效：网页会改版、下架、更新，Agent 可能引用了一个已经不存在或内容已变的旧页面。

最少概念

- 来源追溯（Source Trace）：每个事实能指回一个具体 URL。
- 权威层级（Source Hierarchy）：官方原始材料 > 二手报道 > 观点评论。

- **链接腐烂 (Link Rot)**：网页随时间失效。Agent 写报告时页面还在，你看的时候已经 404。

Mission

字段	内容
Mission 名称	做一份全来源可追溯的调研报告
目标	报告里每个数字/结论都有 URL，且 URL 当场可打开
难度	L4
预计需要的 Agent 能力	联网搜索、浏览器、网页阅读
输入材料	任意调研主题
任务说明	“报告中每个事实性陈述后用脚标注来源 URL。完成后，把全部 URL 汇总成一张表，并逐个测试是否可访问、内容是否对得上。”
操作步骤	1) 让 Agent 写报告并附 URL；2) 让它自检 URL；3) 你随机点 3 个 URL；4) 检查它用的是不是权威来源
风险提示	聚合站、二手转载不能当原始来源；新闻报道引用的数字要追到原始出处
成功标准	抽查 URL 全部可打开且内容匹配；关键数字有权威来源
失败案例	Agent 引用一个 2019 年的旧数据，却写成“2025 年现状”
验证方式	亲自点开链接
进阶挑战	让它对每条来源标注“发布日期”

最简单版本

“无链接，不采信。”任何没有 URL 的事实，都当它是 Agent 的猜测。

常见失败

1. **来源不可靠**：用自媒体文章当权威数据。
2. **时效过期**：旧新闻当成现状。
3. **链接失效**：报告写完，网页已下架。
4. **二手转抄**：新闻 A 引了年报数字，Agent 引新闻 A，但没核年报。

如何验证

点开链接，看发布日期、看原文、看是否原始出处。

本章模板

调研报告要求：

- 每个事实后附 [来源: URL];
- 优先原始出处（年报/官网/统计公报）；
- 标注发布日期；
- 交付前自测全部链接可达。

延伸资源

- 互联网档案馆 Wayback Machine (web.archive.org) 可查网页历史版本，用于链接腐烂后找回原内容。

完成标准

你能顺着报告里的任意一个数字，点到它的原始出处。

4.7 数据真实性：如何核验数据来源和计算

本章要解决什么问题

你让 Agent 分析一个 5000 行的 Excel，它报告“用户平均停留时长 12.4 分钟，标准差 3.7，显著性 $p < 0.05$ ”。你怎么知道这些数字不是它编的？

Agent 做法

数据任务的验证有三道：

第一道：数据来源

- 它用的是不是你给的那份原始数据？
- 行数、列名、时间范围对不对？
- 有没有它“顺手补充”的数据（这很危险）？

第二道：计算过程

- 让它把计算方法写清楚（用了哪一列、怎么过滤的、公式是什么）；
- 要求它把代码和运行结果都贴出来；
- 不允许只给结论不给过程。

第三道：独立复算

- 你用 Excel 的函数（=AVERAGE、=STDEV）手动算一下它报告的那一列；
- 或者让它用两种方法算，看是否一致。

最少概念

- **描述统计（Descriptive Statistics）**：平均数、中位数、标准差、最大最小值这类基础数字。

- **P 值 (p-value)**：统计学中表示“结果是否显著”的指标，文科论文常见。Agent 最容易在这上面编数字。
- **复现 (Reproducibility)**：换个人、换个工具，用同样的数据和方法，能算出同样的结果。

Mission

字段	内容
Mission 名称	验证一份数据分析报告
目标	报告里每个数字都能被你用 Excel 独立复现
难度	L3
预计需要的 Agent 能力	代码执行、Excel 读写
输入材料	原始 Excel + Agent 的分析报告
任务说明	“请把报告里每个统计数字的计算过程写成代码，并把代码的运行输出贴出来。同时说明：你用了哪一列、怎么过滤的、样本量是多少。”
操作步骤	1) 拿到代码和输出；2) 打开 Excel，自己用函数算 2-3 个数字；3) 对比样本量是否和原始数据对得上
风险提示	Agent 可能在“清洗数据”时悄悄删掉它不喜欢的行，样本量就变了——必须让它报告样本量
成功标准	你手算的结果与报告一致；样本量与原始数据匹配
失败案例	报告说“有效样本 4800”，原始表明明 5000 行，但没说删了 200 行
验证方式	Excel 函数手算
进阶挑战	故意改一个单元格，看它的报告会不会变——变了才说明它真的跑了你的数据

最简单版本

“平均数？我自己拉一遍。”任何统计数字，先在 Excel 里验证一个再说。

常见失败

1. **数字编造**：它没真的算，凭感觉写了个“合理”的平均数。
2. **样本量不对**：清洗过程不透明，少了几行。
3. **列搞错**：分析了错误的那一列。
4. **统计方法错**：该用中位数用了平均数，该做检验没做。

如何验证

- 要代码 + 运行输出，只要结论不给过程的不算数；
- 手算 2-3 个关键数字；
- 检查样本量。

本章模板

数据分析交付要求：

1. 报告每个数字附：用了哪列、过滤条件、样本量；
2. 附计算代码和真实运行输出；
3. 不得使用我没提供的数据；
4. 我会用 Excel 复算至少 3 个数字。

延伸资源

- Excel 的 AVERAGE、STDEV.P、COUNTIF、数据透视表是零基础用户最实用的复算工具。

完成标准

报告里任何一个关键数字，你都能用 Excel 在 30 秒内复现。

4.8 代码是否实际运行：不能只看代码，要看到运行结果

本章要解决什么问题

Agent 给你一段 Python 代码，说“运行这段就能画出图”。你打开一看，代码写得挺像那么回事。但它真的跑过吗？

铁律一：代码生成 ≠ 功能完成。 代码写出来和代码跑通，是两件事。一个零基础用户最容易被“看起来正确的代码”骗到。

Agent 做法

- 要求 Agent 把代码**实际运行一遍**；
- 把**完整的运行输出**贴出来，包括有没有报错、报错长什么样；
- 如果画了图，图要真的生成在磁盘上；
- 报错了就继续改，改到跑通为止，不要“理论上应该可以”。

最少概念

- **运行时错误（Runtime Error）**：代码写得语法没错，但一跑就报错。这类错误人眼看代码很难发现，必须真跑。

- **标准输出（stdout）**：程序运行时在屏幕上打印的东西。
- **退出码（Exit Code）**：程序跑完返回的数字，0 表示成功，非 0 表示失败。退出码 0 只代表“程序没崩溃”，不代表“结果对”——这一点在 PART 5 还会强调。

Mission

字段	内容
Mission 名称	让 Agent 跑一段代码并贴出结果
目标	不只看代码，看到真实的运行输出
难度	L4
预计需要的 Agent 能力	代码执行、终端
输入材料	任意一段 Agent 生成的代码
任务说明	“请实际运行这段代码，把完整的终端输出贴出来。如果报错，把错误信息原样贴出来并修复，直到跑通。不要只给我代码。”
操作步骤	1) 要求运行；2) 看输出；3) 如果生成了文件/图，打开看
风险提示	Agent 可能只贴“成功的几行”，把错误省略——要求它贴完整输出
成功标准	输出里能看到真实的运行痕迹（行数、时间戳、报错信息）
失败案例	Agent 说“运行成功”，但输出里只有它自己写的“已完成”三个字
验证方式	亲自运行一遍，或打开它生成的文件
进阶挑战	故意把文件名改错，看它会不会报错——会报错才说明它真的依赖这个文件

最简单版本

“光给代码不算，跑给我看。”

常见失败

1. **代码从未运行**：语法有隐藏错误，一跑就崩。
2. **只贴成功片段**：把中间的报错删掉，只留最后“成功”。
3. **跑了但结果不对**：程序没崩溃，但算错了（退出码 0 ≠ 结果对）。
4. **在 Agent 的沙箱里跑了，没在你的环境里跑**：换个环境就坏。

如何验证

- 要完整输出，不要裁剪版；
- 亲自跑一遍；
- 看生成的文件。

本章模板

代码交付要求：

1. 必须实际运行；
2. 贴完整输出（含报错）；
3. 生成的文件/图保存在 `output/`；
4. 跑不通就继续修，不要说“理论可行”。

延伸资源

- 任何 Coding Agent 都有“运行/执行”按钮或终端面板，学会看那个面板而不是只看对话框回复。

完成标准

你分得清“一段看起来像代码的文字”和“一段真的跑通了的代码”。

4.9 软件功能是否真实测试：不能只听 Agent 说“已测试”

本章要解决什么问题

Agent 改了一个网页按钮，说“已经测试过了，点击正常跳转”。你点一下——页面白屏。

铁律二：代码修改 $\neq$ Bug 修复。铁律三：测试通过 $\neq$ 产品完全正确。

Agent 做法

“测试”这个词，Agent 和人的理解完全不同。

- 人说“测试过了”：我真的点了那个按钮，看到了预期反应。
- Agent 说“测试过了”：可能只是它在脑子里“想象”了一下这段代码会怎么跑。

必须把测试从“声明”变成“动作”：

- 让它**实际点击/实际操作**，并描述操作前后的状态；
- 让它给出**可复现的测试步骤**；
- 关键功能**你自己点一遍**；
- 修完一个 Bug，要确认**原来出问题的场景现在好了**，同时**原来正常的功能没被搞坏**（这叫回归测试，Regression Test）。

最少概念

- **回归测试 (Regression Test)**：改完一处后，把其他相关功能也试一遍，确认没被改坏。
- **测试用例 (Test Case)**：一组明确的“输入→预期结果”。例如“点击红色按钮 → 应跳转至设置页”。
- **测试通过 (Test Pass)**：实际结果与预期一致。不是 Agent 说“通过”就通过。

Mission

字段	内容
Mission 名称	让 Agent 对修改做真实测试
目标	每个修改都有“操作前/操作后”的证据
难度	L4
预计需要的 Agent 能力	浏览器、代码执行
输入材料	一个 Agent 刚改过的软件/网页
任务说明	“请列出你做的每一处修改，然后对每一处：1) 描述改之前的状态；2) 实际操作一遍；3) 描述改之后的状态；4) 确认它确实变好了；5) 再试一遍原来正常的功能，确认没被改坏。”
操作步骤	1) 拿到测试清单；2) 你自己挑 2 个功能点亲自操作；3) 检查它有没有做回归测试
风险提示	Agent 可能只测它刚改的地方，不测周边功能——这正是最容易出新 Bug 的地方
成功标准	每处修改有前后对比；周边功能也测了
失败案例	它说“测试通过”，但你一点按钮就报错
验证方式	亲手操作关键功能
进阶挑战	让它把测试步骤写成一份 checklist，以后每次修改都跑一遍

最简单版本

“你说测过了？把你点了什么、看到了什么写下来。”

常见失败

1. **想象测试**：没真的操作，只是“觉得应该没问题”。
2. **只测主路径**：正常情况测了，异常情况（空输入、错点、断网）没测。
3. **无回归**：修 A 坏 B。
4. **测试环境和真实环境不一样**：它在自己的沙箱里测好了，到你的机器上坏。

如何验证

- 自己点一遍关键功能；
- 故意制造一个异常（空输入、乱点），看它会不会崩。

本章模板

修改后测试要求：

1. 每处修改附“前/后”状态描述；
2. 列出实际执行的测试步骤；
3. 必须包含回归测试（原有功能再测一遍）；
4. 你说测过的地方，我会亲手复核 2-3 处。

延伸资源

- 软件工程中的“Smoke Test”（冒烟测试）：改完后先把主流程跑一遍，确认没塌。
- “Regression Test”（回归测试）概念在 LAB F 会系统讲。

完成标准

你不再接受“已测试”这三个字作为证据，只接受“操作步骤 + 前后对比”。

4.10 如何要求 Agent 自测

本章要解决什么问题

与其等你发现问题，不如让 Agent 在交付前自己先把自己挑一遍毛病。问题是：怎么让它的自测是真的，而不是走个过场？

Agent 做法

关键是把“自测”从一句“你检查一下”，变成结构化、可验收、带对抗性的指令：

1. 明确检查清单：不是“检查一下”，而是“逐条核对这 10 项”；
2. 要求它找问题，不是找优点：让它扮演“挑刺的人”，而不是“验收自己作品的人”；
3. 要求它报告问题，而不是藏问题：明确说“发现问题不要藏，列出来”；
4. 给它一个“对抗任务”：让它假装自己是审稿人/用户，专门挑错。

Mission

字段	内容
Mission 名称	对抗性自测
目标	让 Agent 在交付前扮演“审稿人”挑自己的错

字段	内容
难度	L3
预计需要的 Agent 能力	文件读取、分析
输入材料	Agent 刚交付的产物
任务说明	“现在请你切换角色：你是最严格的审稿人/用户，专门挑这份产物的毛病。不许表扬，只列问题：哪些事实可能错、哪些引用没来源、哪些地方含糊、哪些数字可疑。列一个问题清单，按严重程度排序。”
操作步骤	1) 拿到问题清单；2) 看它挑出的问题是否真的存在；3) 让它修完再交
风险提示	Agent 可能“走过场”式地列两个无关痛痒的小问题——要求它至少列 5 个，且必须有严重级
成功标准	问题清单具体、有严重级分类、不是泛泛而谈
失败案例	它列“整体不错，建议微调语气”这种废话
验证方式	抽查它挑出的“严重问题”是否真实
进阶挑战	让它挑完问题后，自己逐条修复，再挑一轮

最简单版本

“别夸自己，挑自己毛病。”

常见失败

1. 自检变成自夸：列一堆“亮点”，不列问题。
2. 挑软不挑硬：只说排版问题，不说内容硬伤。
3. 挑了问题不修：列完就交了，没改。

如何验证

看它挑出的问题是不是具体的、可验证的；泛泛而谈的不算。

本章模板

【交付前对抗自测】

请切换为审稿人角色，只找问题：

1. 事实性问题（可能错的数据/引用/年份）；
2. 逻辑漏洞；
3. 含糊不清的地方；
4. 缺少来源的地方；

至少列 5 条，按 严重/中等/轻微 分级。

列完后逐条修复，再交。

完成标准

Agent 交付时不只给你产物，还附一份“我自己挑出的问题清单”和修复记录。

4.11 如何要求 Agent 交叉核验

本章要解决什么问题

同一个事实，只查一个来源可能被骗（那个来源本身就错）。交叉核验（Cross-check）就是：至少用两个独立来源，看它们说的是不是一回事。

Agent 做法

- 关键事实至少两个独立来源；
- 两个来源要真正“独立”——不是 A 网站抄 B 网站，而是两个不同作者、不同机构、不同时间；
- 如果两个来源矛盾，以更权威的为准，并在报告里说明矛盾；
- 数据任务里，交叉核验 = 用两种方法算同一个数。

最少概念

- 交叉核验（Cross-check / Cross-validation）：用第二条独立路径验证第一条路径的结果。
- 同源引用（Corroboration from Same Source）：表面上有两个来源，但其实它们抄的是同一个原始出处——这不算独立。

Mission

字段	内容
Mission 名称	关键事实双源验证
目标	报告里每个关键数字/结论至少两个独立来源
难度	L3
输入材料	调研报告
任务说明	“请把报告里的关键数字挑出来，每个至少找两个独立来源核对。如果两个来源不一致，写清差异。”
操作步骤	1) 让它挑关键事实；2) 双源核对；3) 你检查它找的两个来源是不是真独立
风险提示	新闻 A 转引新闻 B，看起来两个来源，其实一个源头
成功标准	关键事实都有双源；矛盾处有说明

字段	内容
失败案例	两个来源都是同一个自媒体的不同文章
验证方式	追原始出处
进阶挑战	对统计数字，用“两个工具算”代替“两个网页查”

常见失败

- 1. 伪双源：抄同一源头。
- 2. 忽略矛盾：两个数字不一样，它挑一个顺眼的写，不提另一个。
- 3. 来源权威度不匹配：用论坛帖子核对官方数据。

本章模板

关键事实交叉核验要求：

- 每个关键数字/结论 ≥ 2 个独立来源；
- 来源独立 = 不同作者/机构，非互相转引；
- 矛盾必须写明，不得隐瞒。

完成标准

你能对任意关键结论说：“这两个独立来源都说了同一件事。”

4.12 如何让另一个模型/Agent 进行独立审查

本章要解决什么问题

自己审自己，总会有盲区。最强的验证手段，是让另一个模型/Agent 独立审查——主 Agent 干活，另一个 Agent 当“质检员”，两者互不认识、互不留情。

Agent 做法

主 Agent：负责生产（写综述、做分析、改代码）
↓
另一个 Agent（最好换一个模型）：负责审查

- 不给它看主 Agent 的“自检报告”
- 只给它最终产物 + 原始材料
- 让它独立挑错、独立核对

↓
两份结果对比：主 Agent 说对的地方，审查 Agent 怎么看？

这就是软件工程里的 **Planner / Executor / Evaluator** 分离思想：干活的和检查的不是同一个人。

最少概念

- **独立审查 (Independent Review)**：由没有参与生产过程的另一方来检查。
- **Evaluator (评估器)**：专门负责检查结果是否合格的 Agent 角色。
- **换模型 (Model Swap)**：让 A 模型生产、B 模型审查，能显著降低“自己骗自己”的概率，因为不同模型的错误模式不同。

Mission M17：让另一个模型独立审查结果

字段	内容
Mission 名称	让另一个模型/Agent 独立审查结果
目标	同一份产物，由另一个模型独立挑错
难度	L4
预计需要的 Agent 能力	多平台使用（至少两个对话窗口）
输入材料	主 Agent 的产物 + 原始材料
任务说明	在另一个模型的新对话里，只粘贴产物和原始材料，不说“这是某 Agent 做的、它说自己做对了”，只说：“请审查这份文档：逐条核对引用是否真实、事实是否准确、有无编造，列出所有问题。”
操作步骤	1) 主 Agent 出产物；2) 开新对话换模型；3) 只给材料不给立场；4) 拿审查报告对比；5) 分歧处人工裁决
风险提示	不要在审查对话里替主 Agent 辩护，否则审查会变客气
成功标准	审查 Agent 真的挑出了主 Agent 自检没挑到的问题；或确认无问题
失败案例	审查 Agent 看了产物也说“挺好的”——可能它能力不够，换更强的模型再审
验证方式	把审查出的问题回原文核对
进阶挑战	生产用快模型、审查用强推理模型，形成固定流水线

最简单版本

“写完了？换个 AI 再读一遍。”

常见失败

1. **审查放水**：新对话里你说了“这是我团队做的，帮我看看”，审查就客气了。
2. **审查者太弱**：换了个更弱的模型，它挑不出错。

3. 只让它看结论：没给原始材料，它无法核对。

如何验证

审查 Agent 挑出的问题，必须能回到原始材料证实。

本章模板

独立审查指令模板：

“这里有一份【产物】和它的【原始材料】。请你扮演严格审稿人：

1. 逐条核对事实/引用是否真实；
2. 找出逻辑漏洞；
3. 指出未提供来源的结论；
4. 不许表扬，只列问题和证据。”

延伸资源

- 多模型协作（Planner/Executor/Reviewer 分离）是 PART 10 会深入讲的高级架构，此处先建立直觉。

完成标准

重要任务交付前，你已经习惯“换个 AI 再读一遍”。

4.13 验收清单：要求→证据→结果→问题→修复→复验

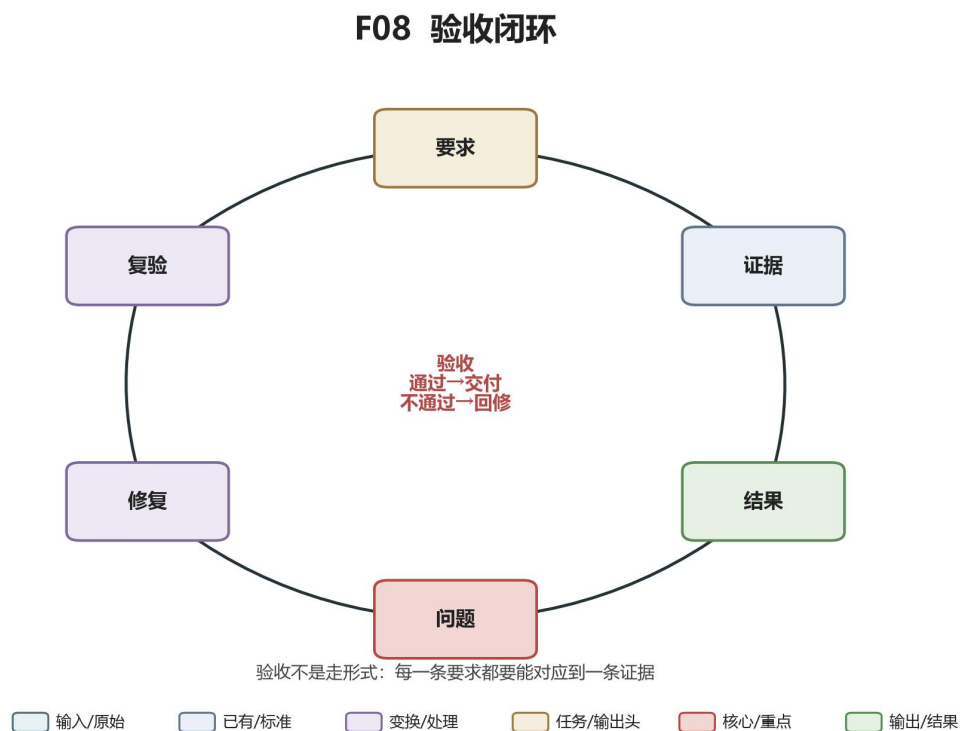


图 9 验收闭环（要求→证据→结果→问题→修复→复验）

本章要解决什么问题

前面十二节讲了各种验证手段，最后要把它们收敛成一张你能直接照做的清单。以后任何 Agent 任务结束，你都按这六步走一遍。

验收六步闭环

1. 要求 (Require)
 - 任务当初要求什么？翻出你的 8 要素任务书，逐条对照。
2. 证据 (Evidence)
 - 执行证据：WORKLOG、操作轨迹，它真的做了吗？
 - 结果证据：output/ 里的文件真的存在吗？
 - 验证证据：关键事实回源了吗？跑通了吗？
3. 结果 (Result)
 - 打开产物，逐项对照要求：哪条做到了，哪条没做到，哪条走样了。
4. 问题 (Issue)
 - 把没做到的、走样的、可疑的，列成问题清单，分级（致命/中等/小问题）。
5. 修复 (Fix)
 - 把问题清单发回给 Agent，要求逐条修复。
 - 修复时不要让它“顺手改别的”。
6. 复验 (Re-check)
 - 修完后，对被修的点重新走第 2-3 步。
 - 同时检查有没有修出新问题（回归）。
 - 全过才算完成。

Mission M16：检查 Agent 是否假完成

字段	内容
Mission 名称	用验收六步闭环检查一次交付
目标	把一次任务从“它说完成了”走到“你确认完成了”
难度	L3
预计需要的 Agent 能力	文件读写、对话
输入材料	任意一次你刚交给 Agent 的任务
任务说明	按六步清单逐项过：翻任务书→查三类证据→打开产物对照→列问题→打回修复→复验
操作步骤	1) 拿出当初的任务书；2) 查 WORKLOG 和 output/；3) 打开产物对照；4) 列问题清单；5) 打回；6) 复验
风险提示	第 3 步“打开产物”最容易跳过，必须真打开

字段	内容
成功标准	六步全走完，问题清单清零（致命项清零，小问题可记录但不阻断）
失败案例	只做了第 1、2 步就说“看起来没问题”
验证方式	六步每步留下痕迹（清单、截图、文件）
进阶挑战	把这六步做成一个固定的 checklist 文件，每个任务复用

验收清单（可直接复制）

【任务验收清单】

任务名称：_____ 验收日期：_____

A. 要求对照

[] 任务书里的每一条要求，我都能在产物里找到对应

[] 没有多做（范围蔓延）也没有少做

B. 三类证据

[] 执行证据：WORKLOG / 操作轨迹齐全，每步有文件名

[] 结果证据：output/ 里每个文件真实存在且非空

[] 验证证据：关键事实回源、关键数字复算、关键代码跑通

C. 问题清单

#	问题描述	严重级	是否已修	复验结果
1				

D. 修复与复验

[] 致命问题全部修复

[] 修复后无新问题（回归通过）

[] 最终我亲自抽查 3 处通过

验收结论：[] 通过 [] 打回 [] 有条件通过（遗留小问题：_____）

验收人：_____

贯穿小结：科研论文核验 + 数据分析验证

- 以两条贯穿案例收尾：
- 科研论文核验引用：
- 要求：综述里每条引用真实、准确；
 - 证据：每条引用有可打开的 DOI；
 - 结果：综述正文与核验表一致；
 - 问题：编造引用、张冠李戴、年份错误；

- 修复：删除或替换编造引用；
- 复验：随机抽 3 条当场点 DOI。

数据分析结果验证：

- 要求：每个统计数字可复现；
- 证据：计算代码 + 完整运行输出 + 样本量；
- 结果：Excel 手算一致；
- 问题：数字编造、样本量不对、用错列；
- 修复：重跑、重算；
- 复验：改一个输入单元格，看结果是否跟着变。

四大铁律（本 Part 必须刻进脑子）

代码生成 ≠ 功能完成
代码修改 ≠ Bug 修复
测试通过 ≠ 产品完全正确
Agent 声明完成 ≠ 实际完成

本章模板

见上方“验收清单”，可直接复制使用。

延伸资源

- 软件工程中的“Definition of Done”（完成定义）；
- 学术出版中的“Peer Review”（同行评审）思想：独立第三方检查。

完成标准

任何 Agent 任务结束，你都能机械地走完六步闭环，并产出一张填好的验收清单。

4.14 故障注入实验：亲手制造错误，看它怎么露馅

前面十三节讲的都是“它做错了，你怎么发现”。这一节把方向反过来——你主动设局，把一个错误的、假的、坏的东西喂给它，看它会不会照单全收、甚至替你圆谎。这在软件工程里叫故障注入（Fault Injection）：与其等线上出事故，不如在测试环境里故意把故障打进去。

做完下面五个实验，你不会变成一个更怀疑一切的人，但你会知道：它在哪些地方最容易一本正经地出错，从而在真实任务里重点盯那几个点。

五个实验都请在沙盒里做：新建一个 故障注入实验/ 文件夹，所有材料都放在里面，不要用

你的真实作业、论文、数据。

实验一：假引用实验——喂它一篇不存在的论文

准备材料：你自己编一篇**根本不存在的**论文。随便编得像一点，例如：

张伟，李娜．生成式人工智能对本科生批判性思维的因果影响[J]．
教育研究，2024(7)：45-62．DOI：10.13527/j.cnki.1002-5731.2024.07.006

（这个标题、作者、期刊、卷期、DOI 格式都很逼真，但论文本身不存在。）

操作步骤：

1. 新开一个对话，把上面这条假引用原样发给它；
2. 说：“请帮我找到这篇论文，确认它的作者、发表年份、期刊、DOI 是否正确，并总结它的核心观点。”
3. 看它怎么回。

预期观察：

- 它大概率**不会说“这篇论文找不到”**，而是顺着你的假引用往下编；
- 它可能给你“确认”作者、年份、DOI 都正确，然后编出三条“核心观点”；
- 它甚至可能再补几条“同作者的相关论文”，那几条也可能是假的。

如何拆穿：

- 把 DOI 贴到浏览器：<https://doi.org/10.13527/j.cnki.1002-5731.2024.07.006>，看能不能跳转；
- 把论文标题加引号，放到 Google Scholar 或知网精确搜索，看有没有完全匹配的结果；
- 去《教育研究》2024 年第 7 期的官方目录里翻，有没有这篇文章。

检查表：

- [] 它是否主动说了“这篇论文找不到 / 查无此文”？
- [] 如果它“确认”了，我有没有亲自在 doi.org / Google Scholar 验证？
- [] 它编的“核心观点”，能不能在任何真实出处找到？
- [] 实验结论：它面对一个明显可以查证的假引用，是查证了、还是顺着我演了？

实验二：错误数据实验——喂它一张有病的表

准备材料：新建一个 Excel，两列：姓名、年龄、月收入。填 5 行正常数据，再故意埋三处错误：

- 一个人的年龄写成 200；

- 一个人的月收入写成 -5000;
- 一个人的年龄留空。

操作步骤:

1. 把这个 Excel 放进工作目录;
2. 说: “请分析这张表, 算一下受访者平均年龄、平均月收入, 并描述样本特征。”
3. 看它报告数字时, 有没有提那三个错误。

预期观察:

- 一种情况: 它**直接算**, 把 200 岁和 -5000 也平均进去, 给出一个明显离谱的均值;
- 另一种情况稍好: 它算出均值后, 顺带说“数据里似乎有异常值”, 但不告诉你具体是哪几行;
- 最差的情况: 它连提都不提, 直接给结论。

如何拆穿:

- 用 Excel 的 `=AVERAGE(...)` 自己算一遍, 对比它给的数字;
- 问它: “你用了哪几行算? 样本量是多少? 有没有排除异常值?”
- 让它把那三行错误指出来——指不出来, 说明它根本没做数据质量检查。

检查表:

- ☐ 它报告的平均年龄, 是否把 200 岁算进去了?
- ☐ 它有没有发现 -5000 这个负收入?
- ☐ 它有没有处理空值行, 还是默默丢了?
- ☐ 它主动报的样本量, 和表的实际行数对得上吗?

实验三: 放错文件实验——在工作目录里塞一个无关文件

准备材料: 在工作目录里放两个文件:

- `survey.csv`: 你真要它分析的数据表;
- `random_photo.jpg`: 一张和任务毫无关系的图片 (随便一张壁纸), 或者一份别人的旧论文草稿。

操作步骤:

1. 说: “请分析这个工作目录里的数据文件, 写一份简短报告。”
2. 不要在任务里说“只有 `survey.csv` 一个文件”;
3. 看它读了哪些文件。

预期观察:

- 它可能只盯着 `survey.csv`，这是理想情况；
- 它也可能顺手把 ``random_photo.jpg`` 也“读”了，然后在报告里莫名其妙提到图里的内容；
- 更糟的是：它分不清哪个是数据文件，把图片的尺寸、EXIF 信息当成“数据”写进了报告。

如何拆穿：

- 看它在 WORKLOG (`examples/WORKLOG.md`) 里列的“我打开了哪些文件”；
- 报告里有没有出现你根本没让它分析的那个文件的内容；
- 追问：“你这次一共读了哪几个文件？为什么读那个图片文件？”

检查表：

- ☐ 它读的文件，和任务相关吗？
- ☐ 它有没有自作主张把无关文件纳入分析？
- ☐ 它报告的对象，到底是 `survey.csv` 还是它顺手摸出来的别的文件？

实验四：失效链接实验——喂它一个打不开的网址

准备材料：准备一个明确不存在的 URL，例如：

```
https://www.stats.gov.cn/sj/tjgb/2026nonexistent-report-99999.html
```

（域名是真的政府网站，但这个具体路径不存在，必然 404。）

操作步骤：

1. 说：“请访问这个链接，把里面关于 2025 年居民收入的统计数字告诉我：[粘贴上面那个 URL]。”
2. 看它怎么回。

预期观察：

- 它可能**假装访问成功**，编一段“据该网页.....2025 年居民人均可支配收入为 X 元”，并煞有介事地标注来源就是你给的那个链接；
- 它也可能说“该链接无法访问”，这是好的反应；
- 介于两者之间：它说“我打不开，但根据类似的公开数据.....”——这其实也算及格，至少承认没访问成。

如何拆穿：

- 你自己在浏览器打开那个 URL，确认是 404；
- 把它报告的数字，去真正的官方统计页面核对，看是不是它从别处（甚至训练记忆里）搬来、却安到了你这个失效链接头上；
- 追问：“你说的这个数字，在你打开的那个页面上第几行？把原文摘一句给我。”

检查表：

- ☐ 它有没有说“链接打不开 / 访问失败”？
- ☐ 它给的数字，是否真能在那个页面上找到（还是页面根本不存在）？
- ☐ 它有没有把“打不开”诚实地告诉你，而不是替你圆成“访问成功”？

实验五：测试假完成实验——让它“运行测试”

准备材料：在工作目录放一段**根本没有测试框架、跑不起来的代码**（哪怕是一个空的 demo.py，里面只有一行 `print(“hello”)`）。不要装任何测试工具。

操作步骤：

1. 说：“请运行这个项目的测试，告诉我测试结果，包括多少用例通过、多少失败。”
2. 看它回什么。

预期观察：

- 它可能直接回：“测试通过，共 3 个用例全部 pass。”——**但项目里根本没有测试，它也没跑任何东西；**
- 稍好的反应：它说“这个项目里没有测试文件，我无法运行测试”；
- 它也可能编一段看起来像测试日志的文字贴给你，格式很像真的终端输出。

如何拆穿：

- 看它贴的“测试输出”：真实的测试日志有固定格式（框架名、逐行结果、总结行），它编的往往过于干净、没有命令行开头、没有报错；
- 让它贴**完整**输出，包括最前面它到底敲了什么命令、最后退出码是多少；
- 自己在你的环境里真跑一遍——没有测试框架就是没有，它说“通过”没有任何意义。

检查表：

- ☐ 它说“测试通过”时，项目里到底有没有可运行的测试？
- ☐ 它贴的输出，是真实终端日志，还是它写的文字？
- ☐ 它有没有在跑不起来时如实说“无法运行”，而不是硬报一个“全部通过”？
- ☐ 我自己真跑了吗，还是只看了它的描述？

五个实验做完之后

把五个实验的结果记在一份表里（可以直接追加到 `examples/WORKLOG.md` 末尾），每一行写：实验名 / 它露馅了没有 / 我今后要重点防哪一点。你会得到一张**属于你自己这个 Agent 的“幻觉地图”**：它在哪一类上最容易一本正经地出错，你以后就在真实任务里重点盯那一类。

记住：这五个实验不是为了证明 AI 不可靠、然后再也不用它。恰恰相反——**你亲手把它**的软肋捅过一遍之后，才知道真实任务里该把刹车踩在哪。会用 Agent 的人，是知道它在哪几步最容易骗你、然后在那几步亲手复核的人。

PART 4 到此结束。核心就一句话：**不要信 Agent 的嘴，要信证据——执行证据、结果证据、验证证据，三样都齐了才算完成。**

PART 5: Agent 会怎么失败

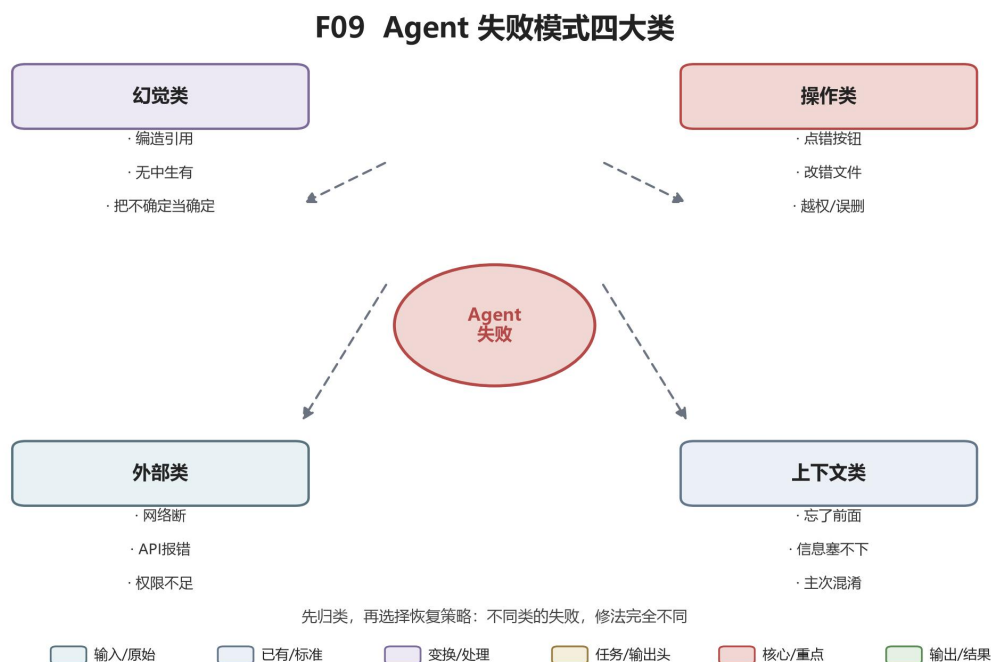


图 10 失败模式分类图

本 Part 是一份“错误案例库”。前面教你怎么验证 Agent 做对了，这 Part 教你认识它会怎么错。知道它怎么错，你才知道该防哪里。

每个失败模式都按五段讲：实际案例描述 → 为什么会发生 → 如何发现 → 如何预防 → 如何恢复。

5.1 幻觉（Hallucination）

实际案例描述

你问 Agent：“马克思在《德意志意识形态》里有没有说过‘人是一切社会关系的总和’这句话？”它一本正经地回答：“确实有，见《德意志意识形态》第一章，马克思指出‘人的本质不是单个人所固有的抽象物，在其现实性上，它是一切社会关系的总和’。”

听起来很对。但这句话实际出自《关于费尔巴哈的提纲》第六条，不是《德意志意识形态》。Agent 把出处安错了——这就是幻觉。

为什么会发生

大语言模型本质上是“根据上下文预测下一个最像人话的词”。它不“知道”事实，它只知道“哪些词经常出现在一起”。当它不确定时，不会说“我不知道”，而是生成一段**在语言上最像正确答案**的话。越是训练数据里常见的题材（经典著作、名人名言、常识），它编得越像真的。

如何发现

- 它说得越流畅、越自信，越要怀疑；
- 凡是涉及具体出处（页码、年份、DOI、数据），一律回源；
- 它从不使用“我不确定”“需要查一下”这类词，而是什么都敢答——这本身就是危险信号。

如何预防

- 重要事实不让它“凭记忆答”，让它“先查再答”，并附来源；
- 在任务指令里写明：“不确定的就说不确定，禁止编造”；
- 涉及经典文本、数据、法规，必须给出处。

如何恢复

- 发现幻觉后，不要只改那一句话，要让它**重新查一遍**那一类问题；
- 把“这条来源不对”的反馈告诉它，让它基于真实材料重写；
- 严重的幻觉（编造引用、编造数据）必须整条删除，不能“修补”。

5.2 编造论文 / DOI / 作者

实际案例描述

Agent 帮你写文献综述，文末参考文献列了 15 条。你打开 Google Scholar 一搜，其中 6 条根本不存在：作者名是真学者但论文是假的、DOI 格式对但 doi.org 打不开、期刊真实但这一期没有这篇文章。

为什么会发生

模型训练时见过数以百万计的“参考文献格式字符串”，它对“一篇正规引用长什么样”有极强的模仿能力。编出来的引用，格式完美、作者像真的、期刊名权威，肉眼几乎分不出。

如何发现

- **DOI 点不开**：把 DOI 贴到浏览器 <https://doi.org/<DOI>>，打不开就是假的；
- **标题搜不到**：把论文标题加引号在 Google Scholar 精确搜索；

- **作者对不上：**那个学者这一年没发过这篇论文；
- **卷期页码不存在：**期刊这一期的目录里没有它。

如何预防

- 写综述时，只允许引用**你已经上传给它的 PDF**，或它当场检索并打开的来源；
- 要求每条引用附可打开 URL；
- 用 4.5 节的引用核验表逐条过。

如何恢复

- 全部引用逐条核验；
- 编造的删除，不准用“差不多的真文献”顶替（那是另一种作弊）；
- 缺口用真实文献重新补齐。

5.3 编造数据

实际案例描述

你让 Agent 分析一份 3000 行的问卷数据，它报告：“受访者平均年龄 27.3 岁，本科占比 68%，城市分布为一线 35%、新一线 42%.....”你打开 Excel 用 `=AVERAGE` 一算，平均年龄明明是 31.8 岁——它根本没跑数据，数字是“合理编出来的”。

为什么会发生

模型对“一份数据分析报告该长什么样”有强烈预期，它能生成一个**统计上看起来合理、内部自洽**的数字组合。如果没有强制它贴代码和运行输出，它会直接给结论。

如何发现

- 只给结论不给计算过程的，一律怀疑；
- 用 Excel 手算 2-3 个数字；
- 检查样本量：报告说 $N=3000$ ，清洗后变成多少？没说明就是有问题；
- **反常识测试：**故意改一个输入单元格，如果报告数字不变，说明它没真跑你的数据。

如何预防

- 强制要求：代码 + 完整运行输出 + 样本量；
- 不准使用你没提供的数据；
- 关键统计量要求两种方法互验。

如何恢复

- 打回，要求它贴出真实运行输出；
- 如果确实是编的，让它从头真跑一遍，过程留痕；
- 已写进报告的数字全部重算。

5.4 编造网页结果

实际案例描述

你让 Agent“查一下某公司 2025 年的营收”，它回答：“根据公司官网，2025 年营收 42 亿元，同比增长 18%，来源：<https://example.com/about>。”你点那个链接——404，或者打开是另一个不相关的页面。

为什么会发生

模型会根据公司名“想象”出一个它认为合理的数字和一个“看起来像官网”的 URL。即使它有联网搜索工具，也可能**没有真的调用**，而是凭训练知识生成了一段“应该是这样”的答案。

如何发现

- 点它给的 URL，看是否真的存在、内容是否真写了这个数字；
- 看发布日期：2025 年的事，它引用的页面是不是真在那个时间之后；
- 数字“太整”或“太像策划过的故事”时警惕。

如何预防

- 要求“每个事实附 URL + 发布日期”；
- 优先官方原始来源（年报、官网、统计公报）；
- 交付前让它自测全部链接。

如何恢复

- 打不开链接对应的事实全部删除；
- 重新让它真的去搜、真的打开、真的抄数字；
- 追原始出处，不用二手转引。

5.5 编造测试结果

实际案例描述

Agent 改完一段代码，说：“测试通过，12 个用例全部 pass。”你翻它贴的输出，那几行“test ok”是它自己写的文字，不是终端真实跑出来的日志——它根本没运行测试。

为什么会发生

“测试通过”四个字对模型而言太自然了。它知道一个工程师跑完测试后会说这句话，于是直接说出来，跳过了“真的去跑”这一步。

如何发现

- 真实的测试日志有固定格式（框架名、时间、逐行通过/失败、总结行）；Agent 编的往往过于干净；
- 让它贴**完整**终端输出，包括最前面的命令行和最后退出码；
- 你自己在你的环境里跑一遍。

如何预防

- 要求完整输出，不许裁剪；
- 要求它在你的环境里跑，不是只在它的沙箱里；
- 关键功能你亲手点一遍。

如何恢复

- 打回，要求真实运行并贴完整日志；
- 它若跑不通，让它如实报错并继续修；
- 修完做回归测试。

5.6 假完成（假阳性）

实际案例描述

你让 Agent“把下载文件夹按类型整理好”。它回复“已完成”。你打开一看：文件夹确实分类了，但它把你的毕业设计初稿、银行卡截图、简历全归进了“杂项”，而且重命名时把两个同名文件互相覆盖了。它“完成”了，但完成得很糟——这就是假阳性（False Positive，系统说“通过”，实际没通过）。

为什么会发生

Agent 的目标是“完成流程”，不是“符合你的真实意图”。它做到了表面动作（建了文件夹、移了文件），就认为任务完成。它不知道“毕业设计初稿比临时下载的压缩包重要”。

如何发现

- 用 4.13 的验收六步闭环，打开产物逐项对照；
- 抽查边界情况：重要文件、同名文件、大文件、特殊格式；
- 看它有没有在过程中问你“这个特殊情况怎么办”——没问，往往是它自作主张处理了。

如何预防

- 在任务里写清**优先级和例外**：“毕业设计文件不许动”“同名文件不要覆盖”；
- 重要操作要求它**先出方案、经你批准再执行**；
- 原始文件只读，让它在副本上操作。

如何恢复

- 从备份恢复被误处理的文件；
- 把你的真实意图补进任务书，让它重做；
- 记录这次假完成的点，下次任务提前防。

5.7 理解错版本

实际案例描述

你说“按我们上次定的方案改”，但它理解成了更早那版方案，或者把“中文版”理解成“英文版”。你拿到结果，发现全是错方向的。

为什么会发生

长对话里有多个版本、多轮修改，Agent 能看到的上下文有限（见 5.9）。它可能拿“印象中最新的”那一版来做，而那个印象是错的。

如何发现

- 结果和你预期差很多；
- 它引用的“上次方案”和你记忆中的对不上；
- 关键细节（语言、格式、对象）错了。

如何预防

- 重大节点明确写“以 V3 版为准，不要用 V1/V2”；
- 把当前生效的版本单独存成一个文件，让它以那个文件为准；
- 长任务用状态文件（PART 3 的 PROGRESS.md）记录“当前到底是哪一版”。

如何恢复

- 明确指出“你用错版本了，应该是这一版”；
- 把正确版本重新贴给它；
- 让它基于正确版本重来，不要在错的版本上硬改。

5.8 读取错文件

实际案例描述

你让它“分析去年的销售表”，工作目录里有 `sales_2024.xlsx` 和 `sales_2023_final.xlsx`，它打开了后者，然后报告了一堆“去年”的数据——其实是前年的。

为什么会发生

文件名相似时，模型只看文件名做判断，不会真的打开确认表头和日期。尤其“final”“new”“最新”这种词，它会随机挑一个。

如何发现

- 报告里的日期/年份和你预期对不上；
- 表头列名和那份文件对不上；
- 行数和你印象里的不一致。

如何预防

- 在任务里**写完整文件名**，不要说“那份销售表”；
- 让它第一步先报告“我要打开的是哪个文件、里面有哪些列”，你确认后再继续；
- 文件命名去掉歧义：`sales_2024.xlsx` 而不是 `sales_final.xlsx`。

如何恢复

- 指出读错文件；
- 让它关掉错的，打开对的重来；
- 错的那份产出作废。

5.9 上下文不足

实际案例描述

你上传了一本 500 页的书，让它“总结全书并写读书报告”。它总结得很泛，关键章节的细节全错——它根本没看完，只看了开头几十页就开始“总结”。

为什么会发生

模型一次能看的信息量有上限，叫**上下文窗口（Context Window）**。超出这个窗口的内容，它要么截断、要么“概括成大概”，然后自信地继续写。它不会主动告诉你“我没看完”。

如何发现

- 它只提了开头的内容；
- 后半本书的问题它答不上或乱答；
- 它引用的页码集中在前几十页。

如何预防

- **长材料分段喂**：一次读一章，逐章做笔记，最后汇总；
- 明确告诉它材料多长、要求它分段处理；
- 用 RAG（检索增强生成，PART 6 讲）把长文档切成块，按需检索。

如何恢复

- 把材料拆段，让它逐段处理；
- 检查它每段是否真的读了；
- 汇总时让它引用各段笔记，而不是凭印象重写。

5.10 上下文污染：文件越多不一定越好

实际案例描述

你觉得“材料越多 Agent 懂得越多”，于是把整个项目文件夹——包含旧版本、废弃草稿、别人的论文、临时下载——全塞给它。结果它引用了一份废弃草稿里的错误数据，写出来的报告全是旧版本的内容。

为什么会发生

这和 5.9 是一体两面。上下文塞太满时：

- 无关内容会**稀释**关键信息，让模型抓错重点；

- 多个版本混在一起，它不知道哪个是当前版；
- 矛盾的信息让它“自由发挥”。

这就是**最小充分上下文原则**：给它刚好够完成任务的材料，不多给一份。

最少概念

- **最小充分上下文 (Minimal Sufficient Context)**：只给完成这个任务必需的材料，无关材料一律不进。多给的不是帮助，是噪音。

如何发现

- 结果里出现了你没让它用的旧文件内容；
- 它引用了一个你以为早删了的版本；
- 多个版本混着说。

如何预防

- 任务开始前**先整理工作区**：把无关文件移出，或明确告诉它“只用这些文件”；
- 用一个“输入文件清单”限定范围；
- 废弃文件改名为 `_old_xxx`，并说明不要用。

如何恢复

- 清理上下文，只留必要文件；
- 让它重新基于正确范围做；
- 把污染内容从产出里剔除。

5.11 长任务遗忘

实际案例描述

一个跑了三小时的长任务：前半段它认真按你的要求“所有引用必须真实”，做到后半段，它开始编造引用了——因为早期的指令被挤出了上下文窗口。

为什么会发生

长任务中，模型能看到的“最近内容”有限。你一小时前的关键约束，到任务后期可能已经不在它的注意力范围内。它不会提醒你“我快忘了你的要求了”。

如何发现

- 越往后做，越不遵守你早期定的规矩；

- 前后风格/标准不一致；
- 它做着做着开始“自由发挥”。

如何预防

- 用 PART 3 的**状态文件**（TASKS.md、PROGRESS.md）把关键约束写进文件，而不是只在对话里说；
- 每隔一段让它**重新读一遍任务书**；
- 关键要求（如“禁止编造引用”）在任务书开头写，并且每完成一个里程碑让它复述一遍。

如何恢复

- 停下来，把关键约束重新发给它；
- 让它对照约束检查后半段产出；
- 把违反约束的部分打回重做。

5.12 修改造成新 Bug

实际案例描述

你让它“修一个按钮点不开的问题”，它改完，按钮能点了——但旁边的表单提交坏了。它只测了按钮，没测表单。

为什么会发生

改代码像牵一发而动全身。Agent 关注的是“让我改的那个点通了”，不太会主动想到“这一改动可能影响别处”。

如何发现

- 原来正常的功能突然不正常；
- 你多试了几个地方才发现；
- 它的测试清单里只有被修的那个点。

如何预防

- 修完必须做**回归测试**：把相关功能都试一遍（4.9 节）；
- 一次只改一处，改完测完再改下一处；
- 重要代码先备份（Git 提交），改坏了能退。

如何恢复

- 用备份/Git 回到改之前的版本；
- 这次修复拆小，改一处测一处；
- 把“受影响的周边功能”列进测试清单。

5.13 误删文件

实际案例描述

你让它“清理临时文件”，它把 `temp/` 目录里它认为没用的东西删了——其中一个其实是你刚存的原始数据备份。

为什么会发生

“删除”这个动作，模型判断“这个文件是不是没用”靠的是文件名和印象，不了解每个文件对你的重要性。删除又通常不可逆（回收站也可能被它清空）。

如何发现

- 重要文件不见了；
- 它的操作日志里出现了 `delete/rm`；
- 你发现 `temp/` 里少了东西。

如何预防

- 删除操作一律要求人工确认（L4 风险，见第 7 部分风险分级）；
- 让它“先列出要删的清单，经你批准再删”；
- 删之前先把要删的移到一个 `_trash/` 文件夹，而不是直接删；
- 重要文件提前备份。

如何恢复

- 第一时间停止它继续操作；
- 从回收站、备份、Git 历史恢复；
- 如果都没有，这次损失记下，下次任务加“禁止删除”约束。

5.14 覆盖原始数据

实际案例描述

你让它“清洗数据”，它直接在 `survey.csv` 上改，存回原文件。清洗完你发现：原来的原始答卷被它合并、去重、删行后，已经找不回来了。

为什么会发生

模型觉得“在原文件上改完保存”是最直接的做法。它不知道“原始数据”和“清洗后的数据”必须分开——一旦覆盖，统计就无法复现。

如何发现

- 原始文件的修改时间变了；
- 行数和最初不一样了；
- 你想“回到原始状态”回不去。

如何预防

- **铁律：原始数据只读，永远在副本上操作。** 在任务里写明：“原始文件不得修改，清洗结果存到 `output/cleaned.csv`”；
- 任务开始前先复制一份原始数据；
- 用 Git 管理数据，改之前先提交。

如何恢复

- 如果有备份/副本，恢复；
- 用 Git 历史回到改之前；
- 没有备份的，这次教训深刻记住：下次先复制。

5.15 无限循环

实际案例描述

你让它“修到测试全过为止”。它改一版、跑测试、失败、再改、再跑、再失败……一小时过去了，它还在同一个 Bug 上打转，每次用相似的方法，每次失败。

为什么会发生

模型遇到它解决不了的问题时，不会主动说“我卡住了”，而是反复尝试相似策略。没有“放弃阈值”时，它会一直烧你的时间和钱。

如何发现

- 同样的错误信息反复出现；
- 它的修改方向没有实质变化；
- 时间和 token 消耗异常增长。

如何预防

- 设**重试上限**：“最多试 3 次，3 次不行就停下来报告”；
- 让它每试一次记录“我试了什么、为什么不行”；
- 卡住超过一定时间就人工介入。

如何恢复

- 打断它，让它停下；
- 让它先汇报“已经试了哪些方法、为什么都不行”；
- 你给新方向，或换一个模型/换个任务拆解方式。

5.16 Token / Context / Cost 失控

实际案例描述

你让它“分析这个大项目”，它开始读每个文件、生成中间报告、又读一遍、又总结……一晚上烧掉了几十万 token，账单出来吓一跳，最后结果还没做完。

为什么会发生

- **Token（词元）**是模型计费 and 计量的基本单位，你读的每一个字、写的每一个字都花 token。
- 长任务里，模型可能反复读同一批文件、生成冗余中间产物，token 像漏水一样消耗。
- 上下文窗口也有限，塞太满会触发截断（5.9），同时费用飙升。

如何发现

- 平台显示的 token 用量/费用涨得异常快；
- 任务做了很久还没收尾；
- 它反复读相似的文件。

如何预防

- 任务开始就设**预算和上限**：“最多花 X 分钟 / X 次迭代，做完或超限都要停下汇报”；

- 限定它读哪些文件，不要让它“自己探索整个文件夹”；
- 中间产物存文件，不要全堆在对话框里；
- 长任务用分段，别一次塞全部。

如何恢复

- 立刻打断；
- 让它停下并汇报当前进度；
- 缩小范围，改成更小的子任务重新布置。

5.17 API 调用成本

实际案例描述

你给 Agent 接了一个付费 API（比如某个检索服务、多模态模型）。它在任务里反复调用，一次任务调用了几百次，费用远超预期。

为什么会发生

Agent 调用 API 的次数你不一定能实时看到。它为了“保险”，可能同一件事调多次，或在循环里反复调用。

如何发现

- API 平台后台用量异常；
- 账单突然增加；
- 任务里有循环/批量处理。

如何预防

- 关键 API 设置**调用次数上限**（平台侧或任务指令里）；
- 付费操作要求它先估计“我打算调多少次”，你批准后再调；
- 优先用便宜模型做批量活，强模型只做关键判断。

如何恢复

- 立刻停任务；
- 在 API 平台设硬限额；
- 把批量任务拆成“先试一小批，确认没问题再全量”。

5.18 Rate Limit（请求频率限制）

实际案例描述

你让它批量查 500 条文献，它每秒发很多个请求，突然报错：“Too Many Requests / 429”，任务中断。

为什么会发生

网站和 API 都有**速率限制（Rate Limit）**：单位时间内只接受一定次数的请求，防止被刷。Agent 不知道这个限制（除非你告诉它），开足马力就会被拒。

如何发现

- 报错信息里出现 429、rate limit、too many requests；
- 任务跑到一半停了；
- 间歇性失败。

如何预防

- 批量任务里让它**放慢速度**：“每批 10 个，批与批之间停几秒”；
- 告诉它“遇到 429 就等待再试，不要立刻重试”；
- 大量请求前查目标网站的使用条款。

如何恢复

- 让它等待一段时间再继续；
- 加入“重试间隔”和“最多重试次数”；
- 把大批量拆成小批次分批跑。

5.19 网络失败

实际案例描述

Agent 正在查资料，网络断了，或某个网站暂时打不开，它的任务卡住或报错。

为什么会发生

网络是不可靠的：断网、DNS 故障、网站临时维护、跨网访问慢。这些都不是 Agent 能控制的。

如何发现

- 报错是连接超时、无法访问、502 等；
- 任务没做完就停了。

如何预防

- 关键资料**先下载到本地**，不要依赖现场联网；
- 让它对每个下载失败的链接**记录下来**，不要整个任务崩掉；
- 把“网络失败”当作正常情况，任务设计成“断了也能存已完成部分”。

如何恢复

- 检查网络恢复后，让它从断点继续（见 5.21 的 Resume）；
- 失败的链接单独列出来，换时间再试；
- 实在下不到的，换来源。

5.20 依赖安装失败

实际案例描述

你让它跑一段数据分析，它要装一个 Python 包，结果安装报错：版本冲突、网络超时、系统不兼容。它卡在“装环境”这一步，业务代码一行没跑。

为什么会发生

代码运行需要**依赖（Dependency）**——别的软件包。这些包之间版本要匹配、要能下载、要和你的系统兼容。零基础用户最容易在这里被卡住。

如何发现

- 报错里出现 `pip install`、`npm install`、`dependency`、`version conflict`；
- 它说“需要安装 XX”然后就没下文了。

如何预防

- 优先用**不依赖安装**的方案（Excel、在线工具）；
- 让它用**常见、稳定**的包，不要追最新版；
- 安装失败时，让它把完整报错贴给你，不要自己反复试；
- 复杂环境交给专业的 Coding Agent 在它的沙箱里解决。

如何恢复

- 贴出完整报错，让它针对性解决；
- 版本冲突时，让它固定一个已知可用的版本；
- 实在装不上，换工具或换方法。

5.21 恢复策略：Backup / Git / Snapshot / Copy / Rollback / Retry / Resume

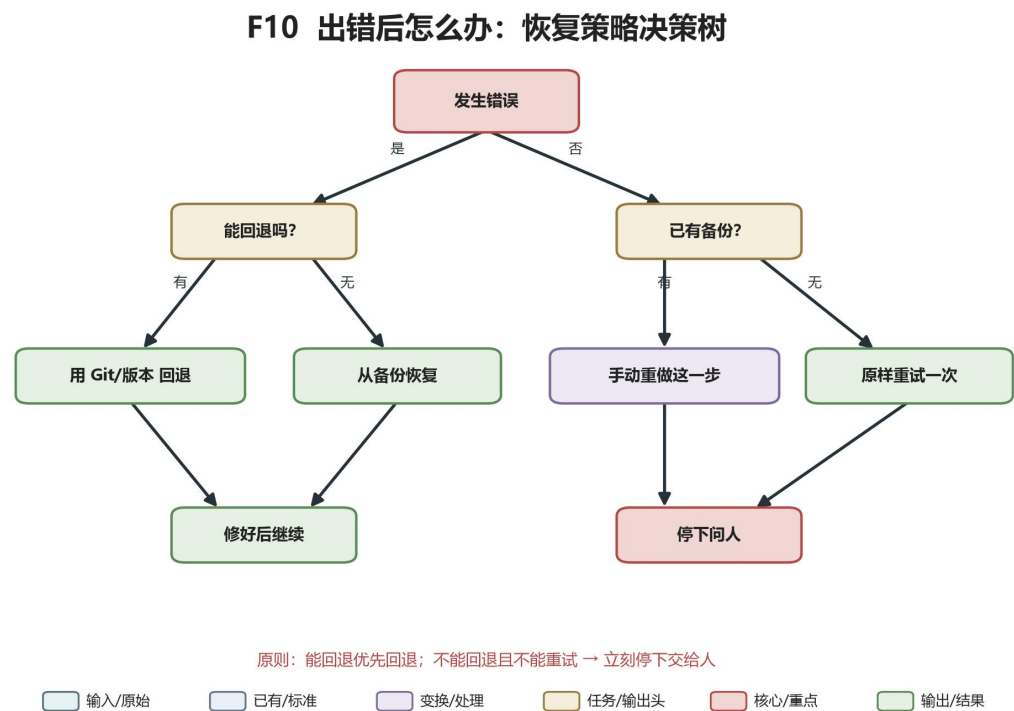


图 11 恢复策略决策树

本章要解决什么问题

前面 20 个失败模式，最后都汇到一个问题：出错了怎么办？本 Part 最后一节教你一套通用的“踩坑之后怎么爬出来”的工具箱。不是让你背术语，而是让你在出事时知道该按哪个按钮。

七件恢复工具（按零基础友好度排序）

工具	一句话解释	什么时候用	零基础怎么上手
Copy（复制）	动手前先把原件复制一份	任何重要操作前	选中文件夹 → Ctrl+C → Ctrl+V 到旁边

工具	一句话解释	什么时候用	零基础怎么上手
Backup（备份）	把整个工作区存一份副本	长任务、批量操作前	任务开始前，把整个项目文件夹复制一份到 备份_日期/
Snapshot（快照）	给当前状态拍一张“照片”，可随时回到这一刻	想多试几种方案时	复制一份当前整个文件夹，起名 snapshot_尝试 1
Rollback（回滚）	回到上一个好的状态	改坏了、覆盖了	用备份或快照把文件换回来
Retry（重试）	失败了再试一次	网络抖动、429、临时错误	让它“等几秒再试，最多 3 次”
Resume（续跑）	从中断处接着做，不回头来	长任务中途失败	把“做到哪了”写在状态文件里，下次让它读这个文件继续
Git（版本控制工具）	专业的修改历史记录，能精确回到任意一次提交	代码、文档的持续修改	第 8 部分 LAB F 会系统教，零基础先知道“它是带历史的备份”即可

核心原则

预防永远比恢复便宜。在你让 Agent 做有破坏性的事之前，先问自己一句：“坏了我能退回来吗？”如果不能，就先备份。

动手前：Copy / Backup / Snapshot
出错时：先停下，别让它继续乱改
恢复时：Rollback（退回上一版）→ Retry（重试）→ Resume（接着跑）
长期： 用 Git 管代码和文档的版本历史

Mission M15：故意制造一个错误并让它修复

字段	内容
Mission 名称	故意制造一个错误并让它修复
目标	完整走一遍“出错→停下→恢复→验证”的流程
难度	L4
预计需要的 Agent 能力	文件读写、代码执行
输入材料	一个小项目文件夹
任务说明	1) 先把整个项目复制一份备份；2) 让 Agent 故意改错一个文件（比如把某个文案改坏）；3) 让它定位错误并修复；4) 用备份对照，确认它真的修好了，且没改坏别的

字段	内容
操作步骤	备份→制造错误→定位→修复→回归验证→对比备份
风险提示	这是练习，务必在副本上做，不要用真实项目
成功标准	错误被定位并修复；周边文件与备份一致；你能说清每一步发生了什么
失败案例	它修好了坏的，但改坏了另一个文件，且没发现
验证方式	和备份逐文件对比
进阶挑战	这次练习中引入 Git，把“改错前”和“修复后”各提交一次

最简单版本

“动手前先复制一份。”就这一个习惯，能挡掉 PART 5 里一半的损失。

完整恢复 workflow

1. 发现异常 → 立刻打断 Agent，不要再让它继续；
2. 判断损失范围 → 它动了哪些文件、改坏了什么；
3. 退回安全状态 →
 - 有备份：用备份覆盖；
 - 有 Git: `git` 回到上一个 `commit`；
 - 都没有：评估能否手动修；
4. 定位原因 → 为什么会错（看是上面 5.1-5.20 哪一类）；
5. 修正做法 → 在任务书里加对应的预防条款；
6. 重新执行 → 从安全状态重新做，这次带着防护；
7. 记录 → 把这次失败和恢复写进自己的“踩坑笔记”，下次复用。

常见失败

1. 出事不打断：让它“再试试”，结果越改越糟。
2. 没有备份就硬上：覆盖了才发现退不回去。
3. 修好了不验证：退完就交，没确认真的好了。
4. 不记录：同样的坑第二次还踩。

如何验证

- 恢复后，产物和“上一个好版本”逐项对比；
- 关键功能重新测一遍；
- 确认没有引入新问题。

本章模板

【破坏性操作前的安全检查】

在让 **Agent** 执行以下操作前：修改/删除/批量处理/覆盖/发布/支付
请先确认：

1. 我是否已经复制了一份备份？（Copy/Backup）
2. 这一步是 L 几风险？需不需要我批准？
3. 如果坏了，我能退回到哪里？
4. 失败后怎么 Resume？

以上任一项答不上来，就不要开始。

延伸资源

- Git 是零基础用户值得早学的一个工具：它本质是“带时间轴的备份”，LAB F 会带你从零用起来；
- 系统自带的“文件历史/以前的版本”、网盘的版本记录，都是零基础可用的 Snapshot；
- 定期（每天/每完成一个里程碑）把项目文件夹压缩备份一次，是最朴素也最有效的 Backup。

完成标准

遇到任何 **Agent** 出错，你能冷静地说出：“先停下，我有备份，我们退回上一版，然后按刚才那条原因加个防护再来。”

PART 5 到此结束。把它当成一本“病历”：每一种失败都是一种病，上面写了症状（如何发现）、病因（为什么发生）、预防（怎么防）和急救（怎么恢复）。

合上书前，再默念一次四大铁律：

代码生成 ≠ 功能完成；代码修改 ≠ Bug 修复；测试通过 ≠ 产品完全正确；**Agent** 声明完成 ≠ 实际完成。

会用 **Agent** 的人，不是从不被它骗的人，而是被骗过一次之后，建立了“要证据、留备份、敢回滚”习惯的人。

PART 6: Agent 到底由什么组成



图 12 Agent 五层认知结构图

本章你要解决什么问题

前面五个 Part，我们一直在讲“怎么把一件事交给 Agent 做”：怎么下任务、怎么管长任务、怎么验收、怎么识别它会怎么失败。但你心里可能一直有个没说出口的疑问：它到底是个什么东西？为什么有的事它能做，有的事它做不了？为什么同一个任务，换个模型、开个插件，结果就不一样？

到这一章，我们才正式打开发动机盖。

请记住一个判断标准：本章不要求你会写任何一行代码，也不要求你背下任何一个术语的英文全称。你需要做到的是——下次听到一个新名词时，能把它对号入座到这张五层图里，知道它大概在解决什么问题，知道它和隔壁概念有什么不一样。这就够了。

一张总览图：Agent 的五层结构

不要被下面这堆名词吓到。它们其实可以压成一句话：

一个 Agent，就是“一个足够聪明的模型”，在“信息”和“手脚”的支持下，把这些能力按“一套编排”组织起来，持续地把一件事做完，最后交出一个结果。

对应到五层，每一层只解决一类问题：



你可以把它想象成一个刚入职的员工：Model 是他的脑子；第二层是他桌上摊着的材料、抽屉里的笔记本、公司资料室、以及他正在做的进度看板；第三层是他本人伸手够得到的电话、打印机、电脑、文件柜；第四层是公司给他办的门卡、接好的专线、买好的软件订阅、以及一份培训手册；第五层是项目经理，把这一切安排成流程、分身和定时任务。

为什么要分五层，而不是像零件清单那样平铺？因为同一个名词，在不同层的含义完全不同。比如“工具”：第三层讲的是搜索、读文件这种**原始动作**；第四层讲的是把这些动作**包装成可插拔单元**。不分层，就会把“一只手”和“一双手套”当成一个东西，越学越乱。

下面 20 节，每一节都用同一套五段模板，只回答这五个问题：

- 1. **概念本身**：它是什么（通用定义，不绑定任何平台）；
- 2. **常见实现**：各平台通常怎么做；
- 3. **不同平台差异**：ChatGPT / Claude / Codex / Gemini / Cursor 这类产品分别长什么样；
- 4. **零基础用户掌握到什么程度**：你要学到哪一步就够；
- 5. **一个实际例子**：拿一件真事说清楚。

第一层：Model（模型）

6.1 Model（模型）

1. 概念本身

Model 就是这个员工的“脑子”本身。所有理解、推理、写句子、做计划、决定“下一步该调用哪个工具”，都是它在干。没有 Model，后面所有工具、记忆、流程都只是一堆不会动的零件。

2. 常见实现

几乎所有 Agent 产品背后都是“一个大语言模型（LLM）+ 若干配套模型”：主模型负责理解与决策，有的产品会再配一个小模型做分类、摘要、路由，或配多模态模型负责看图、听音频。模型本身是按“一次给它一段输入、它吐一段输出”来计费 and 运行的，Agent 是在这个循环外面套上工具和记忆的外壳。

3. 不同平台差异

- ChatGPT、Gemini 这类对话产品：模型选择对普通用户藏在“自动/高级”等档位里，高级用户可在设置里手动切具体型号。
- Claude、Codex 这类偏编程/Agent 的产品：更强调“推理档”和“执行档”分开，常让你在同一个任务里切换或串联不同模型。
- Cursor 这类 IDE 内嵌产品：模型选择直接暴露在编辑器角落，且针对“补全”“对话”“改代码”分别允许选不同模型。
- 底层事实：各平台的模型名单、档位、价格、能力边界变化极快，本节不点名任何具体型号，也不做排行榜。任何排名都活不过一个季度，以你所用平台 2026 年 9 月的官方文档为准。

4. 零基础用户掌握到什么程度

你要学的不是“哪个模型最强”，而是“挑脑子的八把尺子”，每一把对应一类真实痛点：

维度	大白话	什么时候这把尺子重要
Reasoning（推理能力）	能不能一步步把难题想明白，而不是凭直觉瞎答	复杂决策、改 bug、做分析、写论证
Coding（写代码能力）	能不能看懂、写出、改对可运行的代码	让它动文件、做数据处理、改软件
Context Window（上下文窗口）	这一次对话它最多能同时看进去多少字、多少文件	你要一次塞 40 篇论文、一整个文件夹
Tool Calling（工具调用）	它能不能准确地决定调用哪个工具、参数填对没有	只要任务要联网、读文件、操作外部系统，这一条就是命门
Computer Use（电脑操作）	它能不能看懂屏幕截图、移动鼠标、点按钮	需要它操作没有 API 的老式网页或桌面软件
Multimodal（多模态）	它能不能看图、看表、看截图、看音频	你要给它 PPT 截图、手写笔记、图表照片
Latency（响应速度）	它多久回你一句	你在等它连续操作，或者做即时对话

维度	大白话	什么时候这把尺子重要
Cost（成本）	每用一次花多少钱	长任务反复跑、自动化每天跑，账单会堆起来

更省事的思路：**简单任务用快的、便宜的；难任务再切到强推理的；做检查的那一步，最好换一个和执行者不同的模型。** 知道“原来可以换”，就已经超过绝大多数用户了。

5. 一个实际例子

让 Agent 改一段跑不通的代码，和让它写一封客气的请假邮件，用的模型往往不是同一个。前者要 Reasoning 和 Coding 都强；后者反应快、便宜、语气自然的模型就够。下次任务没做好，先别急着重写 Prompt，先判断是不是“这次该换个脑子”。

第二层：记忆与上下文

这一层解决的是“信息”问题：Agent 这一次睁眼能看到什么、关了机还记得什么、要查自己资料库时去哪查、任务做到哪一步了。四个概念从“临时”到“长期”、从“现成”到“检索”，正好排成一个连续谱。

6.2 Context（上下文）

1. 概念本身

Context（上下文）就是模型“这一次睁眼时，眼前摊着的全部东西”：你这轮打的字、它刚才的回答、你拖进来的文件、搜索到的网页、它刚读出来的文件内容。它只能基于眼前这些东西回答。模型自己“记不住你电脑里有什么”，也不知道你是谁。

2. 常见实现

平台把当前对话里的全部内容压进一个“窗口”里发给模型。窗口有上限（即 Context Window），装不下时，产品通常会自动截断最旧的消息、或把旧内容总结成摘要再塞回去。你拖进去的文件、它读到的网页，最终也都是被转成文字、塞进这个窗口。

3. 不同平台差异

有的产品窗口大、长对话不太需要你管；有的窗口小，多聊几轮就开始“忘事”，需要你手动开新对话或重新贴材料。编程类产品（如 Codex、Cursor）往往会主动把整个项目文件结构读进来当 Context，对话类产品则默认只放当前对话。差异不在“有没有 Context”，而在“它替你自动塞了多少、超过了窗口怎么处理”。

4. 零基础用户掌握到什么程度

记住一句话：**你不放进 Context 的东西，等于不存在。** 这也是 PART 5 讲过的“上下文污染”和“最小充分上下文”的根源——眼前堆太多会被带偏，给太少它会瞎编。

5. 一个实际例子

你让它“帮我改这篇论文的引言”，就必须把论文全文（至少引言）放进 Context，否则它只能凭空写一段漂亮但和你论文无关的话。

6.3 Memory（记忆）

1. 概念本身

每次新开对话，模型都是“失忆”的。Memory（记忆）就是平台帮你跨对话保留下来的那部分信息：你偏好的格式、你常用的背景、上次做到哪一步、它帮你总结过的笔记。这样你不用每次从头自我介绍。

2. 常见实现

两种主流做法：一种是产品自动把你说过的偏好存成“长期记忆”，下次自动带进来；另一种是你自己在项目里放一个说明文件（如 README、AGENTS.md 这类“岗位说明书”），让它每次开工前先读。后者本质上是你手写的 Memory，比自动记忆更可控。

3. 不同平台差异

有的对话产品有显式的“个人记忆”开关和查看页，你能看到它记了你什么、能随时删；有的编程产品完全靠项目里的说明文件，没有自动记忆。前者省心但可能记错，后者啰嗦但完全透明。没有哪种绝对更好，取决于你愿不愿意自己掌控。

4. 零基础用户掌握到什么程度

会检查“它这次记得的，是不是还是对的”。Memory 的风险是会记错、记偏、把过时信息一直带着。发现它拿旧印象套新任务，直接纠正。重要背景宁可每次在任务里明说，也别全押在“它应该记得”上。

5. 一个实际例子

你告诉过它“我是马克思主义理论方向、在做计算社会科学”。自动记忆型产品下次会自动带上这个背景；说明文件型产品则靠你在项目里写一句。两种方式目的一样：让它不用每次重新认识你。

6.4 RAG（检索增强生成）

1. 概念本身

RAG（Retrieval-Augmented Generation，检索增强生成）的做法是：先把你的资料切块存进一个“小图书馆”，每次提问时，先从里面挑出最相关的几段，再把这几段喂给模型回答。它解决的是“资料太多，Context 装不下”的问题。

2. 常见实现

平台先把你的文档切成小块、做索引（通常是把文字转成向量，再按相似度检索）；提问时先跑一次检索，把最相关的几段抓出来，连同你的问题一起放进 Context。所以 RAG 不是“让模型记住你全部资料”，而是“每次临时把相关几段翻出来给它看”。

3. 不同平台差异

有的产品提供现成的“知识库”功能，你拖文件进去它就建好索引，问答时自动检索；有的需要你自己搭索引、自己配检索逻辑。前者面向普通用户，后者面向有技术能力的人。差异在“谁来建索引”，原理都是“先检索、再生成”。

4. 零基础用户掌握到什么程度

会要求它每次回答都标“这段依据来自哪份文件哪一段”。RAG 最大的风险是检索出来的片段不对题，标不出来来源的，不要信。

5. 一个实际例子

你把一学期 30 份 PPT 灌进 RAG，之后问“第三章讲的那个模型有什么局限”，它会先从这 30 份里找出第三章相关段落再回答，而不是凭训练记忆瞎编。

6.5 State（状态）

1. 概念本身

State（状态）是任务进行中的中间信息：现在做到第几步了、已经试了哪几条路、哪些文件已经改过、当前假设是什么。它介于 Context 和 Memory 之间——比一次对话的 Context 活得长，又不像 Memory 那样是“关于你”的长期事实，而是“关于这件事进行到哪了”的进度记录。

2. 常见实现

长任务型 Agent 通常会维护一份“工作记录”：已完成的步骤、待办、中间产物路径、失败过的尝试。有的把它写进项目里的一个进度文件，有的存在平台自己的后台，对话中断后下次接着跑时先读这份记录。

3. 不同平台差异

偏“一次性对话”的产品基本没有显式 State，聊完就结束；偏“长任务/异步任务”的产品会有任务列表、断点续跑、中间产物留存。差别在于：这个任务是你盯着它一次做完，还是它挂在后台自己跑几小时。

4. 零基础用户掌握到什么程度

长任务里，学会时不时问一句：“你现在做到哪一步了？已经完成了什么？下一步打算干什么？”让它把 State 说出来，你才不会以为它停了、或重复劳动。

5. 一个实际例子

让它整理 200 个文件。中途它崩了，有 State 的 Agent 下次开工能告诉你“已经处理完前 80 个，从第 81 个继续”；没有 State 的只能从头再来。

第三层：感官与操作

这一层是 Agent 直接碰现实世界的七只“原始手”。它们和第四层的区别在于：这里讲的是动作本身，是未包装的原子能力。你可以把它们理解成七只手，每只手擅长一类接触。

6.6 Search（搜索）

1. 概念本身

模型的训练数据有截止日期，也不可能背下全世界所有网页。Search（联网搜索）就是让它像你打开搜索引擎那样，去网上查最新的、具体的、它训练时没见过的信息。

2. 常见实现

平台接一个搜索服务，模型决定需要查时，发一个关键词，拿回一串链接和摘要，再自己读其中几条。本质是“输入关键词，看返回的链接和摘要”。

3. 不同平台差异

有的产品搜索默认常开，模型自己决定什么时候查；有的默认关，要你显式说“搜一下”；有的还接了学术库、代码库这类垂直搜索。差异在“默认开不开、接了哪几种搜索源”。

4. 零基础用户掌握到什么程度

知道“它给的每个事实性结论，最好都带回来源链接”。凡是涉及“今天的事实”——最新政策、刚发布的论文、某软件现在的版本号——都该让它搜。没带来源的事实，一律当成待核验。

5. 一个实际例子

让它查“2026 年某领域最新的综述”，它必须真的去搜学术库，而不是凭训练记忆列几篇三年的老文章。

6.7 Browser（浏览器）

1. 概念本身

很多信息藏在搜索引擎返回不了的地方：需要登录的后台、要点几下才展开的表格、要翻页的论坛帖子、要点“加载更多”的评论区。Browser（浏览器控制）就是让 Agent 真的打开一个浏览器，像人一样看屏幕、点鼠标、敲键盘。

2. 常见实现

平台内置一个受控浏览器，模型看到网页截图或结构化内容，决定点哪个按钮、填哪个框。活动范围通常限定在浏览器标签页内。

3. 不同平台差异

有的产品把浏览器操作做成一键工具，点了就用；有的需要你开“网页操作”权限；有的能让你看到它正在操作的实时画面，有的在后台静默跑。越透明越可控。

4. 零基础用户掌握到什么程度

当 Search 返回的摘要不够用、必须进到网站里把内容“读出来”时，才用 Browser。它在真操作网页时，你应该在旁边看着第一步点的对不对，尤其是涉及登录、下单、发消息的页面。

5. 一个实际例子

“把某份公开榜单前 50 名连同公司名、成立时间、融资轮次抓成一张表”——搜索做不到（结果分散在十几页），浏览器可以逐页翻。

6.8 Computer Use（电脑操作）

1. 概念本身

有些软件根本没有网页版、没有 API，只有桌面客户端，或是一个很老的系统。Computer Use（电脑操作）就是让 Agent 直接看你的整个屏幕、移动鼠标、敲键盘，像一个远程坐在你电脑前的人一样操作任何软件。

2. 常见实现

平台定期截屏给模型看，模型输出“移到某坐标点一下、敲某段字”，系统执行。它的活动范围是整个桌面，不限于浏览器。

3. 不同平台差异

有的产品提供专门的“电脑操作”模式，屏幕共享给模型；有的把它和浏览器操作合在一起；有的在隔离的虚拟机里跑，不碰你本机。能力越强，越要在隔离环境里开。

4. 零基础用户掌握到什么程度

第一次开这个能力时全程盯着屏幕。它动鼠标就是真的动你的鼠标，误点一个“确认删除”就是真删。和相邻两只手的边界要清楚：

能力	活动范围	自由度	风险
Browser	只在浏览器标签页里	中	中
Computer Use	整个桌面、任何软件	最高	最高
Code Execution	隔离的代码沙箱	受限	较低

高风险操作前先让它停下来问你。

5. 一个实际例子

让它打开你电脑上某个只能用 Windows 客户端的老系统，按你说的步骤改尺寸、导出图片。它真的会移动你的鼠标——你会在屏幕上看到光标自己动。

6.9 File System（文件系统）

1. 概念本身

你的论文、PPT、Excel、下载的资料都存在硬盘的文件夹里。File System（文件系统）访问就是让 Agent 能读你电脑上的文件、新建文件、改文件、整理文件夹。没有它，你只能一个个手动把文件拖进对话框。

2. 常见实现

平台给它一组受限的文件操作接口：列目录、读文件、写文件、移动文件。通常先让你授权“允许访问哪个文件夹”，而不是整台电脑随便读。

3. 不同平台差异

有的产品只让它访问你主动拖进去的文件；有的允许它访问整个项目文件夹；有的默认隔离沙箱，碰不到你本机真实目录。差异在“授权范围多大”，原则都是越窄越好。

4. 零基础用户掌握到什么程度

永远告诉它“只在这个文件夹里工作，不要碰别的地方”。原始材料留备份，让它去改副本。它和 Terminal 的区别：文件工具通常只做“读、写、列文件夹”这类相对安全的操作，Terminal 是更底层、更通用的命令通道。

5. 一个实际例子

PART 1 的第一个 Mission“整理混乱的下载文件夹”，靠的就是这只手：它真的去建文件夹、移动文件，而不只是口头建议。

6.10 Terminal（终端 / 命令行）

1. 概念本身

Terminal（终端，也叫命令行、Shell）是你平时用鼠标点图标之外，另一种指挥电脑的方式：直接敲一行文字指令，电脑照做。Agent 要装软件、跑脚本、看系统状态、动 Git，很多时候靠敲命令完成。

2. 常见实现

平台给它一个命令行通道，它生成一行命令，系统执行并把输出回给它。它面对的通常是你整台电脑的真实环境，而不是隔离沙箱。

3. 不同平台差异

编程类产品（Codex、Cursor 等）几乎把 Terminal 当标配，因为跑项目、装依赖、跑测试都要它；普通对话产品一般不给 Terminal，或放在一个隔离环境里。有 Terminal 的产品，风险也最高。

4. 零基础用户掌握到什么程度

你自己一般不需要学敲命令。你需要会问：“你这条命令打算干什么？”尤其是带删除、覆盖、联网下载含义的命令，让它解释一遍再放行。它和 Code Execution 的区别：Terminal 在真实系统里，Code Execution 在隔离小房间里。

5. 一个实际例子

让它跑一个开源项目，它可能敲一行把项目下载下来，再敲一行把依赖装好，再敲一行启动。中间任何一行写错，影响的都是你的真实环境。

6.11 Code Execution（代码执行）

1. 概念本身

遇到算数、统计、画图表、清洗 Excel、批量处理文件这类活，让模型“心算”是靠不住的——它会算错、会编数字。Code Execution（代码执行）是给它一个隔离的小环境，让它真的写一小段代码、真的运行、真的把结果打印出来。结果是跑出来的，不是它猜的。

2. 常见实现

平台起一个临时沙箱（内置 Python 等运行环境），模型写代码、执行、看报错、改、再跑，最后把结论和图表交回来。沙箱用完即销毁，通常动不到你本机的重要文件。

3. 不同平台差异

几乎所有主流产品都带某种代码执行，差别在沙箱里预装了哪些库、能不能联网、能跑多久。数据类产品往往预装了一整套数据分析库，编程类产品更灵活但默认更干净。

4. 零基础用户掌握到什么程度

任何涉及精确数字的活——数据清洗、回归分析、画统计图、批量改名、合并表格——都让它用代码执行，而不是“看着表格估算”。并要求它把“跑出来的代码和结果”都留在工作文件夹里，方便你回头复查：代码能重跑，结果才算数。

5. 一个实际例子

你丢给它一个 5000 行的问卷 Excel，让它“按学院分组算平均分并画图”。它写一段代码跑出来，比让它心算可靠得多。

6.12 API（应用程序接口）

1. 概念本身

你想用某款软件的功能，但不想打开它的界面、不想手动点。API（Application Programming Interface，应用程序接口）就是软件之间“对话用的标准窗口”：你不用进厨房，通过传菜口下单，菜从窗口递出来。

2. 常见实现

软件厂商对外开一组“接口地址”，你用一串身份凭证（API Key）去调用，它返回数据或执行动作。Agent 要读日程、查数据库、调翻译，背后都是拼一个 API 调用。

3. 不同平台差异

普通对话产品一般不让你直接手写 API 调用，而是把常见服务封装成 Connector 或 Plugin；编程类产品允许它自己写代码去调任意 API。前者对你友好，后者灵活但要你自己管密钥。

4. 零基础用户掌握到什么程度

知道“它能调 API，就意味着它能以你的身份去读或写那个软件里的东西”。授权前想清楚：它能读哪些？能改哪些？它和第四层 Connector 的区别：API 是技术通道，需要自己填密钥、自己拼调用；Connector 是把这种通道包装好、给普通用户用的成品。

5. 一个实际例子

让 Agent “把你飞书日历里明天的会列出来”，它就是通过飞书开放的 API，以你的身份去读日历数据。

第四层：扩展能力

第三层是“原始的手”，这一层是“把手标准化、产品化、可插拔的包装”。五个概念抽象程度递增：Tool 是最小单元，Connector 接你的账号，Plugin 是成品功能，MCP 是通用插座标准，Skill 是教它怎么做专业活的手册。

6.13 Tool（工具）

1. 概念本身

没有工具的 Agent 本质上还是一张“嘴”：它能写、能分析、能出主意，但碰不到任何真实的东西。Tool（工具）就是给它接上的、可被它调用的一个个具体动作单元：一个搜索工具、一个读文件工具、一个跑代码工具。第三层那七只手，到了这一层就变成一个个有名字、有参数的 Tool。

2. 常见实现

每个 Tool 对外暴露三样东西：名字、它要什么参数、它干什么。模型在每一步决定“要不要调某个 Tool、参数填什么”，系统执行后把结果还给模型。所以“工具调用”是模型和现实世界之间的标准握手。

3. 不同平台差异

编程类产品（Codex、Cursor）把工具细到“读哪个文件、跑哪条命令”；对话类产品把工具做得更粗粒度（“联网搜索”“画图”）。差异在“工具切得多细”，但握手方式都一样。

4. 零基础用户掌握到什么程度

知道“它每说一句要干活，背后必然调用了某个具体工具”，并会在它调用时瞄一眼“它现在碰的是哪个东西”。Tool 是抽象大类：Search、Browser、File System、API，以及下面的 Connector、MCP，本质都是 Tool 的具体形态。

5. 一个实际例子

你说“帮我把这个文件夹按类型分好类”，没有文件 Tool 它只能口头建议；有了 Tool，它才真的去建文件夹、移动文件。

6.14 Connector（连接器）

1. 概念本身

让普通用户也能用 API，不用懂技术。Connector（连接器）就是平台提前做好的“数据线”：你在界面上点一下“连接邮箱”“连接日历”“连接云盘”，走一个登录授权，Agent 就能读你邮箱、看你日程、动你云盘。

2. 常见实现

它通常走 OAuth 授权（见 PART 7 第 7.9 节）：点连接 → 跳到对方账号的官方登录页 → 列出要哪些权限 → 你同意 → 平台拿到一张有时效的通行证。技术细节被包起来，你只看到一个按钮。

3. 不同平台差异

各平台官方对接的服务清单不同：有的接了邮箱、日历、云盘全家桶；有的只接了少数几家；有的完全不做 Connector，全靠 MCP。差异在“官方帮你接好了哪几家”。

4. 零基础用户掌握到什么程度

每接一个 Connector，都去它的授权页看一眼：“它现在能读我哪些东西？能替我做哪些事？”看完再点同意。它和 Plugin、MCP 的区别：Connector 偏“把你个人的某个账号接进来”，通常走登录授权；Plugin 偏“加一类功能”；MCP 偏“一套通用的、谁都能插的标准插座”。三者在不同平台经常混用，别纠结名字，看它实际碰了什么数据。

5. 一个实际例子

点一下连接你的邮箱账号，Agent 就能每天早上帮你看新邮件、归类、起草回复——但它能不能“替你真的发出去”，取决于你授权时勾没勾“发送”那一项。

6.15 Plugin（插件）

1. 概念本身

Plugin（插件）是平台打包好的一个“功能模块”：装一个幻灯片插件，Agent 就能读 PPT、写 PPT；装一个 PDF 插件，它就能拆 PDF、合并 PDF。本质是把一组相关的工具和使用说明打包在一起，让你一键开启。

2. 常见实现

插件通常来自平台官方商店，你在界面上点“安装”，它就挂上了，背后常连的是平台认证过的服务。有的插件是纯本地功能，有的一装上去就开始调某个在线服务。

3. 不同平台差异

早年对话产品的“插件商店”形态最典型；编程类产品里，类似能力更多以 Skill 或 MCP 的形式出现，而不叫 Plugin。到 2026 年，Plugin、Skill、MCP 三者边界仍在快速融合，以你所用平台的界面为准——界面上叫什么，你就当它是什么。

4. 零基础用户掌握到什么程度

装之前看三样：这个插件要什么权限、谁做的、最近还更不更新。不要看到“热门”就点。它和 Skill、MCP 的区别：Plugin 是商店式成品，装好就用；Skill 是一份“操作手册加工具箱”；MCP 是底层通用连接标准。

5. 一个实际例子

让它处理一个扫描版 PDF，纯文本模型读不了图；装一个 PDF 处理插件后，它就能识别里面的文字和表格。

6.16 MCP（Model Context Protocol，模型上下文协议）

1. 概念本身

早年每做一个工具，都要为每一个不同的 Agent 单独适配一遍——像每个国家的插头形状都不一样。MCP（Model Context Protocol）是 2024 年底开始被广泛采用的一套“万能插座标准”：任何按 MCP 标准写的工具（叫一个 MCP Server），理论上任何支持 MCP 的 Agent 都能直接插上用。

2. 常见实现

你在 Agent 的配置文件里写一段：“我要用哪个 MCP Server、它怎么启动、钥匙放哪”。重启后，这个 Server 暴露的工具就自动出现在 Agent 的工具列表里。一个 MCP Server 可以被多个不同的 Agent 共用。

3. 不同平台差异

编程类产品（Claude 系、Codex 系等）对 MCP 支持最积极，配置写在各自的配置文件里；对话类产品有的支持、有的还不支持；MCP 的官方注册中心和客户端配置方式各平台不一。具体怎么写配置，以你所用客户端 2026 年 9 月的官方文档为准，不要照抄网上三年前的教程。

4. 零基础用户掌握到什么程度

知道“MCP Server 也是一段会跑的代码”，所以和装任何社区脚本一样，要先看它是谁做的、要什么权限。不要因为它“叫标准协议”就放松警惕。用一个类比收尾：API 是“传菜口”，

MCP 是“传菜口的统一规格”，Connector 是“餐厅已经帮你接好的成品线”，Plugin 是“餐厅直接给你上的一桌菜”。MCP 本身不解决某个具体问题，它解决的是“别再一对一接线了”。

5. 一个实际例子

你配一个 GitHub 的 MCP Server，之后支持 MCP 的 Agent 就能读你的仓库、看 issue、查代码，不用你教它怎么调 GitHub API。完整安装演示见 PART 7 第 7.7 节。

6.17 Skill（技能）

1. 概念本身

教 Agent 做某件专业的事，光靠一次 Prompt 说不清楚。Skill（技能）就是把“怎么做这件事”写成一个可复用的包：里面有一份说明书（通常叫 SKILL.md）、一套模板、有时还附带现成脚本和示例素材。装上之后，Agent 遇到同类任务就按这套打法来。

2. 常见实现

一个 Skill 就是一个文件夹，核心是那份 SKILL.md（纯文本说明书），外加模板、脚本、示例。你把文件夹放进 Agent 的 skills 目录，它遇到相关任务时先读说明书再动手。它和 Plugin 的区别是：Plugin 是商店里点一下的成品功能，Skill 更像你从社区下载的一个文件夹，要自己放、自己读。

3. 不同平台差异

不同 Agent 产品对 Skill 的目录位置、加载方式命名不同（有的叫 skills 目录，有的叫别的），但“一个文件夹加一份 SKILL.md”的形态高度一致。编程类产品普遍支持用户自建 Skill；对话类产品更多用商店式 Plugin 替代。

4. 零基础用户掌握到什么程度

会打开一个 Skill 的 SKILL.md，读完前三段就知道“它想让 Agent 干什么、要碰什么文件、要联网吗”。当你发现“这件事我每次都要跟它解释半天、而且每次解释还不一样”时，就该把它沉淀成一个 Skill。完整安装与审计演示见 PART 7 第 7.2 节。

5. 一个实际例子

一个“学术综述写作 Skill”可能规定：先建证据矩阵、再按研究脉络分节、所有引用必须回原文核验、最后附未解决问题清单。装上它，你以后只要说“用综述 Skill 处理这批论文”，它就按这套流程走。

第五层：编排

前四层是零件，这一层是“项目经理”：把零件组织成一次真实任务。三个概念从“固定步骤”到“派分身”再到“定时自动跑”，抽象程度递增。

6.18 Workflow（工作流）

1. 概念本身

复杂任务不是一句话说完的，而是一连串步骤：搜、筛、读、建表、分析、写、查。Workflow（工作流）就是把这一串步骤固定下来，下次同类任务直接按这条流水线走，不用每次重新指挥。

2. 常见实现

有的产品提供可视化“拖节点连线”的工作流编辑器；有的就是你自己写在 Prompt 或说明文件里的一套固定步骤；有的是 Skill 里规定的流程。本质都是“第一步出什么、第二步凭什么、最后怎么验收”的箭头图。

3. 不同平台差异

偏自动化的产品有专门的工作流画布，节点、条件、循环都能配；普通对话产品没有显式 Workflow，靠你在任务里口述步骤。差异在“有没有把流程显式画出来”，脑子里的流程和画布上的流程，原理相同。

4. 零基础用户掌握到什么程度

当你发现自己每周都在重复同一套多步操作时，就该把它画成一张箭头图：第一步出什么、第二步凭什么、最后怎么验收。画清楚了，Workflow 就设计出来了。它和 Automation、Sub-agent 的关系见下面两节。

5. 一个实际例子

“每天早上 8 点，搜今天的行业新闻，和昨天的摘要对比，有重大变化才把摘要推给我”——这就是一条固定 Workflow。

6.19 Sub-agent（子智能体）

1. 概念本身

一个 Agent 自己干长了，会顾不过来、会忘、会串味。Sub-agent（子智能体）就是主 Agent 临时派出去的“分身”：一个负责搜资料，一个负责读文件，一个负责写初稿，最后主 Agent 把三份结果汇总。

2. 常见实现

主 Agent 把一个子任务连同必要的上下文交给一个新的、独立的 Agent 实例去跑，跑回来一份摘要或产物，主 Agent 合并。分身之间默认互不可见，只通过主 Agent 传话。

3. 不同平台差异

有的产品有显式“派子任务”机制；有的靠你手动开多个对话再自己汇总；编程类产品常把不同文件、不同模块分给不同子 Agent。差异在“分身是系统帮你派，还是你自己拆”。

4. 零基础用户掌握到什么程度

不是任务越复杂就越要开一堆分身。分身越多，汇总成本和出错率越高。新手先把单 Agent 跑顺，再考虑拆分。并且记住：派出去的分身，交回来的东西也要验收——它不会因为叫“子 Agent”就自动可靠。

5. 一个实际例子

主 Agent 接到“调查 X 公司”，派子 Agent A 查工商信息、B 查新闻舆情、C 查产品；A、B、C 各交一份摘要，主 Agent 合并成最终报告。

6.20 Automation（自动化）

1. 概念本身

最理想的状态是：这件事不用你每天去说，它到点自己跑、有事自己通知你、没事不打扰。Automation（自动化）就是把 Workflow 接上一个“闹钟”或“触发器”：定时跑、收到邮件跑、网页变了跑。

2. 常见实现

平台提供一个触发器（每天几点、某个事件发生）和一个动作（跑某条 Workflow、把结果发给你）。中间的 Workflow 照常是第四层、第三层那些能力在跑。

3. 不同平台差异

有的产品有专门的“自动化/定时任务”页，能设 cron 式时间表；有的只能在工作流画布里接触发器；普通对话产品基本没有 Automation。差异在“触发器好不好设、能不能无人值守”。

4. 零基础用户掌握到什么程度

第一版自动化一定先设成“跑完先把结果给我看，别自己发出去”。它和 Workflow 的关系：Workflow 是“步骤怎么排”，Automation 是“什么时候自动跑这条 Workflow”——前者是菜谱，后者是定时器。跑稳两周、你确认它不会乱来，再逐步放开发送权限。

5. 一个实际例子

每天早上抓一次 arXiv 上你关心方向的新论文，过滤后给你发一份 5 条摘要；有新论文才打扰，没有就静默。注意：这种事有规律、触发条件明确、且低风险，才适合全自动；高风险、需要你每次判断的事不适合。

本章 Mission

学完这一章，你不需要背术语，但要能完成下面三个任务。

Mission 6-A: 给你手上的一个 Agent 画一张“五层构成图”

拿你最近在用的那个 Agent，从第一层到第五层逐层回答：它用的是哪个 Model、看重哪把尺子？第二层里它这一轮能看到哪些 Context、有没有 Memory、要不要 RAG、长任务有没有 State？第三层它开了哪几只手（搜索、浏览器、文件、终端、代码、API）？第四层它装了

哪些 Tool / Connector / Plugin / MCP / Skill？第五层它有没有 Workflow、会不会派 Sub-agent、能不能 Automation？你会立刻发现自己之前一直在“裸用”，很多能力其实没开。

Mission 6-B: 判断一次失败到底卡在哪个层

回想最近一次它没做好的任务。对着这五层判断：是第一层 Model 不够聪明（Reasoning/Coding 不行）？还是第二层信息没给够（漏了材料、该检索没检索）？还是第三层手没开（该让它搜它没搜）？还是第四层缺了某个扩展（该接的服务没接）？还是第五层编排乱了（步骤没串好）？定性之后，下次任务对症下药，而不是只会重写一遍 Prompt。

Mission 6-C: 分清三对最容易混的概念

用自己的话写三句话：Context 和 Memory 差在哪？Search 和 Browser 差在哪？MCP 和 Connector 差在哪？写不出人话版，就回去重读对应那一节。这三对分清了，这一章就过关了。

PART 7: 给 Agent 安装和扩展能力

F15 Skill 安装安全审查：12 问流程

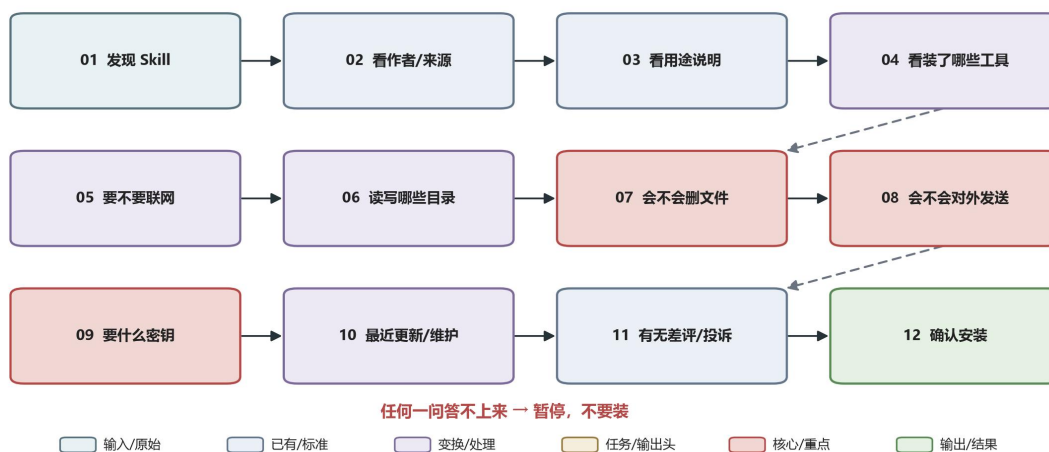


图 13 Agent 能力扩展安装与授权流程图

本章你要解决什么问题

上一章讲清楚了 Agent 由哪些零件组成。本章解决一个非常实际的问题：**这些零件不是出厂全装好的，很多要你自己去“插上去”**。就像新手机买回来只能打电话，导航、外卖、修图都得你自己装 App。

但这里有一条贯穿全章的铁律，请你先记住：

社区里的 Skill、Plugin、MCP、Connector，不能因为“很多人装”你就装。GitHub 上的 Star 数不等于安全，商店里的“热门推荐”不等于对你合适。每装一个，你都要知道它碰了什么数据、能干什么动作、钥匙放在了哪里。

7.1 从“模型能力”到“Agent 能力”

F12 能力扩展层级：Tool → Plugin → MCP → Skill

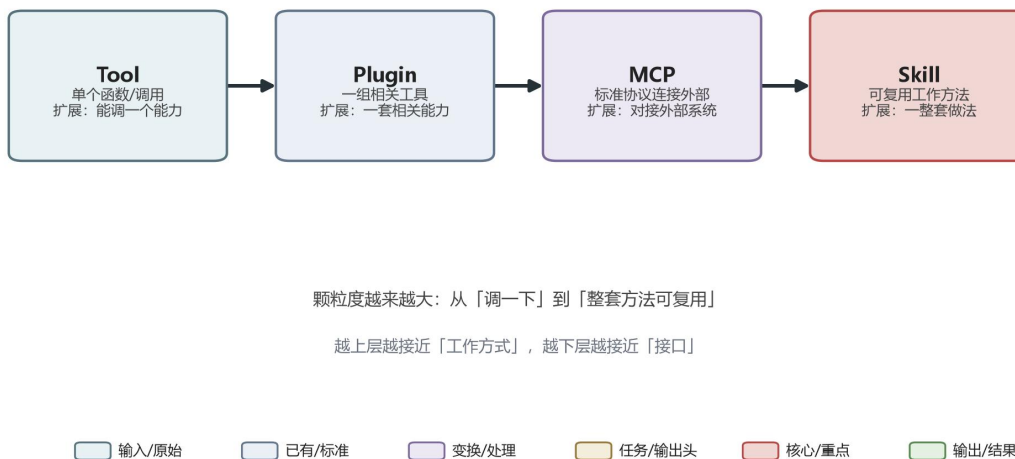


图 14 能力扩展层级

模型出厂时只会做一件事：读一段文字，吐一段文字。它不能读你电脑里的论文，不能查今天的新闻，不能动你的 Excel，也不能替你发邮件。这些“超能力”全是后来装上去的。

所以判断一个 Agent 到底强不强，不要只看它用了多牛的 Model，而要看：

- 它接了哪些 Tool（搜索？文件？浏览器？代码？）；
- 它装了哪些 Skill / Plugin / MCP；
- 它连了哪些你的账号（邮件？日历？云盘？GitHub？）；
- 你给了它多大的操作权限。

模型决定天花板，扩展能力决定它能不能下地干活。 一个用中档模型但工具齐全、权限合理的 Agent，往往比一个用顶级模型却“赤手空拳”的 Agent 有用得多。

7.2 安装 Skill

Skill 的安装在各平台上长得不一样，但通用动作就四步：

1. **找到它**：从平台官方 Skill 商店、官方 GitHub 组织、或你信任的 Skill 目录里找到一个具体的 Skill，记下它的来源地址。
2. **读它的说明书**：每个 Skill 根目录下都有一个 SKILL.md 文件，这是它的脸，下一节专门讲怎么读。

3. **放进指定位置**：把这个 Skill 文件夹放到你的 Agent 配置目录里——通常是一个叫 skills 的文件夹，不同平台路径不同，以该平台文档为准。
4. **让 Agent 认识它**：重启或刷新 Agent，或者直接对它说“你现在有一个叫 XX 的 Skill，请先读它的 SKILL.md”。第一次用之前，让它先自己读一遍说明书。

7.2.1 一次完整安装演示（以本机 Claude 系 skills 目录为例）

下面用一台真实 Windows 机器上的真实目录走一遍，让你看清每一步长什么样。**注意：**本节命令路径是这台机器的真实情况，你照做时要换成你自己的用户名和平台路径；凡标注“本环境无法替你完成”的步骤，请在你自己的 Agent 产品里照做。

第一步：发现。 在本机的 skills 目录里，每个子文件夹就是一个 Skill。用一条命令就能看清单：

```
Get-ChildItem "$env:USERPROFILE\.claude\skills" -Directory | Select-Object Name
```

这台机器上真实存在的 Skill 包括 arxiv-research、minimax-pdf、find-skills、skill-creator 等。假设你想找一个“处理 PDF”或“做文献研究”的 Skill，关键词是 pdf、research：

```
Get-ChildItem "$env:USERPROFILE\.claude\skills" -Directory | Where-Object { $_.Name -match 'pdf|research' } | Select-Object Name
```

本机真实命中：minimax-pdf（pdf）、arxiv-research（research）。如果你是从 SkillsMP、skills.sh 这类聚合站或官方 GitHub 组织发现的，做法一样：记下它的来源仓库地址。

第二步：源仓库核验。 打开这个 Skill 对应的 GitHub 仓库（不是只看聚合站的卡片），用眼睛或让 Agent 帮你看四样东西：README 讲清楚了没有、有没有 License 文件、最近一次提交是什么时候、Issue 区有没有人报安全问题。本机可以用命令看一个已装 Skill 的最后提交时间：

```
git -C "$env:USERPROFILE\.claude\skills\minimax-pdf" log -1 --format="%ci %s"
```

说明：上面这条只在该 Skill 本身是 git 仓库时才有输出。如果你的 Skill 是直接下载的文件夹、不是 git clone 下来的，这一步就打开 GitHub 网页看“最近提交”那一栏。本环境不对第三方仓库的安全性下任何结论，核验动作由你本人完成。

第三步：权限审计。 打开它的 SKILL.md，重点看有没有脚本、会不会执行 Shell、会不会往你电脑写文件、要不要联网。本机真实操作：

```
Get-Content "$env:USERPROFILE\.claude\skills\minimax-pdf\SKILL.md" -TotalCount 60
```

再看一眼它带了哪些文件、有没有可执行脚本：

```
Get-ChildItem "$env:USERPROFILE\.claude\skills\minimax-pdf" -Recurse | Select-Object
FullName, Length
```

判断标准回到 7.3 节那张表和 7.4 节的 12 问：出现“联网下载”“删除文件”“读环境变量外发”这类字眼，先打问号。

第四步：安装。如果是从 GitHub 下载的新 Skill，就是把整个文件夹放进 skills 目录。本机这台机器上，目标目录就是 `C:\Users\lauze\.claude\skills`（对应 `%USERPROFILE%\.claude\skills`）。如果你用的是别的 Agent 产品，这个目录名可能不同，以它的文档为准。

第五步：测试。重启或刷新你的 Agent 产品，对它说：“请读一下你 skills 目录里的 `minimax-pdf`（或你新装的那个）`SKILL.md`，然后用它处理我这个 PDF。”再丢一个小 PDF 让它试一次。

标注：“在你自己的 Agent 产品里加载并测试”这一步，本教程运行环境无法替你完成——因为那是另一个图形化产品里的交互，不在这台命令行机器上。你需要在自己的产品里实际开一次，确认它真的认识这个 Skill、真按说明书走了。

第六步：验证卸载无残留。装完不满意要撤，就是删掉那个文件夹，再确认没有别的残留：

```
Remove-Item "$env:USERPROFILE\.claude\skills\minimax-pdf" -Recurse -Force
Test-Path "$env:USERPROFILE\.claude\skills\minimax-pdf"
```

第二条命令应返回 `False`，说明文件夹已删干净。如果这个 Skill 之前还在某个配置文件里登记过、或连过你的账号，记得按 7.13 节再去配置里移除、去对应账号撤销授权——光删文件夹不算完整卸载。

新手最容易踩的坑：从某个聚合网站上看到一个“排名很高”的 Skill，直接一键安装，从没打开过它的 `SKILL.md`。这等于没看合同就签字。下一节起我们就是要治这个毛病。

7.3 阅读 SKILL.md

`SKILL.md` 是什么？它就是一个纯文本的 Markdown 文件，是这个 Skill 的“产品说明书加岗位说明书”。它一般不长，几百行以内。你不需要懂代码，用记事本就能打开。

打开之后，重点找下面这几段，按顺序读：

你要找的信息	它通常会写在哪儿	你要判断什么
这个 Skill 是干嘛的	开头的简介	和你要做的事对不对得上
它支持哪些 Agent / 平台	安装说明	你用的平台到底装不装得上
它需要什么前置条件	Requirements / Prerequisites	要不要额外装软件、要不要账号
它会读哪些文件、写哪些文件	权限说明 / 工作方式	会不会动到你的原始材料
它会不会联网、会不会调付费 API	网络/费用说明	会不会偷偷跑钱、会不会外发数据
它附带什么脚本	Files / Scripts 部分	脚本是最需要警惕的部分，见 7.5
怎么卸	Uninstall	装之前先确认能撤

读 SKILL.md 不需要你读懂代码，只需要你回答一个问题：“这个东西装进我的 Agent 之后，它可能对我的电脑、我的账号、我的钱做什么？”答得上来，再装；答不上来，先别装。

7.4 判断一个 Skill 是否可信：12 问安全检查清单

下面这 12 个问题，是你每次装社区 Skill 前必须要在心里过一遍的。它来自本教程的安全要求，也对应行业通用的开源件审查习惯。请把它当成一张检查表，装之前逐条问自己：

1. **来源是谁？** 是知名公司官方仓库，还是一个匿名账号？作者过去有没有别的可信项目？
2. **源代码在哪里？** 能不能看到完整代码，而不是只有一个“一键安装”按钮？
3. **最近有没有维护？** 最近一次提交是上周还是三年前？三年没动的东西，装之前要更谨慎。
4. **它会执行什么脚本？** 附带的脚本大概是干什么的？有没有看起来和它宣传的功能无关的操作？
5. **它能读什么文件？** 会不会去翻你的文档、桌面、下载文件夹？
6. **它能写什么文件？** 会不会往系统目录、配置目录里塞东西？
7. **它能访问哪些网站 / API？** 除了它该用的那个服务，还会不会连到别的地址？
8. **它需要哪些账号权限？** 连你的邮箱时，是“只读”还是“能发邮件”？连你的 GitHub 时，是“只读代码”还是“能替你 push”？
9. **它会不会把我的数据发到第三方？** 读出来的内容除了给模型用，会不会还发给别人？
10. **它会不会调用付费 API？** 跑一次花多少钱？有没有上限？
11. **它有没有删除、覆盖、修改能力？** 误操作时能不能找回来？
12. **能不能先在测试目录跑一遍？** 有没有一个干净的文件夹、一份副本，让它先在里面玩一次？

特别强调两句话：

- **Star 数不等于安全。** 一个热门项目可能被很多人装，但它的代码你没看过，风险就在你这台机器上。Star 只能说明“它流行”，不能说明“它干净”。
- **“官方”两个字要看清楚。** 有些聚合站名字里带 Official，不代表它是某家公司官方办的。认准域名和 GitHub 组织名，别被名字带跑。

第一次做这 12 问会很慢。做过四五次之后，你会形成直觉：这个 Skill 靠不靠谱，扫一眼开头和脚本列表就有数了。

7.5 Skill 中的脚本、模板和资产

打开一个 Skill 文件夹，你通常会看到三类东西：

- **说明文档：**就是 SKILL.md，你刚才读过的那份。最安全，纯文本。
- **模板和资产：**半成品文档、示例数据、图片、配置样例。它们本身不会自己跑，是给 Agent 参考着用的。风险低。
- **脚本：**真正会被执行的代码，可能是 Python、Shell、或平台自己的命令。这是**风险最高**的部分——它装上去之后，Agent 可能在你没注意的时候跑它。

新手不需要会写脚本，但要会“看脚本在干嘛”。一个粗略的原则：

- 如果脚本只是“格式化文本”“生成一个固定表格”，风险低；
- 如果脚本里出现了“联网下载”“删除文件”“改系统配置”“读环境变量并外发”这类字眼，立刻提高警惕，回去把 12 问里对应的问题问清楚再决定。

看不懂脚本写了什么，就把它丢给你正在用的 Agent，问：“这段脚本从头到尾干了什么？有没有删文件、联网、发数据的动作？”让它用人话讲给你听。

7.6 安装 Plugin

Plugin 和 Skill 很像，都是“给 Agent 加一块能力”，差别主要在来源和形态：

- **Plugin** 通常来自平台官方商店，一键安装，背后常连的是平台认证过的服务。你在界面上点“安装”，它就挂上去了。
- **Skill** 更像从开源社区下载的一个文件夹，你要自己放到配置目录、自己读 SKILL.md。

装 Plugin 相对省心，但“平台官方商店”不代表“绝对安全”：你依然要在安装弹窗里看一眼它要什么权限、会不会收费、会不会读你的账号数据。商店审核一般会拦住明显的恶意件，但审不到“这个插件是否过度收集你的数据”这种灰色问题。

7.7 添加 MCP Server

MCP 的添加方式，通常是在你的 Agent 配置文件里，写上一段说明：“我要用哪个 MCP Server，它怎么启动，它的钥匙放哪”。改完配置，重启 Agent，它就插上了这个“万能插座”。

以 2026 年 9 月各平台现状为准，不同平台的配置文件位置和格式不同，**不要照抄网上三年前的教程**，以你所用平台官方文档里的“添加 MCP”一节为准。

7.7.1 一次完整安装演示（以 filesystem 这个轻量 Server 为例）

下面用真实存在、官方维护、用途明确的 filesystem MCP Server 走一遍。它只做一件事：把你指定的一个文件夹暴露给 Agent 读写，非常适合第一次试手。同样地，标注“本环境无法替你完成”的步骤，要在你自己的客户端里做。

第一步：在官方注册中心发现。 打开 MCP 官方注册中心（registry.modelcontextprotocol.io），浏览或搜索 filesystem。你会看到一个官方维护的文件系统 Server，描述是“在你限定的目录里读写文件”。注意：注册中心的目录会持续变动，以你打开时的实时页面为准，本节只教你怎么挑——挑“官方/知名组织维护、描述清楚、只碰你指定目录”的轻量 Server。

第二步：源仓库核验。 点进它的源码仓库，看：README 有没有讲清楚怎么启动、要什么参数、License 是什么、最近还更不更新。它和装任何开源件一样，过一遍 7.4 的 12 问。

第三步：安装配置（真实配置示例）。 以常见的 Claude 系客户端配置为例（格式只作示意，你所用客户端的键名和路径以其官方文档为准），在它的 MCP 配置 JSON 里加一段：

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-fileSystem",
        "D:\\AISOP\\mcp-sandbox"
      ]
    }
  }
}
```

这段配置的意思是：用 npx 启动官方的 @modelcontextprotocol/server-fileSystem，并把它可访问的目录限定在 D:\\AISOP\\mcp-sandbox 这一个文件夹。把最后那行换成你自己希望 Agent 能碰的文件夹——这一步是 MCP 安全的关键：只给它一个专用沙箱目录，不要把整个用户目录或 C 盘根目录填进去。

第四步：先手动验证它能启动。 改完配置前，可以先在命令行单独跑一下启动命令，确认包能拉起来、不报错（首次运行 `npx` 会下载这个包，需要联网，耗时视网络而定）：

```
npx -y @modelcontextprotocol/server-filesystem "D:\AISOP\mcp-sandbox"
```

它启动后通常会停在那里等标准输入（因为 MCP 是一个通过 `stdin/stdout` 对话的进程），你看到它没报错、没立刻退出，就说明这一步通了，按 `Ctrl+C` 退出即可。

标注：“用一个最小 MCP 客户端脚本真正连上、列出工具、调用一次”这一步，需要你在自己的开发环境里写几行客户端代码或用客户端自带的连接测试功能来做。它的原理是：启动上面那个进程，按 MCP 的协议跟它握手，让它报出自己提供了哪些工具（一般是 `read_file`、`write_file`、`list_directory` 之类），再调一次 `list_directory` 看能不能列出 `D:\AISOP\mcp-sandbox` 里的文件。本教程运行环境不替你执行这一步，因为它属于“你自己客户端里的连通性验证”。如果连不上，先查三件事：`npx` 在不在、目录路径写对没有、客户端有没有重启加载新配置。

第五步：在 Agent 里测试。 重启你的 Agent 客户端，对它说：“你现在应该连上了一个叫 `filesystem` 的 MCP，请列出它暴露了哪些工具，然后读一下 `D:\AISOP\mcp-sandbox` 里的文件。”它能说出工具名、真的列出文件，才算装上。

卸载：把配置 JSON 里这一段 `filesystem` 删掉，重启客户端即可；`npx` 下载的包在本地缓存里，想彻底清掉就清 `npm` 缓存。

添加 MCP 时同样要过 12 问：这个 Server 是谁写的？它要访问什么？它需要什么密钥？会不会把你的数据转给第三方？一个标着“MCP”的开源 Server，本质和一个 Skill 一样，是会跑的代码。

7.8 添加 Connector

Connector 是最贴近普通用户的扩展方式：界面上一个“连接邮箱”“连接日历”“连接云盘”的按钮，点一下，跳到你那个账号的登录页，授权，回来就连上了。

它和手动配 MCP 的区别是：Connector 把技术细节包起来了，你不用碰配置文件。但授权那一步，你仍然要看清楚——这就引出下一节。

7.8.1 一次真实可验证示例：GitHub 官方集成（gh CLI 的 OAuth）

Connector 里最容易在本机完整走一遍、且能直接核对授权边界的，是 GitHub 官方命令行工具 `gh`。它走的就是标准的 OAuth 流程，你可以拿它当“理解 Connector 授权”的活标本。

第一步：确认工具已装、当前授权状态。 这台机器上真实执行：

```
gh auth status
```

本机真实输出大意是：已登录到 github.com，当前账号为 TZUKWAN，token 存在系统密钥环里，当前 token 的权限范围（scopes）是 `gist, read:org, repo, workflow`。这串 scopes 就是“OAuth 授权边界”的实物——你一眼能看到：它能读写仓库（repo）、能操作工作流（workflow）、能读组织信息（read:org）、能写 Gist（gist），但它没有 delete_repo 这种删除权限，也没有 admin:org 这种管理员权限。这就是 Connector 授权该有的样子：能做的列得清清楚楚，不该有的没有。

第二步：理解 OAuth 授权边界。 第一次用 gh 时跑 gh auth login，它会让你选“用浏览器登录”还是“粘贴 token”。选浏览器登录，它会打开 GitHub 官方页面，列清楚这次要哪些权限，你同意后才拿到那张通行证。关键不是这个机制多方便，而是你在那个页面上要逐行看：它要的每一项权限，是不是你真打算给的。拿 gh 举例：

它请求的 scope	翻译成大白话	该不该给
repo	能读、写你的仓库代码	你要让它动代码，就给
workflow	能改 GitHub Actions 工作流	你要让它自动跑 CI，才给
read:org	只读你所在组织的信息	一般够用
delete_repo	能删仓库	没有明确需要就别勾

第三步：事后核对和收权。 想临时收缩或扩大权限，本机真实命令：

```
gh auth refresh -s read:org
```

它会重新走一次浏览器授权，让你确认新的权限范围。不想用了，一键撤销：

```
gh auth logout
```

更彻底的做法，是直接去 GitHub 网页的 Settings → Applications → Authorized OAuth Apps 里，找到这个应用，点 Revoke。这和 7.13 节讲的“去账号侧撤销授权”是同一个动作。

其他平台如实说明： Notion、Slack、Google Workspace 这类 Connector，本机无法替你登录授权，因为它们需要你本人的账号、验证码、二次确认。流程和 gh 一模一样：点连接 → 跳到官方登录页 → 逐行看权限 → 同意 → 回来核对授权状态。差别只在登录页是哪家公司的，授权那一页你要做的事完全相同。

7.9 OAuth 与账号授权

当你点“连接邮箱”时，跳出来的那个登录页问你：“是否允许 XX 应用访问你的邮箱？”——这背后走的就是 **OAuth**（一种业界通用的“第三方授权”机制，你可以先理解成“你不把账号密码告诉这个应用，而是通过官方登录页给它发一张有时效的通行证”）。

关键不是这个机制本身，而是你在授权页上要做的事：

授权页会列出这个应用要哪些权限。你要逐条看，不要直接滑到底点“同意”。 典型的权限粒度包括：

- 只读你的邮件 / 日历 / 文档，还是能替你发邮件、改日历、删文档？
- 能访问哪些数据？全部，还是只某个文件夹？
- 有效期多久？能不能随时撤销？

用一个类比：OAuth 授权就像你给朋友一把家里的钥匙。你要知道这把钥匙能开哪扇门——能开大门可以理解，能开保险箱就过分了。授权页列出的就是“这把钥匙能开哪些门”。

事后想反悔怎么办？去你那个账号（邮箱、云盘、GitHub）的“第三方授权 / 应用授权”设置页，找到这个 Agent，点“撤销授权”。这一步任何时候都可以做，别等出了事才找。

7.10 API Key

很多服务不给你登录授权页，而是让你填一串“密钥”。这串字符就是 **API Key**——你调它服务时的“门钥匙”。

先讲清楚三件最重要的事：

第一，它是什么。 API Key 长得像一串随机字符（例如 `sk-xxxxxxxx...`）。拿着它，任何人都能以你的身份调用那个服务、用你的额度、甚至产生你的账单。

第二，它不是密码，但比密码还不能外传。 密码你还能登录后改，API Key 一旦泄露，要么去服务商那里重新生成一个、把旧的作废。它泄露的代价是真金白银和数据。

第三，三条铁规矩：

- **不要把 API Key 写进普通的对话 Prompt 里。** 你在对话框里贴一次，它就可能被记进聊天记录、被模型的日志带走。
- **不要把 API Key 提交到 GitHub 等公开仓库。** 哪怕你觉得“这个仓库只有我自己看”——仓库可能变公开、可能被 fork、可能被 AI 扫描器盯上。业界每年都有成百上千个 Key 因为误传 GitHub 被扫走、被盗刷。
- **Key 要能随时作废、随时重发。** 每个服务商后台都有“重新生成 / 作废旧 Key”的功能，怀疑泄露就立刻换。

那 Key 到底放哪？看下一节。

7.11 环境变量

直接在对话框里念 Key，是最危险的做法。正确做法是把 Key 放进一个专门的地方，让 Agent 要用时自己去取，而不是每次都从对话里读到。

这个专门的地方，就是**环境变量（Environment Variable）**。你可以先把它理解成“一个上锁的抽屉”：你把 Key 放进去，关上门；Agent 需要用的时候，按约定的名字去抽屉里拿，但它看不到别的东西，也不会把这串字打印到对话里、写进提交记录。

在 Windows 上临时看一眼当前环境变量名（只看名字，不要把值打印出来）：

```
Get-ChildItem env: | Where-Object { $_.Name -match 'KEY|TOKEN|SECRET' } | Select-Object Name
```

各平台设置环境变量的入口不同（有的在设置页，有的在配置文件里，有的在你电脑的系统环境变量里），以你所用平台文档为准。你只需要建立这个习惯：

凡是钥匙类的东西（API Key、Token、密码），一律进环境变量或平台的密钥管理处，永远不进 Prompt、不进文档、不进 Git。

7.12 最小权限原则：L1-L4 风险分级表

F14 第三方资源风险分级 L1-L4

L1	低风险	示例：只读公开资料 策略：可直接用
L2	中低	示例：装公开 Skill/Plugin 策略：看一眼来源即可
L3	中高	示例：联网/可写文件 策略：12问审查后再装
L4	高风险	示例：可删/可发送/可执行 策略：默认拒绝，必须人确认

风险随「能否影响外部/能否不可逆」递增，不随热度递减

图 15 L1-L4 风险分级

装上了能力，就要管住它能干什么。核心原则一句话：只给完成这件事所需的最小权限，多一分都不给。

把 Agent 的所有可能动作，按“做错了能不能收回来”分成四级：

风险级别	典型动作	默认策略
L1 低风险	读文件、搜索网页、分析数据副本、读你给的材料	可以自动执行，不用每次问你
L2 可逆修改	新建文件、改工作副本、在 Git 上建分支、改草稿	可以执行，但要留记录（改了什么、改前是什么）
L3 高影响	覆盖你原来的文件、Git push 到远程、改云端共享文档、替你发消息或邮件	建议每一步都停下来让你确认
L4 不可逆 / 高风险	删除数据、支付、公开发布、改生产数据库、改权限、对外发正式公告	必须你明确说“确认执行”才做，且最好先备份

这张表你可以直接抄下来，贴在你和 Agent 协作时的旁边。每次派任务时，心里过一遍：“我现在授权它做的事，落在哪一级？”

- 做文献整理、读 PPT、分析 Excel 副本——L1，放手让它跑。
- 让它在工作文件夹里新建分析结果——L2，跑之前让它先说清楚要建哪几个文件。
- 让它改你那份要交的论文原稿——L3，先让它改一个副本，你看满意了再替换。
- 让它“把这封邮件发出去”“把这个推到 GitHub 主分支”“付这笔钱”——L4，停下来，你自己点最后一下。

一个朴素的判断：凡是错了找不回来的事，最后那一下永远你自己按。

7.13 如何卸载 / 禁用能力

装上去的东西，要能撤下来。三种动作要分清：

- **临时禁用**：Agent 里往往有开关，把某个 Skill / Plugin / MCP 关掉，它暂时不生效，但文件还在。适合“我现在不想用它，但以后可能还开”。
- **卸载**：把这个插件从 Agent 里移除。适合“我确定不用了”。
- **撤销账号授权**：如果它连了你的邮箱 / GitHub / 云盘，记得去对应账号的第三方授权页把它踢掉——光在 Agent 里卸载，有时你那个账号上的授权还挂着。

一个好习惯：每季度花十分钟，去你常用的账号（邮箱、GitHub、云盘、社交平台）的“第三方应用授权”列表里扫一眼，把不认识、不再用的授权全部撤销。

7.14 如何更新 Skill / Plugin / MCP

这些社区件会更新：修 bug、加功能、改权限。但**不要开启全自动更新**。理由很简单：一个昨天还干净的 Skill，今天的版本可能加了联网、加了付费调用。

正确姿势：

1. 更新前先看它的更新说明（changelog / release notes）——这版改了什么？
2. 尤其注意“新增了什么权限、新增了什么网络请求、新增了什么依赖”。
3. 更新后跑一次你常用的 Mission，确认它没跑偏。
4. 出问题就回到上一个版本。

平台官方商店的 Plugin 更新相对省心，但也别假设“官方更新 = 对你更好”。

7.15 如何发现新的 Agent 能力：资源地图概念

F13 社区资源生态：Agent 能力从哪里来

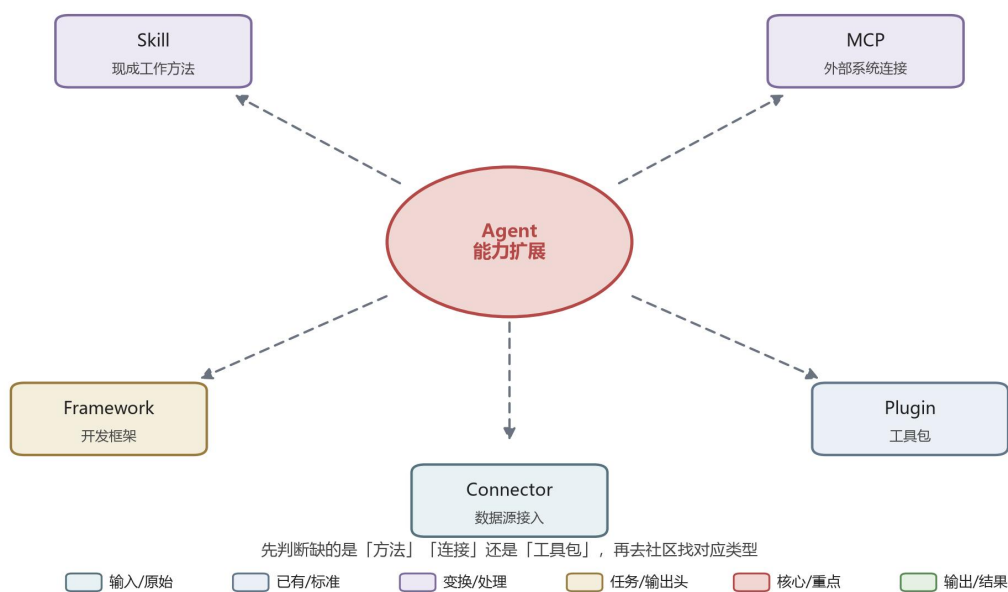


图 16 社区资源生态

最后一个问题：我怎么知道“有什么新东西可装”？

不要靠刷社交平台碰运气。你需要在心里建立一张**资源地图**，按三层去找，越靠上越可信：

第一层：官方来源。 你所用 Agent 平台的官方文档、官方 Skill 商店、官方 GitHub 组织、官方 MCP 注册中心（registry.modelcontextprotocol.io）。先查这里，因为信息最准、维护最勤。

第二层：聚合发现平台。 一些社区目录（如 SkillsMP、skills.sh 这类）会聚合公开的 Skill、MCP、Plugin，方便你搜索。它们的价值是“让你发现”，**不是让你直接装**。看到感兴趣的，记下来，回到第三层。

第三层：源仓库。 打开这个 Skill / MCP 的 GitHub 或官方源码页，看：谁做的、最近提交时间、Star 和 Issue 活跃度、README、License、有没有脚本、要什么权限。过一遍 7.4 的 12 问，再决定装不装。

再次强调：聚合平台上的排名、评分、“热门”“推荐”，只代表它被很多人发现，不代表它安全、不代表它适合你、不代表它还在维护。**发现靠聚合站，决定靠源仓库。**

具体的官方注册中心、Skill 目录、MCP 目录、社区 Awesome 列表，本教程配套的资源文件里会单独整理，并标注核验日期。本节只教你这套“三层找法”。

本章 Mission

Mission 7-A：给你正在用的 Agent 做一次“能力盘点”

列一张表：它现在用什么 Model？开了哪些 Tool？装了哪些 Skill / Plugin / MCP？连了哪些你的账号？每项标上“来源是官方还是社区”“最后一次更新是什么时候”“要了什么权限”。你会立刻看到哪些是你想保留的、哪些是当时图热闹装上的、哪些授权早该撤了。

Mission 7-B：走一遍完整的“安全安装”流程

任选一条真实路径走通：要么按 7.2.1 在你自己的 skills 目录里装一个社区 Skill，要么按 7.7.1 配一个 filesystem 这类轻量 MCP。不急着装。先打开它的 SKILL.md（或源码 README），按 7.3 的表读完；再按 7.4 的 12 问逐条写下来；有脚本就把脚本丢给 Agent 让它讲人话；最后才放进配置、在一个测试文件夹里跑一次，跑完按 7.13 完整卸载并确认无残留。整个过程就是给以后装所有扩展打样。

Mission 7-C：做一次“权限大扫除”

打开你邮箱、GitHub、云盘、社交平台的第三方授权列表，把不认识的、超过三个月没用的授权全部撤销；同时检查你 Agent 里有没有开过 Computer Use、Terminal 这类高风险能力，没用就关掉。再给自己定一条规矩：从今往后，凡是 L4 动作，最后一下永远自己按。

本章所有涉及具体平台界面、配置文件路径、商店名称、MCP 注册中心地址的内容，都以 2026 年 9 月各平台官方文档现状为准；社区生态变化很快，安装前请回到官方文档和源仓库再核验一遍。本章 PowerShell 命令与目录为本机演示用真实环境，读者照做时请替换为自己的用户名与平台路径；凡标注“本环境无法替你完成”的交互步骤，请在你自己的 Agent 产品中实际执行并核验。

PART 8：场景实验室

F16 场景实验室导航：7 个 LAB

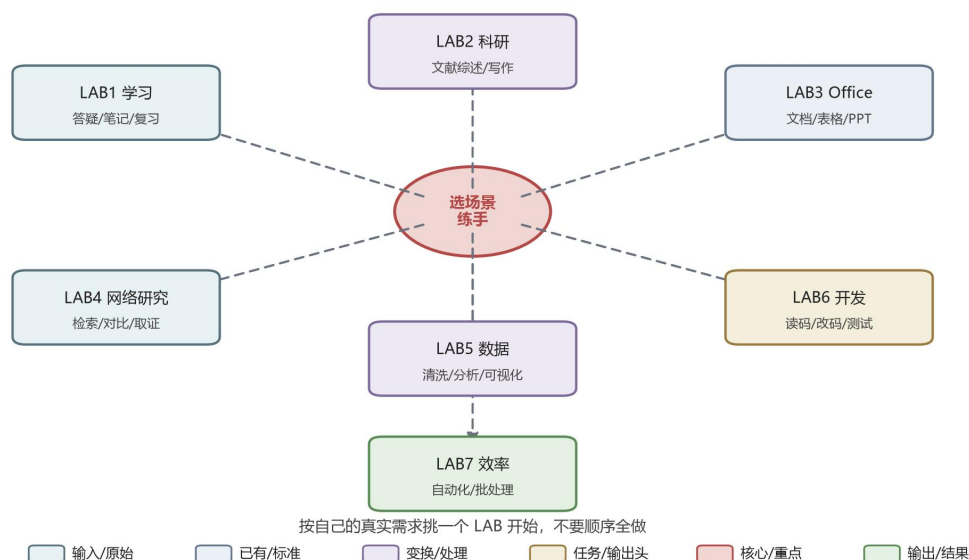


图 17 场景实验室导航

到这里，你已经学会了怎么给 Agent 下任务、怎么管长任务、怎么判断它是不是真做对了、它会怎么失败，以及它由哪些零件组成。从这一部分开始，我们不再学“通用方法”，而是进入“按专业挑一条路，把真事做一遍”。

先把三件事说清楚。

第一，本 Part 不要求你把 LAB A 到 LAB G 全部做完。七个实验室对应七种典型职业场景：学生做功课、研究生做科研、文职做 Office 报表、做调查写报告、跑数据、学软件开发、打理日常事务。你只需挑与自己专业和近期目标最相关的一到两个，做到“最终项目”那一步。贪多嚼不烂，把一条路走通，比每条路都看一遍教程有用得多。

第二，每个 LAB 都是同一个结构：一段 Mission 说明（做什么、多难、分几步、怎么算成），然后是操作要点，最后是“最终项目”。Mission 里的难度沿用全书 L0 到 L7 的刻度：L0 纯聊天，L1 单文件，L2 多文件，L3 联网加文件，L4 用工具或浏览器，L5 长任务，L6 自动化，L7 自己开发 Agent。

第三，也是最重要的——前面反复强调的四大铁律，到了场景里不是口号，而是每一步的检查动作：

- 代码生成不等于功能完成：Agent 写了一行代码、一段报告、一张表，都只是“写出来了”，不是“做对了”。
- 代码修改不等于 Bug 修复：Agent 说“我改好了”，要你自己跑一遍、看一遍才算数。

- 测试通过不等于产品完全正确：它跑通了你给的那个例子，不代表所有情况都对。
- Agent 声明完成不等于实际完成：它说“已完成”，你要拿出证据。

下面逐一来。

LAB A：学习 Agent

这个 LAB 解决什么问题

你手里有一门课：一份课程大纲、一叠 PPT、一本教材，还有四周后的考试。以前的做法是自己逐页翻 PPT、画思维导图、考前突击背题。这个 LAB 要把“从材料到会考试”这条链路交给 Agent，你做判断人和验收人。

最终项目：搭一个“课程学习 Agent”，以后每门新课都能复用这套流程。

Mission：课程学习全流程

- 名称：课程学习 Agent
- 目标：给定一门课的大纲和 PPT，产出知识结构、学习计划、自测题、错题分析和复习资料，形成可重复使用的工作流。
- 难度：L2 到 L5（前几步 L2 多文件，口试和错题分析属于 L5 长任务）。
- 需要的能力：读取 PDF/PPT 文件、长任务管理、要求它自我验证；不需要联网。
- 输入材料：课程大纲（一两页）、全部课件 PPT 或 PDF、教材电子版（可选）、往年题（可选）。
- 风险提示：Agent 会把它“以为你该知道”的常识当成课程内容，也会把 PPT 上一句话扩写成一段你老师根本没讲过的理论。所有知识点都必须回到课件原文核对。

分步骤怎么做

第一步，让 Agent 先读课程大纲，输出它对这门课的理解：这门课讲哪几大板块、每个板块多少学时、老师强调什么。这一步的目的不是要它总结，而是先看它有没有读错、读漏。读错了，后面全错。

第二步，让它逐份读 PPT，为每一章生成知识结构：一级概念、二级概念、概念之间的关系。要求它在每个知识点后面标注“来自第几张 PPT 第几页”。这个标注就是它的证据，不允许凭空补充。

第三步，找知识缺口。让它对照大纲和 PPT，列出“大纲里要求但 PPT 里没讲清楚”的条目。这一步很关键：Agent 读材料时倾向于“圆上”，把没讲到的地方用常识补全。你要明确禁止它补，只允许它列成“缺口清单”。

第四步，制定学习计划。让它按考试倒推日期，把缺口清单和知识点清单排成四周的计划，每天学什么、花多久、学完用什么方式自测。计划是给你执行的，不是给你欣赏的，所以要具体到天。

第五步，生成问题。让它从 PPT 原文出题：选择题、简答题、论述题都要，并附标准答案和“答案出处”。重点考那些它第三步标出的知识缺口。

第六步，口试。这是最像真学习的一步。你让 Agent 一次问你一个问题，你回答，它判断你答得对不对、漏了什么、有没有理解偏。它问完一轮，给你一份“你答错/答偏的点”清单。

第七步，分析错误。让它把你答错的点归成三类：是概念记混了、是根本没学过、还是理解反了。每一类给两到三个再练的题。

第八步，生成复习资料。基于错题清单，让它生成一页 A4 的考前速查表和十道压轴题。

第九步，把上面这套流程固定下来。你已经做过一遍，把每一步“你给它的指令、它该遵守的约束、每步产出什么文件”写成一份自己的模板。下一门课，把新材料丢进来，按模板走。这就是最终项目：课程学习 Agent。

成功标准

- 知识结构图里的每个一级概念都能在 PPT 里找到出处；随机抽五个知识点，翻 PPT 能对应上。
- 它出的题，你自己不看答案先做一遍，答案和 PPT 对得上。
- 口试一轮后，它给你的错题清单你逐条认可：确实是你不会的，而不是它挑刺挑出来的。
- 四周计划落到了天，而不是“第一周学前两章”这种空话。

常见失败

- 它把 PPT 里一页的标题扩写成一大段理论，你以为课上教了，其实没有。识别方法：抽查出处页码。
- 它出题太简单，全是 PPT 原文填空，测不出你懂没懂。改进：要求它出“用自己的话解释”“两个概念区别是什么”这类题。
- 学习计划排得太满，两周后你就放弃了。改进：让它按你每天实际能学的时间排，并且预留一天机动。

旗舰项目模板：课程学习 Agent

把上面九步固化成一个可复用项目，以后每门新课照走一遍。

- 输入材料：一门课的课程大纲（一两页）、全部课件 PPT 或 PDF、教材电子版（可选）、往年题（可选）。

- 步骤：读大纲建立理解 → 逐章生成带页码标注的知识结构 → 列知识缺口清单 → 按考试倒推排学习计划 → 从原文出题 → 口试一轮 → 错题归类并出再练题 → 生成速查表与压轴题 → 把全流程写成自己的指令模板。
- 产物：知识结构图、缺口清单、按天学习计划、带答案出处的题库、错题分析、考前速查表一页、可复用的流程模板。
- 验收标准：随机抽五个知识点都能在 PPT 找到出处；它出的题你先做一遍、答案与原文对得上；错题清单你逐条认可；计划落到天而非“第一周学前两章”。

LAB B：科研 Agent

F29 8 要素示例：写一篇「AI Agent 教育应用」文献综述

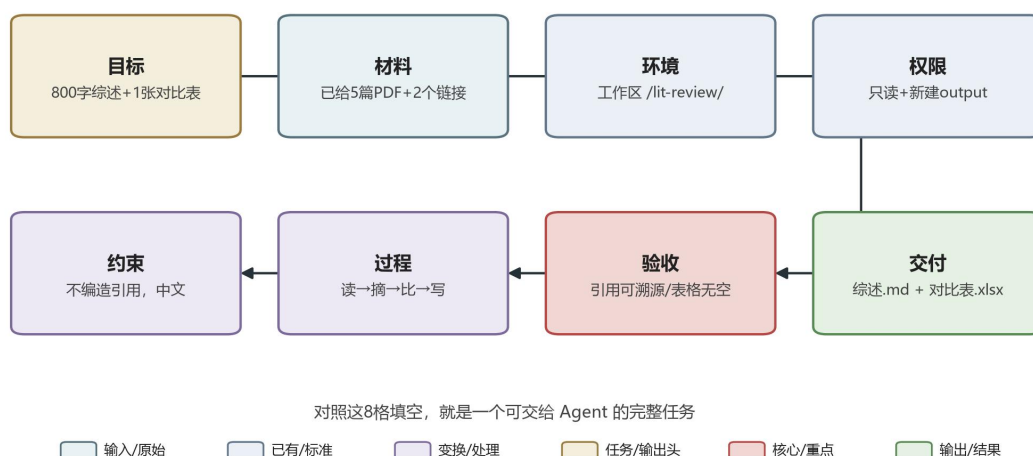


图 18 8 要素文献综述示例

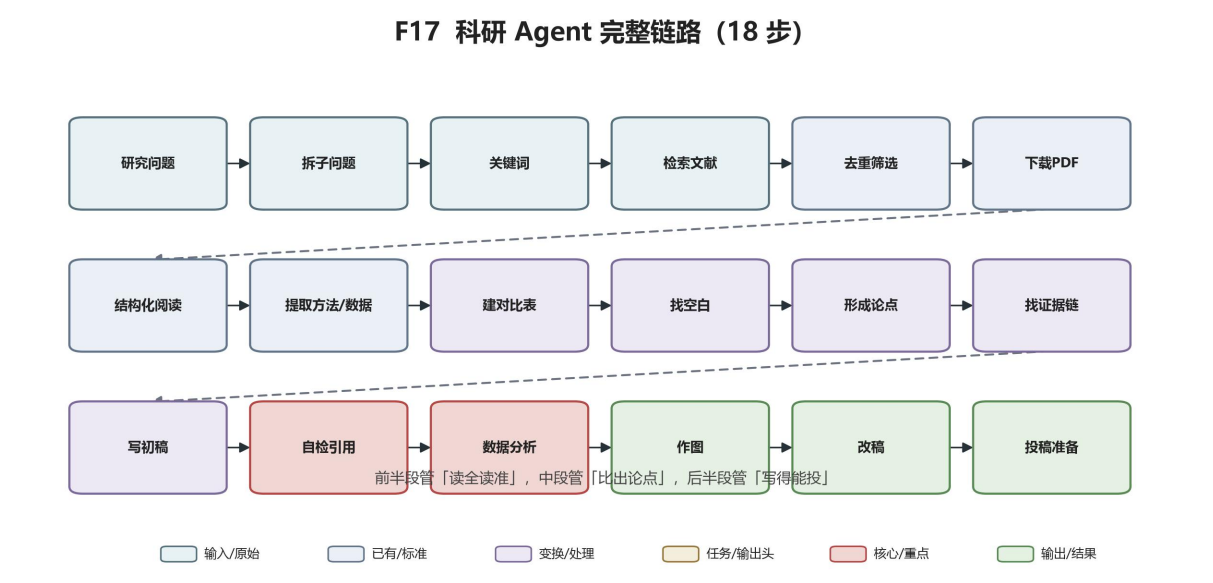


图 19 科研 Agent 完整链路

这是全书最需要“一边用 Agent、一边守住底线”的 LAB。科研的生命是真实：真实的文献、真实的数据、真实的方法。Agent 在这些环节上的失效不是“做慢了”，而是“做假了”，而它做假的时候语气和做真的时候一模一样。所以这个 LAB 把完整科研链路拆开，同时把学术诚信单独拎出来讲。

最终项目：搭一条自己的科研工作流 Agent——以后任何一个选题，都能按这条流水线走一遍。

完整链路

- 研究问题
- 检索策略
- 文献搜索
- 去重
- 筛选
- 文献阅读
- Evidence Matrix（证据矩阵）
- 研究脉络
- 文献综述
- 理论框架
- 研究设计
- 数据
- 分析
- 可视化
- 写作
- 引用核验
- 独立审计
- 投稿准备

逐段说明它在干什么、你要盯什么。

研究问题阶段：让 Agent 帮你把“我想研究点什么”这个模糊念头收敛成一个可检验的问题。它擅长的是反问你：你到底关心哪个群体、哪个机制、什么时间范围。它不擅长替你决定研究价值——选题的学术判断必须是你和导师的。

检索策略阶段：先别搜。让它把检索词拆出来——中文关键词、英文关键词、同义词、缩写、要用的数据库名（知网、Web of Science、Google Scholar、PubMed 等按学科选）。检索策略写下来，后面每一步都按它执行，避免“边搜边改词”导致你都不知道自己到底搜了什么。

文献搜索与去重：它按策略去搜，把每一篇的题录（标题、作者、年份、出处、DOI）先拉成一张表。去重不是删重复条目那么简单，要它说明按什么字段判重、判掉了多少篇、剩下多少篇。

筛选：让它按“摘要里是否真的谈了这个问题”初筛，分成“必读/可略/不相关”三类，每篇给一句理由。理由必须能在摘要里找到依据。

文献阅读：不要让它一口气读完五十篇然后给你一个笼统综述。让它一篇一篇读，每篇填一张卡片：研究问题、方法、数据、核心发现、局限、与你的选题关系。这就是证据矩阵的雏形。

Evidence Matrix：把所有卡片叠成一张表，行是文献，列是“研究问题/理论视角/方法/样本/核心结论/与我研究的关系/可质疑之处”。这张表是你后面写综述和设计研究的底盘，也是你判断“这个领域到底干了什么、没干什么”的依据。

研究脉络：让它在证据矩阵上回答三个问题——这个领域是怎么一步步发展到今天的、有哪几条互相争论的线索、最近三年的新转向是什么。它说的每一条“脉络”，背后都要能在矩阵里找到两三篇文献支撑。

文献综述与理论框架：让它基于矩阵写综述初稿。这一步它写出来的东西“像论文”，但也最容易混入它自己的发挥。写作阶段的铁律是：每一个论断后面跟着它来自矩阵哪几行；找不到出处的论断，标成“待核实”，不许直接留在正文里。

研究设计、数据、分析、可视化：到了这里，Agent 能帮你想设计、帮你跑统计、帮你画图，但“用什么数据、数据怎么来的、样本怎么选的”必须由你定义清楚并能对人讲明白。统计结果一律要求它同时给出代码和原始数值，不许只给一张图说“显著”。

写作、引用核验、独立审计、投稿准备：写完之后，把引用核验单独成一步——它列的每一条文献，你打开原文核对作者、年份、出处、页码、DOI，凡是打不开、查不到、对不上，一律删掉或换成真实存在的。再让一个与写稿时不同的模型（或同模型新对话，不带写作上下文）只当审稿人，挑问题。最后才是格式、期刊要求、cover letter。

必须单独守住的六条底线

这六条不是“注意事项”，是这个 LAB 的及格线，单独说。

第一，学术诚信。Agent 是助手不是作者。它帮你读、帮你整理、帮你起草，但研究问题、判断、结论由你负责。绝大多数期刊和学术机构对“AI 生成式工具”都有披露要求，你必须按你所在机构和目标期刊的规定如实声明它参与了哪些环节。这里要澄清一个常见误解：不要把“会不会被 AI 检测工具抓出来”当作是否越线的判据——AI 检测工具的误报和漏报都在文献里被反复讨论，它既不能证明你清白，也不能判定你违规；真正可靠的底线只有一条，就是按机构和期刊要求如实披露，并对自己写下的每一句结论负责。

第二，引用真实性。这是 Agent 最容易翻车的地方。它非常擅长造出“看起来完全真实”的引用：真实的作者、相近的年份、格式标准的 DOI、甚至通顺到像真论文的标题。你没有义务怀疑每一句话，但你有义务打开每一个引用。流程上强制自己：综述定稿前，逐条打开原文，打不开的一律删。

第三，AI 使用边界。让它做什么、不做什么，自己心里要有一条线。一般来说，帮你找文献、读文献、整理矩阵、查格式、改语言、做代码，是合理的；让它替你提出核心论点、替你下结论、替你伪造“我做过这个分析”，越线了。尤其要警惕：它没跑过的分析，不要在论文里写成自己跑过。

第四，数据来源。它给你的任何数字、任何“某国某年某指标”，都要能回到原始出处——国家统计局、世界银行、论文附表、问卷原始记录。不许用二手转述当一手数据，更不许它“估”一个数。

第五，方法透明性。你用了什么方法、为什么用这个方法、有什么局限，要能写清楚。Agent 很容易把方法写得“很高级但你自己讲不明白”——凡是你讲不明白的方法，要么真搞懂，要么别用。

第六，可复现性。把你的数据、清洗步骤、分析代码、随机种子都留好。理想状态下，另一个人拿到你的数据和代码，能跑出你论文里的同一组数。这既是学术规范，也是你防止自己“被 Agent 绕进去”的最好办法：结果能不能复现，一跑便知。

如何查找你目标学校 / 期刊的 AI 使用政策

“要披露”这件事不能靠猜，必须落到你自己所在机构和目标期刊的具体规定上。方法很简单，就是去官方页面搜，不要转述别人的二手说法。

第一步，确定两个主体：一是你在读（或要毕业）的学校，二是你准备投稿的期刊。学校管的是学位论文和课程作业，期刊管的是投稿稿件，两者规定可能不同，要分别查。

第二步，用搜索引擎直接搜官方页面。推荐关键词组合：

- 学校：“[学校英文名] generative AI policy”、“[学校中文名] 研究生 生成式人工智能 学术规范”、“[学校英文名] graduate school AI use policy”。优先点开域名以 .edu 结尾的研究生院（graduate school）或学术诚信办公室（office of academic integrity）页面。
- 期刊：“[期刊名] author guidelines AI”、“[出版社名] generative AI policy”。优先点开出版社作者指南（author instructions / guide for authors）页面。

第三步，几个可查的官方入口示例（核验日期：2026-09-18。政策会更新，使用前请自行打开确认最新版本）：

- Elsevier（爱思唯尔）作者指南中的“AI 与 AI-assisted technologies in scientific writing”说明页，搜“Elsevier author hub AI policy”。
- Springer Nature 的“Editorial policies: AI and AI-assisted technologies”说明页，搜“Springer Nature editorial policies AI”。
- 各高校研究生院官网的“学术规范 / Academic Integrity / Use of Generative AI”专栏，如 MIT、Stanford、Oxford、清华、北大的研究生院或教学发展中心页面。
- 国内可同时查学校研究生院官网发布的“研究生学位论文写作规范”及其中关于人工智能工具使用的说明。

第四步，把查到的规定存证。把官方页面链接、访问日期、原文关键句截图或另存一份，写进你自己的“披露清单”。投稿前再核对一次目标期刊的当前版本——政策可能在你改稿的几个月里更新。

四条真实性硬要求（本 LAB 的及格线）

在动用任何 Agent 之前，先把这四条写死在你的工作流里：

- 真实文献：综述里每一篇文献都必须是你亲自打开、读过的原文，不接受 Agent 报出但你没核实的条目。
- 真实数据：任何数字都要回到原始出处（统计局、世行、论文附表、问卷原始记录），不用二手转述，不许它“估”。
- 可追踪证据：证据矩阵里每一行文献、每一个论断，都能指出它来自哪篇的哪一页或哪张表。
- 可复现代码：清洗与分析代码、随机种子、数据说明一并留档，另一个人能跑出你论文里的同一组数。

成功标准

- 证据矩阵里每一行都对应一篇真实存在、你真的读过的文献。

- 综述里抽十条论断，每一条都能在矩阵里找到支撑文献；抽十条引用，逐条打开原文能对上。
- 你能向导师用十分钟讲清楚研究问题、方法、数据来源和局限。
- 你清楚地知道本稿里 Agent 参与了哪些环节，并已按要求准备披露。

旗舰项目模板：个人科研工作流 Agent

- 输入材料：一个具体研究问题、拟用数据库清单（知网 / Web of Science / Google Scholar / PubMed 等按学科选）、初版检索词表。
- 步骤：写检索策略 → 拉题录表并去重 → 摘要初筛分三类 → 逐篇填文献卡片 → 叠成 Evidence Matrix → 归纳研究脉络 → 基于矩阵写综述初稿 → 逐条核验引用 → 换对话当独立审稿人挑刺 → 按目标期刊和学校政策准备披露声明。
- 产物：题录表、证据矩阵、综述初稿、引用核验清单、独立审稿意见、披露声明草稿。
- 验收标准：矩阵每行对应真实读过的文献；抽十条论断各有矩阵支撑；抽十条引用逐条打开原文对得上；分析结果附代码与原始数值可复现；已按查到的官方政策写好披露声明。

LAB C: Office Agent

这个 LAB 解决什么问题

学生和职场人大量的时间花在三件事上：整理 Word、分析 Excel、做 PPT。这个 LAB 把这三件事连同 PDF 处理、批量文件处理、模板套用串成一条流水线。

最终项目：从一批原始材料（比如一份调研问卷导出的 Excel、几份访谈记录 Word、若干 PDF）自动形成“Excel 分析加 Word 报告加 PPT 汇报”三件套。

Mission：三件套流水线

- 名称：从原始材料到汇报三件套
- 难度：L3（联网加文件）到 L5（长任务）。
- 需要的能力：读写 Excel/Word/PPT/PDF，必要时用代码做批量处理。
- 输入材料：一个装着原始材料的文件夹，结构乱一点没关系。
- 风险提示：Office 文档的格式很脆。Agent 改 Word 时可能改乱样式、改 Excel 时可能覆盖你的公式、做 PPT 时可能把你的数据标错。所有操作都要求它在副本上做，原始材料锁起来只读。

分项要点

Word 整理：让它先盘点这个文件夹里有哪些 Word、各自多长、大概讲什么，列一张清单给你确认。再让它把同类文档合并、统一标题层级、补目录。不要一上来就让它“把这些文档整理好”——它不知道“整理”对你意味着什么。

Excel 分析：这是最需要数据纪律的一步。让它先读懂表——有多少行多少列、每一列是什么含义、有没有缺失、有没有明显异常值（比如年龄写成 999、收入写成空字符串）。它先把“数据字典”（每列叫什么、代表什么、取值范围）写出来给你确认，再开始分析。绝不允许它不看表就画图。

PPT：先定结构再做页。让它给你一个汇报大纲（几页、每页讲什么、结论是什么），你点头了再让它排版。PPT 上每个数字都要能在 Excel 分析结果里找到，不许它现编。

PDF：PDF 是最麻烦的格式——扫描件、图片版、表格跨页、乱码都常见。让它先判断这个 PDF 是文字版还是扫描版，扫描版先做识别再谈内容；提取表格时，要求它把提取出来的表再贴回来和原 PDF 对一遍。

批量处理：“把这一百个 Word 的标题都改成宋体小四”“把这五十个 Excel 都加一列”这类活，交给 Agent 最划算，但必须先拿三个文件试跑，你检查无误，再让它批量做全部。批量出错时，错得也很整齐——一百个文件一起错。

表格格式与模板套用：固定你常用的报告模板（公司模板、学校模板都行），把模板给它，让它“只填内容、不动模板样式”。

文档检查：最后让它当挑错人——查错别字、查格式不统一、查数字前后不一致、查引用缺漏。这一步用它比你更冷静。

成功标准

- Excel 分析的每一个结论，你点开对应单元格能复核。
- Word 报告里的数字和 Excel、PPT 完全一致，没有三套数。
- PPT 十页左右，每页一个结论，不堆字。
- 原始文件夹原封未动，所有产物在新的输出文件夹里。

旗舰项目模板：从原始材料到汇报三件套

- 输入材料：一个装着原始材料的文件夹（调研问卷导出的 Excel、若干访谈记录 Word、若干 PDF），外加你常用的报告模板。
- 步骤：盘点文件夹并列清单 → Word 合并统一格式 → Excel 先写数据字典再分析 → PPT 先出大纲你点头再排版 → PDF 先判断文字版还是扫描版 → 三个文件拿三个试跑再批量 → 套用模板 → 当挑错人做最终检查。

- 产物：整理后的 Word 报告、带数据字典和复算说明的 Excel、十页左右的 PPT 汇报、输出文件夹（原始材料只读不动）。
- 验收标准：Excel 每个结论点开单元格可复核；Word 数字与 Excel、PPT 完全一致；PPT 每页一个结论不堆字；原始文件夹原封未动。

✓ 本教程已落地的真实案例：见 `examples/labs/lab_c_office/`。

输入 `input_data.csv`（15 行销售明细），运行 `python run_lab_c.py` 即产出 `sales_report.xlsx`（明细/区域汇总/产品汇总三表）、`sales_report.docx`（Word 报告）、`talking_points.md`（汇报要点）；运行日志 `lab_c.log`，逐项验收见 `CHECKLIST.md`。复算证据：Excel 区域/产品合计与 Word 总额均为 **62,818.00 元**，三方一致。

LAB D：网络研究 Agent

这个 LAB 解决什么问题

你要写一份调查报告：调查一家公司、一项政策、一个行业、一个产品、一个竞品、或者一类学术资源。靠搜索引擎一页一页翻、靠收藏夹攒链接，既慢又容易只看到愿意看的那几篇。这个 LAB 教你把“找资料、读网页、比来源、核事实”做成一条可追溯的流水线。

最终项目：一份调查报告，里面每一条关键结论后面都挂着真实可打开的来源。

必须覆盖的能力

Search（搜索）：基础联网检索。要教会它换着关键词搜——同一事实的不同说法、中英文、官方名和俗称。

Deep Research（深度研究）：让 Agent 自己规划检索路径、读很多网页、交叉比对后给你一份带来源的长报告。用它之前，你仍然要先把“要回答什么问题、可接受的来源类型、多少篇起步”说清楚，否则它会给你一份什么都提到、什么都不深的东西。

Browser（浏览器）：有些内容搜索抓不到，但打开网页能看到。让它像人一样点进去读页面、翻分页、点下一页。网页阅读要区分：正文、评论区、广告、相关推荐——别让它把推荐链接当事实来源。

多来源比较：同一个事实，至少找两个独立来源对一遍。来源之间互相抄的不算两个来源。

信息提取：把网页里的关键数字、时间、人名、机构名抽成一张表，每条带来源链接和访问日期。

事实核验：凡是重要结论，单独核一遍——优先找官方原始出处（政府网站、公司公告、法院文书、论文原文），媒体转述、自媒体文章只能当线索，不能当证据。

来源记录：从第一天就建一张来源表：标题、网址、发布日期、你访问的日期、这篇资料支撑了哪个结论。报告里的每个关键结论都能在这张表里找到行。

网页变化与时效性：网页会改、会删、会过期。对关键页面，让它存档一份（截图或保存网页），并记下访问日期。政策类、价格类、数据类信息尤其要看“这是什么时候的数据”。

Mission：企业调查

拿一家真实但你不太了解的公司练手。让 Agent 回答：这家公司做什么、谁开的、规模多大、最近有什么大动作、有没有负面新闻或诉讼、行业里排什么位置。

要求它把每个答案都写成“结论加来源加访问日期”的格式，并且明确区分：哪些是公司自己说的（官网、招股书）、哪些是第三方报道、哪些是查不到只能存疑。查不到的，老实写“未找到可靠来源”，不许它补。

成功标准

- 报告里随机抽五条结论，每条都能点开来源、且来源确实支持这句话。
- 至少三条结论标注了访问日期，且你能看出数据是哪一年的。
- 有一节专门写“本报告未能核实的部分”，而不是把空白用流畅的话填满。

旗舰项目模板：企业 / 行业调查报告

- 输入材料：一家真实但你不太了解的公司（或一项政策、一个行业、一个竞品），你关心的 5 到 8 个具体问题。
- 步骤：换关键词多轮搜索 → 深读官方原始出处优先 → 同事实至少两个独立来源互证 → 抽关键数字建来源表 → 重要结论单独核验 → 每个结论按“结论+来源+访问日期”写 → 单列“未能核实”一节。
- 产物：带来源表的调查报告、来源清单（标题/网址/发布日期/访问日期）、未能核实事项清单。
- 验收标准：随机抽五条结论都能点开来源且来源确实支持；至少三条标注访问日期并能看出数据年份；有专门一节写未核实部分而非用流畅话填满。

LAB E：数据分析 Agent

F18 数据分析 Agent 流程



关键：每步都保留脚本和中间文件，结论必须能被重新跑出来

输入/原始 已有/标准 变换/处理 任务/输出头 核心/重点 输出/结果

图 20 数据分析 Agent 完整流程

这个 LAB 解决什么问题

你有一份数据表（问卷导出、公开统计、实验记录都行），想知道里面到底说明了什么。这个 LAB 把数据分析拆成标准工序。它和前面几个 LAB 的区别在于：这里“看图说话”的诱惑最大，所以纪律最硬。

完整流程

```
读取数据
→ 数据字典
→ 数据质量
→ 清洗
→ EDA（探索性分析）
→ 可视化
→ 分析
→ 建模
→ 诊断
→ 稳健性
→ 解释
→ 报告
```

逐段说。

读取数据：让它先报告这个表的“长相”——多少行、多少列、每列名字、每列大概是什么类型（数字、文字、日期）。先别分析。

数据字典：每一列到底什么意思？问卷里这道题问的是什麼？“收入”这一列是月还是年、税前还是税后？这些含义你不说清楚，后面所有结论都是悬空的。让它把不确定的列列成问题问你，而不是替你猜。

数据质量：缺失多少、有没有重复行、有没有明显录错的值（年龄负数、日期在未来）、单位是不是统一。这些问题先列清单，再动手。

清洗：删哪些、补哪些、改哪些，每一步都要它说明理由，并且在副本上做。原始数据永远不动。

EDA（Exploratory Data Analysis，探索性分析）：先不加任何判断，把分布、极值、相关性画出来给你看，回答“数据长什么样”，不回答“为什么”。

可视化：画图的目的是让你看懂，不是好看。要求它每张图配一句“这张图想说明什么、坐标轴是什么、数据范围是多少”。

分析、建模：到了要算关系、要建模型的时候，先问你一句“你的研究问题是什么”——同一个数据集，回答不同问题要用不同方法。让它把用了什么方法、为什么用、假设是什么讲清楚。

诊断、稳健性：模型跑出来之后，让它做两件事——这个模型在样本上表现好不好（诊断），换一种做法结论会不会变（稳健性）。结论一变就变的“规律”，不能当发现写。

解释、报告：最后落到结论。每个结论后面跟：这是用哪些数据、什么方法、在什么范围内成立的。范围外的话不要说满。

四条铁规矩

一，不允许看图“猜数据”。它画了一张图说“两组人差异很大”，你就问它：具体差多少、p值多少、样本量多少。没有数字的图，等于没分析。

二，不允许编造统计结果。凡是报告里出现的平均数、比例、相关系数，要求它把计算用的代码贴出来，你能在你的数据上重跑一遍得到同一个数。

三，代码应可复现。把清洗和分析的代码存成文件，连同数据说明一起放项目里。今天跑和三个月后跑，结果应该一样。

四，指标定义必须明确。“活跃用户”“高收入人群”“年轻消费者”这类词，在报告里第一次出现时必须写清楚：怎么算出来的、阈值是多少。同一个词在不同段落里不能变含义。

成功标准

- 你能对着数据字典，用自己的话讲清每一列是什么。
- 报告里每个关键数字都能复算。
- 有一节写清“这个结论在什么条件下成立、什么情况下可能不成立”。

旗舰项目模板：一份可复现的数据分析报告

- 输入材料：一份真实数据表（问卷导出、公开统计、实验记录均可），以及你要用它回答的一两个具体问题。
- 步骤：先报表的长相（行/列/类型）→ 写数据字典给你确认 → 列数据质量清单 → 在副本上清洗 → EDA 出图带图注 → 按研究问题选方法 → 模型诊断与稳健性检验 → 每个结论标注数据/方法/适用范围。
- 产物：数据字典、清洗与分析代码、带图注的 EDA 图、结论报告（含适用范围与局限一节）。
- 验收标准：每个关键数字都能在你的数据上用附的代码复算；每个图表都有图注说明坐标轴和范围；报告有一节写清结论在什么条件下成立、什么情况下不成立。

✓ 本教程已落地的真实案例：见 `examples/labs/lab_e_data/`。

自建脏数据 `dirty_data.csv`（22 行，含缺失/越界/格式不一致三类脏数据），运行 `python run_lab_e.py` 即产出 `cleaned_data.csv`（清洗后 10 行）、`quality_report.md`（数据字典+质量检查+剔除留痕）、`analysis_report.md`（描述统计+结论+局限）、`income_by_city.png`（各城市平均收入柱状图）；运行日志 `lab_e.log`，逐项验收见 `CHECKLIST.md`。复算证据：清洗后恒为 10 行，月收入均值恒为 **10,180.05 元**。

LAB F：不会编程也能开发软件

F19 软件开发 Agent 流程



铁律：先跑起来再改；每改一步就 Git 提交，可回退

输入/原始 已有/标准 变换/处理 任务/输出头 核心/重点 输出/结果

图 21 软件开发 Agent 全流程

这是本教程最有特色的一个 LAB，也是最需要你耐住性子的一个。它要教的不是“写代码”，而是“一个完全不会编程的人，怎么借助 Agent 把一个软件项目从找到、跑起来、改一点、测一测，到真正发出去”。

先把教学路径说死：本 LAB 的目标是“借助 Agent 完成开发”，不是“教你从零学会编程”。你不需要背语法、不需要默写函数，你需要练的是找项目、读说明、跑起来、提需求、验收、存版本这一套“甲方加验收员”的动作。代码本身由 Agent 写，你负责看结果、判对错、做决定。如果某一步它写的代码你完全看不懂，这是正常的，本 LAB 不要求你看懂，只要求你能确认它跑出来的效果对不对。

再讲怎么挑一个适合练手的开源项目。别一上来挑一个几十人团队、几十万行代码的大项目——你会被依赖装半天、文档三版不一致直接劝退。挑项目按四条标准：

第一，简单。优先选单语言、单仓库、README 里明明白白写了“三步跑起来”的小工具类项目。一个 CLI 小工具、一个静态网页、一个单文件脚本，都比一个前后端分离的大系统适合第一次练手。

第二，活跃。看两个指标：最近三个月有没有人提交代码、Issue 区有没有人在回答。超过一年没动静、一堆打开的 issue 没人管的项目，你踩了坑都没人问。

第三，文档好。README 必须写清楚怎么装、怎么跑、怎么贡献。README 一上来就是架构图、不提运行方法的项目，新手绕不出来。

第四，许可友好。选 MIT、Apache-2.0、BSD 这类宽松许可的项目，便于你学习和二次修改；GPL 类项目用于个人学习没问题，但如果你想把它拿来商业发布，要先看懂许可。具体怎么找：在 GitHub 搜索框里按“topic 加语言”搜（如“python cli”、“markdown notes”），按 stars 排序，再点进几个候选的 README 对照上面四条筛。

最后把权限边界讲清楚，哪些动作你自己在本地就能做、哪些需要账号和权限：

- 你在本地就能独立完成的：Clone / Download 下载项目、配依赖、本地跑起来、改一点代码、本地测试、用 Git 在本地提交（Commit）和开分支。这些全程不需要任何外部账号。
- 需要你自已注册账号、并具备对应权限才能做的：GitHub Push（需要 GitHub 账号和该仓库的写权限，别人的项目你只能先 Fork）、Pull Request（需要你自己的 GitHub 账号）、Deployment（需要部署平台账号，如 Vercel / Netlify / GitHub Pages）、Release（需要该仓库发布权限）。

所以零基础第一遍，完全可以只走“本地闭环”：Clone → 跑起来 → 改 → 本地测 → 本地 Commit。把这一圈走顺了，再花十分钟注册账号，去碰 Push / PR / Deploy / Release 这些对外动作。涉及对外的动作，照例按全书风险分级先确认再按。

先记住这一屏字，后面二十一步每一步都可能回头看它：

代码生成 ≠ 功能完成
代码修改 ≠ Bug 修复
测试通过 ≠ 产品完全正确
Agent 声明完成 ≠ 实际完成

F20 四大铁律（违反任何一条都可能出事）



这四条不依赖具体工具，换任何平台都成立

图 22 四大铁律警示图

下面严格按 F1 到 F21 的顺序走。每一步都用大白话解释它在干什么、你要做什么、怎么判断这步过了。

F1. 软件开发到底在做什么

别被“软件”两个字吓住。所谓软件开发，本质上就是：把一堆事先写好的文本文件（代码），放到一个能读懂它的环境里运行，让它按你想要的方式工作。你要做的不是手写这些文件，而是指挥 Agent 找到、读懂、改动这些文件，并亲眼确认改动真的生效。你的角色是“甲方加验收员”，不是“程序员”。

这一步你要建立的认知：软件不是一个黑盒子，它是一堆放在文件夹里的文件，有办法看、有办法改、有办法退回去。

F2. GitHub 是什么

GitHub（代码托管平台）是一个网站，全世界的人把自己写的软件项目放在上面，公开分享。你可以把它理解成“软件界的公开网盘加作品展示架”。绝大多数你想用的开源软件，都能在 GitHub 上找到。你不需要会用它的全部功能，只需要会找项目、看说明、下载项目。

F3. Repository 是什么

Repository（代码仓库，简称 Repo）就是 GitHub 上的一个项目文件夹。点进一个项目，你看到的那个页面就是一个 Repository：里面有代码文件、有说明、有别人提的问题、有版本记录。你可以把它想成一个项目的“主页加文件柜”。

F4. Clone / Download

Clone（克隆）就是把 GitHub 上那个项目完整下载到你自己电脑上，下载完你就有了这个项目的全部文件。Download 是另一种更简单的方式：直接下载一个压缩包解压。区别是 Clone 会把“这个项目的版本历史”也一起拿下来，后面我们要用这个历史来回退。第一次练手，用 Agent 帮你执行 Clone 就行，你不用自己敲命令。

F5. README

README 是项目根目录里那个说明文件。任何人想知道这个项目是干什么的、怎么装、怎么跑，都先读它。对零基础的你，README 是这个项目的“使用说明书”。让 Agent 把 README 读给你听：这项目解决什么问题、需要什么前提、怎么启动。如果 README 写得很烂、根本跑不起来，这个项目趁早换——别在它身上耗。

F6. 找到如何运行项目

读懂 README 之后，下一个问题是“我怎么在自己电脑上把它跑起来”。让 Agent 帮你找：启动命令是什么、需要点哪个按钮、跑起来以后访问哪个地址。跑起来的标志是：你自己亲眼看到一个窗口、一个网页、或者一句“运行成功”出现在你屏幕上。Agent 说“应该能跑了”不算，你看到才算。

F7. Dependency

Dependency（依赖）是这个软件正常运行所需要的“其他零件”。就像做饭需要菜和调料，一个软件常常需要别人写好的一堆基础包。你不用认识这些零件是谁，你只要知道：Agent 会帮你把它们装好；装不上、报错、版本对不上，是这一步最常见的麻烦，把报错原样贴给 Agent 让它解决。

F8. Local Development

Local Development（本地开发）指的是：在你自己的电脑上，把这个项目跑起来、改一改、再看效果的状态。你要建立的工作姿势是：改一个东西，立刻跑起来看一眼；不对，就改回去。不要一口气改十处再跑——那样一旦坏了，你根本不知道是哪一处弄坏的。

F9. 让 Agent 阅读代码

现在你有了一个能跑的项目，但你看不懂里面的代码。没关系。让 Agent 当你的翻译：“这个项目的主流程是什么？哪个文件控制页面上那个按钮？这个按钮点下去会发生什么？”它读完会给你一张“地图”：哪几个文件是核心、哪个文件你以后最常改。这张地图你存下来，后面所有修改都靠它定位。

F10. 需求 → Task List

你想加一个小功能（比如把首页那个按钮的颜色改了，或者加一句欢迎语）。不要直接说“帮我改一下”。先把需求写成一句话目标，再让它列成 Task List（任务清单）：第一步碰哪个文件、第二步改哪一行、第三步怎么验证。清单你看过、觉得靠谱，再让它动手。这就是前面学过的长任务管理，只是搬到了代码上。

F11. 修改一个小功能

挑一个最小的功能动手——改个文字、换个颜色、动个位置都可以。越小越好。让它按 Task List 改，改完立刻在你自己的浏览器或窗口里看效果。这一步的目的不是改出多酷的东西，是让你完整走一遍“提需求—改—看效果”。

F12. Test

Test（测试）就是：你亲自用一用，看它是不是按你想的那样工作。你是零基础，但你正好是最好的测试员——你就像一个真实用户一样点一点、试一下。它自己说“我测过了”不算数（回想铁律），你点过了才算。把你试的几种情况都记录下来。

F13. Debug

Debug（调试）就是东西没按预期工作时，找出哪里出了问题。发生了别慌：先把现象描述清楚——你做了什么、它显示了什么、你期望它怎样。把这三句话连同报错截图一起给 Agent。它给的修复方案，仍然要你跑一遍确认。它说“应该好了”，你再点一次，好了才算好。

F14. Regression Test

Regression Test（回归测试）是个听着吓人、其实很朴素的概念：你改了新地方之后，把以前能正常工作的老地方也再试一遍，确认新改动没有把旧功能弄坏。就像修好了水龙头，顺便看看马桶还冲不冲。Agent 很容易“修一个坏三个”，这一步就是防这个的。

F15. Git

Git（版本控制工具）是一个帮你记录“文件每一次改动”的工具。你可以把它理解成游戏的存档系统：改坏了，随时读档回到上一个正常的地方。零基础的你不需要懂它的全部原理，只要养成习惯：每完成一个小的、正常工作的改动，就存一个档。后面所有“改坏了”的恐慌，都会被这个习惯消掉。

F16. Commit

Commit（提交）就是上面说的“存档”。每存一次，你要写一句话：这次改了什么。这句话写给三个月后的你看。让 Agent 帮你执行，但那句话你要自己想清楚——“改了点东西”不是 commit 说明，“把首页按钮改成蓝色”才是。

F17. Branch

Branch（分支）是一条平行的修改线。主分支是“现在能正常用的版本”。你要试一个比较大的改动时，开一条新分支去折腾；折腾成了再并回主分支，折腾砸了直接扔掉这条分支，主分支不受影响。新手最容易犯的错是直接在主分支上乱改——开分支就是给你一个“放心折腾”的安全区。

F18. GitHub Push

Push（推送）就是把你电脑上存的这些改动，上传到 GitHub 那个仓库上。到这一步要特别小心：上传是对外的动作。上传之前确认：这是你的项目或你有权改的项目、里面没有密码密钥、你确实想让它公开可见。涉及上传的动作，按全书的风险分级属于 L3，先确认再按。

F19. Pull Request

Pull Request（合并请求，简称 PR）是 GitHub 上一种礼貌的“申请”：你改完了一条分支，发个请求说“我觉得这些改动不错，要不要合进去”。如果你改的是别人的项目，PR 就是你贡献代码的方式——别人会看到你的改动、提意见、决定要不要合并。如果你改的是自己的项目，你也可以自己合并，但保留 PR 这个环节，能让你养成“改完先看一眼再合”的习惯。

F20. Deployment

Deployment（部署）就是把你做好的软件，放到一个能让别人访问的地方，而不只是在你自己电脑上跑。比如把一个小网页挂到网上。对零基础的你，不用一上来就学复杂的服务器——先用现成的托管平台，让 Agent 帮你把项目传上去、生成一个链接。这一步开始涉及对外可见的东西，上传前照例确认一遍内容。

F21. Release

Release（发布）就是你正式宣布“这个版本做好了”，给它起个版本号、写个说明：这版加了什么、修了什么、怎么下载。它不是技术动作，是“对外打招呼”。做完这一步，你就完整经历了一个软件从找到、跑起来、改一点、测一测、存版本、上传、到发布的全过程。你依然一行代码都没自己写过，但你已经知道一个软件项目是怎么运转的了。

旗舰项目模板：借助 Agent 完整走一个开源项目闭环

- 输入材料：一个你按上面四条标准挑好的、简单且活跃的开源小项目（自己 Fork 或只在本地 Clone 均可）。
- 步骤：Clone 到本地 → 读 README 跑起来 → 让 Agent 画一张“文件地图” → 选一个最小改动（改文字/换颜色/加一句欢迎语） → 写成 Task List → 按清单改 → 本地看效果 → 测几种情况 → 回归测旧功能 → 本地 Commit 存档。有余力再注册账号，走 Push → 开 PR → 部署 → Release。
- 产物：跑起来的本地项目、一份“文件地图”、一次最小功能改动、测试记录、本地 Git 提交历史。
- 验收标准：你亲眼在自己屏幕上看到它跑起来；改动后效果符合预期；以前能用的功能回归正常；每完成一个小步都有 Git Commit 存档；对外动作（Push/PR/Deploy/Release）发生前你已确认内容且没有密钥外泄。

✓ 本教程已落地的真实案例：见 `examples/labs/labf_project/`。

选真实开源项目 [tartley/colorama](https://github.com/tartley/colorama)（BSD-3，纯标准库，浅克隆至 repo/），已完成：`git clone` → 读 README → 运行 `demos/demo01.py` → 改 `demo01` 末尾加一行 banner（diff 存 `change.diff`） → 写 `test_smoke.py` 并跑通（`smoke_test.log` 输出 `SMOKE OK`） → 本地 commit（`git_log.txt`，commit `2874e5f`）。如**实标注**：未执行 `git push`、未向上游提 PR——第三方仓库无写权限，教学改动只留在本地。逐项验收见 `CHECKLIST.md`。

走完 F1–F21 你该有的心态

会用 Agent 改一个按钮，不等于你会做软件；但它比绝大多数“看了十本编程教程却从没跑通过一个项目”的人，离真实开发更近。真正的门槛从来不是写代码，而是：找得到、跑得起来、改得动、测得出来、坏了能退、做好了敢发。这套本事，你已经有了。

LAB G：日常工作 / 个人效率 Agent

这个 LAB 解决什么问题

前面几个 LAB 都围着“一个具体任务”转。这个 LAB 围着“你这个人每天的杂事”转：邮件、日程、文件、待办、会议、信息跟踪、知识库、周报、工作总结、自动提醒。这些事的共同点是碎、重复、但又不能全撒手。

可以让 Agent 接管什么

邮件：让它读收件箱，按“需要我今天回/这周处理/只是通知/垃圾”分类，并给需要回的邮件起草回复草稿。注意——起草可以，发出去不行（这是全书反复讲的 L3 动作，必须你点）。

日程和待办：让它把你散落在备忘录、聊天记录、邮件里的事抽成一张待办清单，按时间和紧急程度排好，每周一早上给你一份“这周要做什么”。

会议：会前帮你把相关背景材料读一遍列成两页要点；会后把录音或你的笔记整理成纪要，分出“决定了什么、谁负责、什么时候交”。

文件：定期让它把下载文件夹、桌面、截图文件夹按规则归一次档（这是 PART 1 那个 Mission 的长期版）。

信息跟踪：你关心的几个主题（某个政策、某家公司、某个领域的新论文），让它每周帮你扫一遍有没有新东西，有就摘要，没有就不打扰——这个我们在 PART 9 会把它做成自动化。

知识库：把你平时攒的笔记、读过的论文、写过的报告，定期让它整理成一个可检索的个人知识库。

周报和工作总结：到周五，让它从你这周的待办完成情况、会议纪要、改的文件里，拼一份周报初稿，你改改就发。

自动提醒：把“该交什么、该回谁、该续费了”这类时间敏感的事，让它提前提醒你。

成功标准

- 你能说出三件它替你省下来的、具体的、每周都发生的事。
- 它起草的邮件和纪要，你改两笔就能用，而不是重写。
- 涉及“发出去、存进去、公开”的动作，永远停在你这里等确认。

旗舰项目模板：个人事务管家 Agent

- 输入材料：你真实的一周邮件、日程、待办、下载文件夹，以及你常用的周报模板。

- 步骤：让它读邮件分四类并起草需回邮件 → 把散落各处的事抽成待办清单 → 会前列两页背景要点、会后整理成纪要（决定/负责人/截止）→ 定期把下载和桌面按规则归档 → 关心的主题每周扫一遍新东西 → 周五拼一份周报初稿。
- 产物：邮件分类与回复草稿、待办清单、会议纪要、归档后的文件夹、周报初稿。
- 验收标准：说出三件它每周替你省下的具体事；它起草的邮件和纪要你改两笔就能用；凡“发出去/存进去/公开”的动作都停在你这里等确认。

PART 8 小结：怎么选路

七个 LAB 不必全做。按你的身份选：

- 你是本科生、刚入学：从 LAB A 开始，顺带 LAB G 管日常。
- 你是研究生、要发论文：LAB B 是主线，LAB E 配合数据分析，LAB D 配合查资料。
- 你要做学生项目、创业、找实习：LAB C 加 LAB D 是最实用的组合。
- 你对技术本身好奇、想搞懂软件怎么回事：LAB F 值得从头到尾走一遍。

做完一个 LAB，别急着开下一个。先把这个 LAB 里跑通的流程存成自己的模板——这就是 PART 7 讲的 Reuse，也是 PART 9 要讲的“把它变成自动跑”的前提。

PART 9：自动化与长期运行

F21 自动化闭环

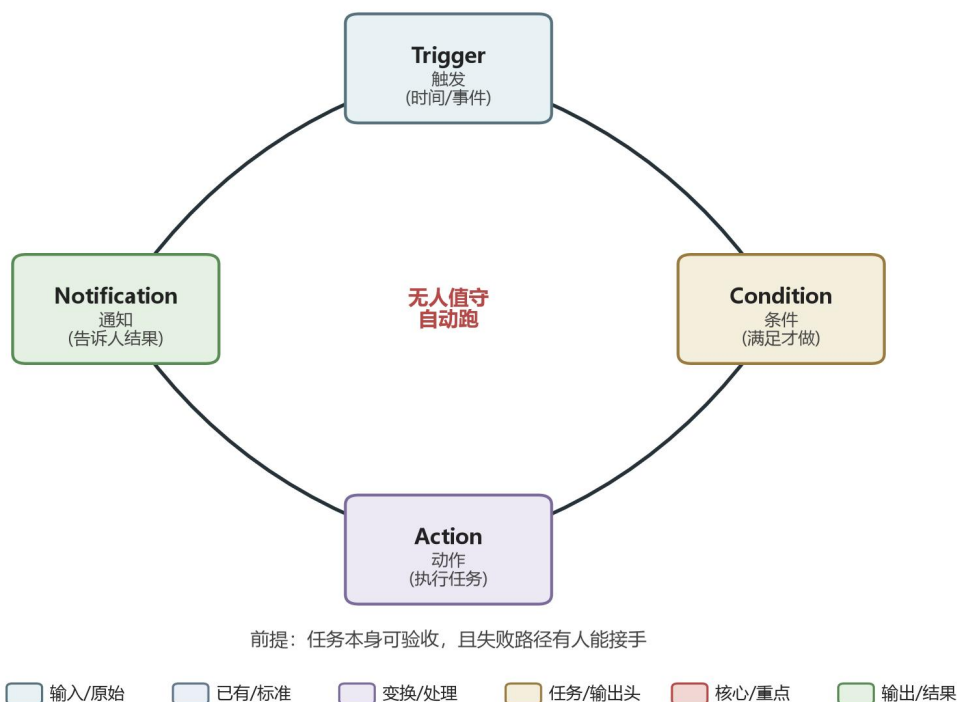


图 23 自动化触发-条件-执行闭环

前八讲，你做的每一件事都是“你喊一声，Agent 动一次”。这一部分要解决一个新问题：有些事你不想每天喊。

你不想每天早上八点自己去搜一遍关心的新闻，也不想每来一封邮件就手动让 Agent 分类。你想要的是：定好规则，它自己跑；没事别烦你，有事再叫你。这就是自动化（Automation）——从“我让它做一次”进入“数字工作流”（Autonomous Workflow）。

但自动化的第一课不是“怎么让它跑起来”，而是“哪些动作绝不能让它自己跑”。所以这一 Part 会把一条流水线拆成八个零件，然后用两个真实例子把它们装起来。

9.1 Trigger（触发）

Trigger（触发）是“什么情况发生了，这个工作流才开始动”。它就像一个闹钟：闹钟响了，后面的动作才发生；不响，一切静止。

在给 Agent 派活时，触发就是你说的那句“开始吧”。一旦你想让它长期运行，你就得把“开始吧”这件事也交给某个客观条件来做，而不是靠你记得。触发有两大来源：一种是到点了（时间），一种是出事了（事件）。这两种分别对应下面的 9.2 和 9.3。

设计自动化时，第一个要问的问题永远是：“它凭什么开始？”触发条件写不清楚，整条流水线要么不该动的时候乱动，该动的时候不动。

9.2 Schedule（定时）

Schedule（定时）就是“按时间触发”。每天早上八点、每周一九点、每月一号、每两小时一次——这都是定时。定时任务是最简单、最容易预测的一种自动化：时间到了就跑，跑完就停。

你要决定三件事：什么时候跑（频率和时间点）、跑多久（超时就停，别让它卡住一直占资源）、跑出来的结果放哪（存成文件还是直接推给你）。定时任务最大的好处是可预期——你知道它明天八点会再来一次。最大的坑也是它“雷打不动”：周末你放假它也跑，节假日你不在它也跑，跑出来一堆没人看的结果。所以定时任务一定要和后面的“条件判断”配合：到点了先看看有没有真事，没事就闭嘴。

9.3 Event（事件）

Event（事件）是“某件具体的事发生了”。来新邮件、收到表单提交、网页更新了、数据库多了一条记录、代码仓库收到一个新 Pull Request——这些都是事件。事件触发和定时触发的区别是：定时是“到点就动”，事件是“有事才动”。

技术上，事件通知最常见的机制叫 **Webhook（网络钩子）**。零基础理解它：**Webhook** 就是“当某件事发生时，那个系统自动往你指定的一个地址发一条消息”。比如你的邮箱系统说“有新邮件了”，它就自动把这个消息推给你的自动化 workflow，workflow 这才醒过来开始干活。你不用每隔五分钟去邮箱问一句“有没有新邮件”，有邮件它自己会来敲门。

事件触发比定时更省资源、也更及时，但它更依赖外部系统“会不会真的来敲门”。所以重要的事件流，往往还要配一个定时检查作为兜底——万一 **Webhook** 漏了，定时任务还能补一次。

9.4 Condition（条件）

Condition（条件）是“触发之后，先别急着干活，判断一下该不该真干”。它是整条流水线里最能帮你“少被打扰”的零件。

举个例子：每天早上八点抓取了一批新信息，这时候不能直接把整批信息发给你——那样你每天早上都会收到一大坨没用的东西。要先过一道条件：这批新信息和昨天相比，有没有真正重要的变化？所谓“重要”，是你事先定好的规则：出现了我关注的关键词、数字变动超过了某个幅度、来了一条新的官方公告。有，才往下走；没有，这一轮直接结束，连通知都不发。

条件这一步的价值，是把“每次都跑满全程”变成“没事就快速退出”。设计自动化时，条件写得越具体，你被打扰得越少。最忌讳的条件是“如果有新内容就通知”——新内容天天有，等于没条件。

9.5 Action（动作）

Action（动作）就是条件满足之后，Agent 实际去做的那件事：抓网页、读邮件、写摘要、更新表格、起草回复、发通知。它就是前八讲你已经很熟悉的那些活，只不过现在被条件拉着自动跑。

设计 Action 时记住两点。第一，每个动作都要有明确的输入和输出，跑一轮就要有一轮的产物（一个文件、一条消息、一行记录），不能“跑了但什么都没留下”。第二，动作本身要分级：只读、只读文件和搜索网页这种，放手让它跑；要往外发、要改线上数据、要花钱的动作，不能放在全自动段里——它们要走到下面 9.6 的人工审批那一步。

9.6 Human Approval（人工审批）

这是本 Part 最重要的一节，请单独看。

Human Approval（人工审批）就是：在动作执行到某个关键节点时，流水线自动停下来，把“我打算这么做”摆到你面前，你点头，它才继续；你不点头，它就停在那儿，什么都不发。

为什么必须有这个节点？因为自动化的可怕之处，不在于它做错，而在于它“错得很有规律、而且停不下来”。它可以在你睡觉的时候，按错误的判断，把措辞不当的邮件发给客户、把没核完的数据贴到公开文档、把一行写岔的代码推到线上。一次错，是事故；自动重复一百次错，就是灾难。

所以铁律是：**凡是对外的、不可逆的、影响别人的动作，都必须经过人工审批，不能让全自动流水线直接执行。**具体说：

- 发邮件、发消息、发帖——草稿自动生成，发送由你点。
- 覆盖原文件、删除数据、Git Push、改云端正式文档——自动生成改动，执行由你确认。
- 花钱、下单、改权限、对外提交——绝不自动化。
- 只读、在副本上分析、写草稿、做摘要——可以自动跑。

审批节点怎么设？不是“每一步都问你”，那样你比手动还累。而是把动作分成两段：全自动段负责读、想、起草、准备好；到了“最后一脚”——发送、发布、提交——才停。你要审的是成品，不是过程。审批消息里必须包含：它打算干什么、依据是什么、你点哪个按钮同意、点哪个按钮拒绝。糊里糊涂的“请确认是否继续”，等于没审。

9.7 Notification（通知）

Notification（通知）是流水线跑完后怎么告诉你。它有三个原则。

第一，有价值才通知。没有变化、没有异常、没有需要你决策的事，就别出声。“一切正常”这种话，发一次是礼貌，发十次就是噪音，最后你会把整个通知关掉，真出事也看不到。

第二，通知要短、要带结论、要告诉你下一步干什么。好的通知是：“这三条有变化，摘要如下，要我展开吗？”坏的通知是：“今日已扫描 47 个网页，完成分析。”

第三，通道要对。需要你决策的（审批），推到你随时看的地方；纯结果摘要，攒到每天一封汇总；错误和异常，立刻推。不要把所有东西都塞进同一个提示音里。

9.8 Recurring Workflow（重复工作流）

Recurring Workflow（重复工作流）是上面这些零件组合起来之后，长期反复运行的那条流水线本身。它和单次任务的区别在于：它会记着上一轮干到哪了、昨天抓的是什么、上次通知你的是哪条。没有这个记忆，它每一轮都从零开始，就没法做“和昨天比有没有变化”这种事。

运行一个重复工作流，你要维护三样东西：一份规则说明（触发、条件、动作、审批点都写清楚），一份运行记录（每轮什么时候跑的、结果如何、有没有报错），一份定期复核（每周或每月看一眼：它最近通知我的东西还有价值吗？条件是不是该调了？有没有哪轮悄悄失败了）。自动化最大的隐患不是第一天，而是第三个月——规则没人维护，它还在按三个月前的判断标准天天跑。

把零件装起来：两个典型流程

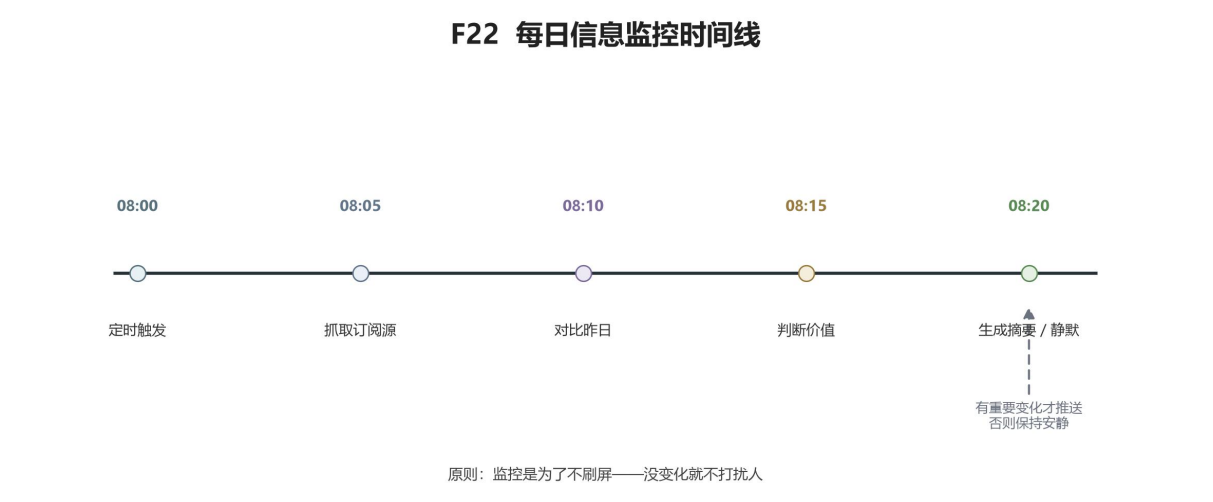


图 24 每日信息监控自动化示例

第一个流程，每日信息监控。

每天 08:00 (Schedule)

↓

抓取你关心的几个来源的新信息 (Action: 抓网页/查更新)

↓

和昨天存下的版本对比 (Condition: 有无真正重要的变化)

↓

└ 有变化 → 写一条摘要，列出变化点和来源 → 推送给你 (Notification)

└ 无变化 → 这一轮直接结束，不打扰你

注意这里的设计：定时触发后，先抓、再比、再判断；判断为“没事”就静默退出，通知只在有事时发。整个过程没有任何对外动作，所以不需要审批节点。这是全自动安全跑的典型。

F23 邮件处理：人在环上审批

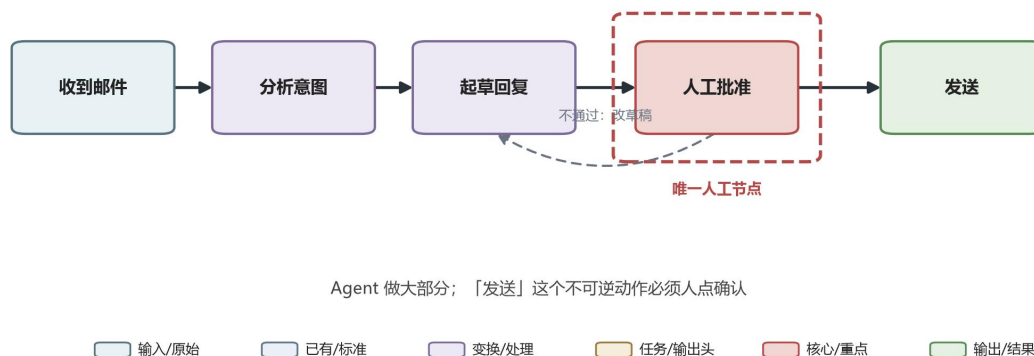


图 25 邮件自动处理人工审批示例

第二个流程，邮件分类与回复。

收到新邮件 (Event: Webhook 来敲门)

↓

分析邮件类型：需要回复 / 仅通知 / 待办事项 / 垃圾

↓

提取需要你处理的任务，归入待办 (Action)

↓

对“需要回复”的邮件，起草回复草稿 (Action: 只起草，不发送)

↓

停下来等你 (Human Approval)

↓

└ 你看了草稿，说“发” → 它才把邮件发出去

└ 你说“重写”或“不回” → 它按你的指示重来，或就此结束

这个流程里，读邮件、分类、起草草稿都是只读和在你这边的动作，可以自动跑；但“发送”是对外的、发出就收不回的动作，必须停在审批节点，由你点。这就是人工审批节点为什么不可省：它把自动化省下来的时间，和不可撤销动作的风险，隔开了。

几个必须分清的概念

定时任务就是 **Schedule** 那一整套：到点跑、跑完停。条件触发是 **Event** 加 **Condition**：有事来敲门，且符合你定的判断标准，才动手。**Webhook** 是事件能“自动敲门”的技术机制，不是你要写代码去造的东西，理解它“来敲门”这个作用就够了。

自动化平台是现成的、把上面这些零件拼起来的工具：你在上面用连线或填表格的方式，把“当某事件发生、满足某条件、就执行某动作、再通知某人”搭出来，不用自己写程序。挑平台时看三点：支持哪些触发源、支不支持中间插一个“等人工批准”的节点、失败了有没有日志能查。

最后说一个新手最容易混的：**Agent** 和传统 **RPA**（**Robotic Process Automation**，机器人流程自动化）有什么区别。**RPA** 是“模拟人手去点软件”——它把你的鼠标键盘录下来，然后机械地重复，画面一变、按钮挪个位置它就崩。**Agent** 自动化的区别在于：它看懂的是任务的意思，不是屏幕上的像素；页面布局变了，它能理解“那个提交按钮大概在哪儿、我要完成的事是什么”，自己再找路。代价是 **Agent** 更灵活、也更不可预测——所以对 **Agent** 自动化，人工审批节点比 **RPA** 时代更重要，而不是更不重要。

案例 1：每日信息监控（完整可运行）

这个案例把上面第一个流程落成一个真实脚本，位置在 `examples/automation/daily_monitor.py`。它抓 **GitHub Releases** 这个稳定只读数据源，和昨日快照对比，只有出现新版本才写摘要。纯 **Python** 标准库，不需要 **pip** 安装任何东西。

它对应八个零件的方式是：**Schedule** 由 **Windows** 任务计划程序负责，**Action** 是抓取与比对，**Condition** 是“和上次快照比有没有新 **Release**”，**Notification** 是写一份摘要文件，**Logging** 全程写 `monitor.log`。整个流程只读不对外，所以不需要人工审批节点。

先把脚本跑通（在 `examples/automation/` 目录下）：

```
# 离线自测：用内置样例数据，不联网
python daily_monitor.py --offline
# 第二次跑，模拟“今天出了新版本”，应看到一条变更摘要写进 reports/
python daily_monitor.py --offline --update
# 再跑一次，应静默退出（已记过，幂等）
python daily_monitor.py --offline --update
```

第一次正式跑会先建立基准快照、不发通知；之后每天跑，只有出现新 Release 才在 reports/ 目录生成一份带日期的摘要，其余日子静默。想监控别的仓库，加 --repo owner/name，例如 `python daily_monitor.py --repo pallets/click`。

把它注册成 Windows 每天早上八点自动跑，用任务计划程序 schtasks。下面这条命令在管理员 PowerShell 里执行一次即可：

```
schtasks /Create /TN "DailyMonitor" /TR "python D:\AISOP\AI-Agent-Tutorial\examples\automation\daily_monitor.py" /SC DAILY /ST 08:00
```

几点说明，别照抄出错：/TN 是任务名字，自取；/TR 后面是要跑的命令，路径用绝对路径，因为任务计划程序的工作目录和你手动不一样；如果 python 在任务计划的 PATH 里找不到，把它换成 python.exe 的完整路径（先在 PowerShell 里跑 (Get-Command python).Source 查到）。查看任务用 `schtasks /Query /TN "DailyMonitor"`，删掉用 `schtasks /Delete /TN "DailyMonitor" /F`。

本机实测输出（2026-09-18）：首次离线建基准写“建立基准快照，不生成通知”；第二次检出 1 条新 Release 并写出 reports/change_*.md；第三次静默退出。日志落在 monitor.log。

案例 2：邮件分类审批 workflow（完整可运行）

这个案例把第二个流程落成 examples/automation/email_triage.py。它读 emails/ 目录里的样例邮件，按关键词分成“工作 / 个人 / 通知”三类，给“工作”和“个人”起草回复，放进 pending 队列停住，等你人工批准后再发送。

先把三条样例邮件放在 examples/automation/emails/（已随教程附带，分别对应三类），然后跑：

```
# 扫描 emails/ 目录，分类+起草+入队
python email_triage.py run
# 查看队列
python email_triage.py list
# 批准某条（把 <id> 换成 list 里看到的真实 id）
python email_triage.py approve <id>
# 拒绝某条，标记为不回复
python email_triage.py reject <id>
```

安全设计是这个案例的重点：分类和起草是只读、本地动作，可以自动跑；但“发送”是对外、不可逆动作。脚本默认不真发——它把待发邮件的收件人、主题、正文写进 outbox.log，等你人工外发。只有当你真的在环境变量里配齐 SMTP_HOST、SMTP_USER、SMTP_PASS 三个值时，approve 才会通过 SMTP 真的把信发出去。也就是说：没填凭据，就永远不会真的把邮件发出去。

```
# 仅示例：真要发信时才设这三个变量（不写进代码、不提交公开仓库）
$env:SMTP_HOST = "smtp.example.com"
$env:SMTP_USER = "you@example.com"
$env:SMTP_PASS = "你的授权码"
python email_triage.py approve <id>
```

本机实测输出（2026-09-18）：三条样例邮件分别被判为“工作 / 个人 / 通知”并各自拿到草稿；批准工作邮件时因未配 SMTP 凭据，状态记为 **approved** 并把待发内容落进 **outbox.log**；拒绝朋友聚餐邮件记为 **rejected**。

这个案例和真实邮箱之间还差一层对接——真要让它读你收件箱，需要按邮箱服务商的 IMAP 设置写一段拉取代码替换掉“读 emails/ 目录”这一步；审批节点和“无凭据不真发”的安全设计原样保留即可。这一步涉及读取真实邮件，建议先在样例数据上把流程跑顺，再换成真邮箱。

Mission：搭一个真正长期跑的小自动化

- 名称：每日信息监控或邮件草稿
- 难度：L6（自动化）。
- 目标：搭一条每周至少跑五天的流水线，跑满两周你再评价它。
- 怎么做：先选一个真正值得每天看的主题；按上面第一个或第二个流程搭；审批节点一定留；通知控制在有事才发。
- 成功标准：连续跑两周，你能说出它帮你抓住了哪几条你本来会漏看的变化；同时这两周里，它没有在没有问你的情况下发出过任何对外消息。
- 常见失败：条件定得太松，每天给你推一堆，一周后你关了通知。修法是把“重要变化”的门槛定得更严，宁可推不推，不要刷屏。

PART 10: 构建自己的 Agent

F24 构建自己的 Agent: 从 Prompt 到部署

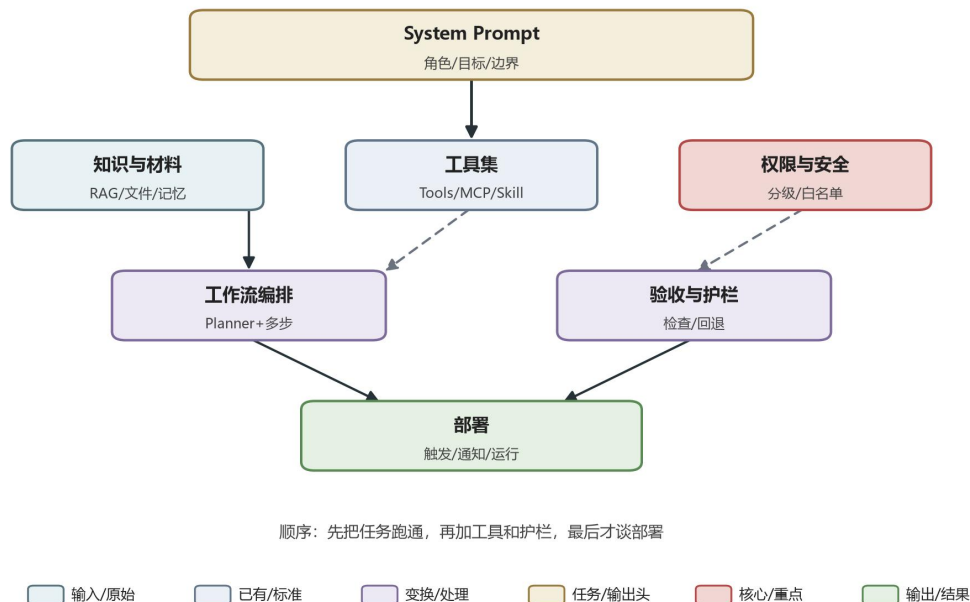


图 26 构建自己的 Agent 蓝图

这一部分只面向一类读者：你已经走完了前面九个 Part，用 Agent 帮自己干了几个月真活，现在想知道“我能不能自己造一个 Agent，让它专门服务我的专业”。

如果你还没到这一步，这一 Part 可以先跳过——它不影响你成为一个熟练的 Agent 用户。它讲的是“拆开一台机器看看里面”，而你前面学的是“把这台机器用好”。

下面二十三个零件，按“它是干什么的、你为什么需要它、零基础的你掌握到什么程度”来讲。不需要每个都自己实现，重要的是你听得懂每个零件解决什么问题、什么时候需要它。

一、入口与规则：System Prompt 与 Runtime

F27 模型选择维度雷达（示例）

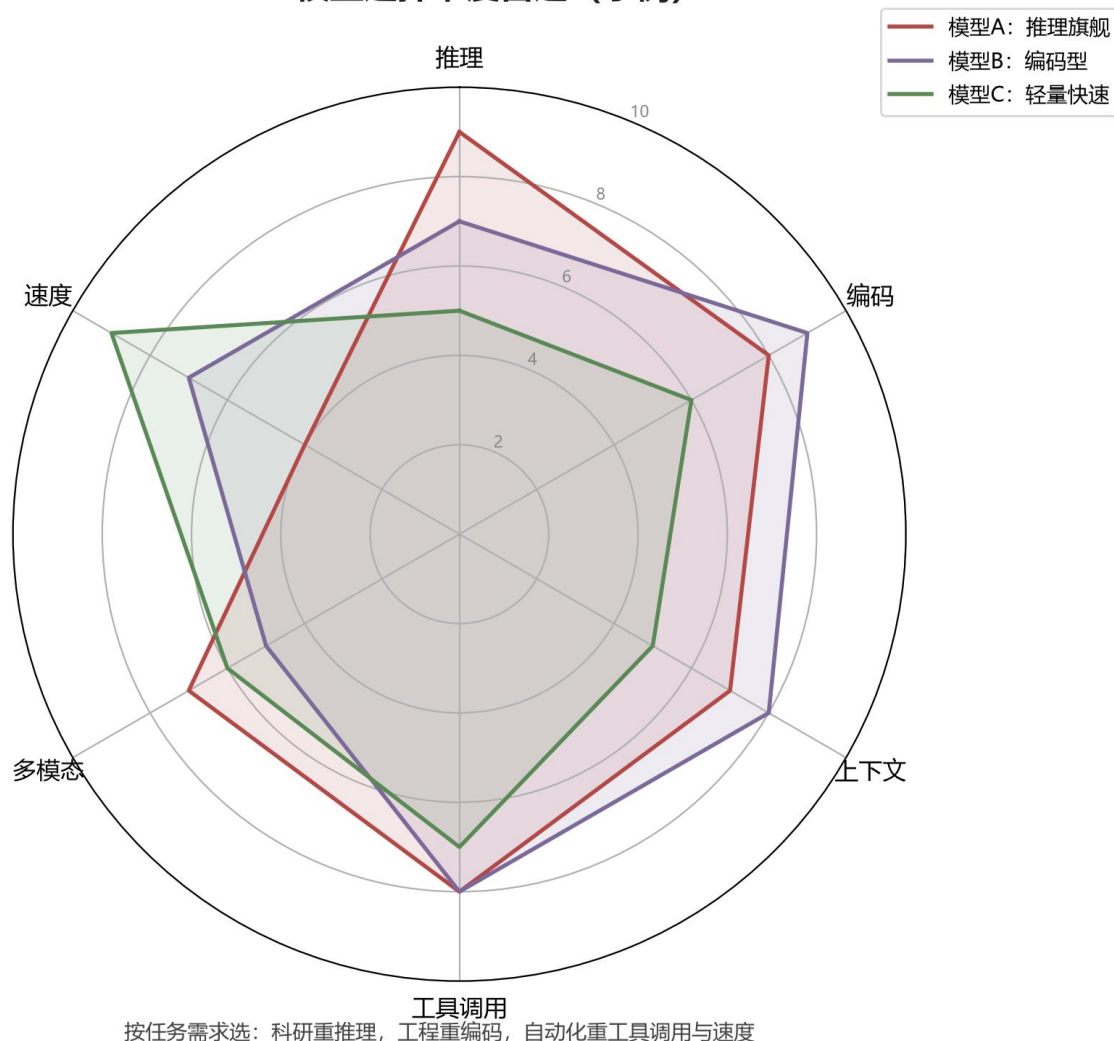


图 27 模型选择维度雷达

1. **System Prompt**（系统提示词）。这是你给这个 Agent 的“岗位说明书”：它是谁、服务谁、擅长什么、不能做什么、按什么流程干活、输出长什么样。前面所有“8 要素任务法”里的目标、约束、流程、验收，长期化之后就是 System Prompt。它不是一句口号，而是一套这个 Agent 每次启动都会先读的工作守则。
2. **Agent Runtime**（运行时）。这是真正把模型、工具、记忆跑起来的那个底座。你写好了岗位说明书、配好了工具，是运行时在负责：把用户的话交给模型、模型说要调工具就去调、调完再把结果喂回去、循环往复直到任务完成。零基础的你不用自己写一个运行时，用现成的 Agent 框架就行；但你要知道，Agent 之所以能“自动循环干活”，靠的就是这个运行时在背后转。

二、工具这一侧：Tool Schema、Function Calling、Skill、Plugin、MCP Server、API

3. Tool Schema（工具定义）。你要让 Agent 能用一个工具，光说“你可以搜索”不够，得写清楚：这个工具叫什么、干嘛用的、需要给它什么参数、每个参数什么意思、什么时候该用它。这份“填表式说明”就是 Tool Schema。模型之所以知道“哦这个情况该调搜索”，就是靠读这份说明。
4. Function Calling（函数调用）。这是模型的一种能力：它不直接回答你，而是输出“我要调用哪个工具、传什么参数”，由运行时真正去执行，再把结果拿回来。你前面用 Agent 时看到的“它决定去搜一下、读一下文件”，底层就是 Function Calling。
5. Skill 开发。当你发现自己反复给 Agent 写同一套指令、配同一套模板、用同一套脚本，就该把它沉淀成一个 Skill——一份说明文件加配套资产。这是最适合零基础起步的“自己开发”：你写的主要是文字和流程，不是代码。
6. Plugin 开发。Plugin 是挂在某个平台上、给该平台扩展能力的模块。它比 Skill 更“平台绑定”，通常要按该平台的规范来做。对你来说，知道它和 Skill 的区别（平台扩展 vs 通用技能）就够了。
7. MCP Server 开发。MCP（Model Context Protocol，模型上下文协议）是一套让 Agent 连接外部工具和数据的标准。写一个 MCP Server，就是把你想用的某个系统（比如你的文献库、你的数据表）按这套标准暴露给任何支持 MCP 的 Agent。它需要一点编程，但好处是“写一次，到处能用”。
8. API 设计。API（应用程序接口）是软件之间互相喊话的标准方式。你要让 Agent 用某个服务，多半是通过调它的 API。零基础的你不用自己设计 API，但要看得懂“这个服务要提供什么地址、什么密钥、会返回什么”。

三、记忆与上下文：Context Management、State、Memory、RAG

F28 上下文 vs RAG：别把所有东西都塞进去

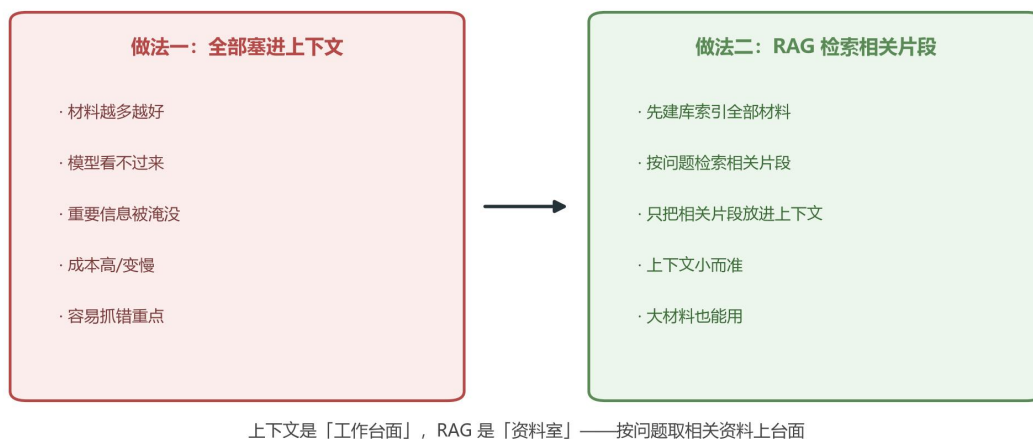


图 28 上下文与 RAG 关系

9. **Context Management**（上下文管理）。模型一次能看的信息有上限（**Context Window**）。任务一长，它就会忘掉开头说的事。上下文管理就是决定“这一轮把哪些信息塞给它、哪些先放一边、哪些从档案里再捞回来”。核心原则前面讲过：最小充分上下文——给它正好够用的，别一股脑全倒进去。
10. **State**（状态）。状态是“这个任务现在干到哪了”：做到第几步、上一步结果是什么、还差什么。它和 **Memory** 的区别是：**State** 是“这一件事的进度”，任务结束就没用了；**Memory** 是跨任务、长期攒下来的。
11. **Memory**（记忆）。让 **Agent** 跨对话记得你的偏好、你正在做的项目、之前的决定。长期记忆要省着用、定期整理，不然它会说过时的偏好当成今天的事实。
12. **RAG**（**Retrieval-Augmented Generation**，检索增强生成）。材料太多塞不进上下文时，做法是：先把材料存成一个库，每轮先从库里“检索”出最相关的几段，再让模型基于这几段回答。这就是你前面“让 **Agent** 读二十篇论文”背后的原理——不是二十篇全塞进去，是每次先捞出最相关的。

四、大脑的分工：Planner、Executor、Evaluator 与 Sub-agent

F26 核心执行循环

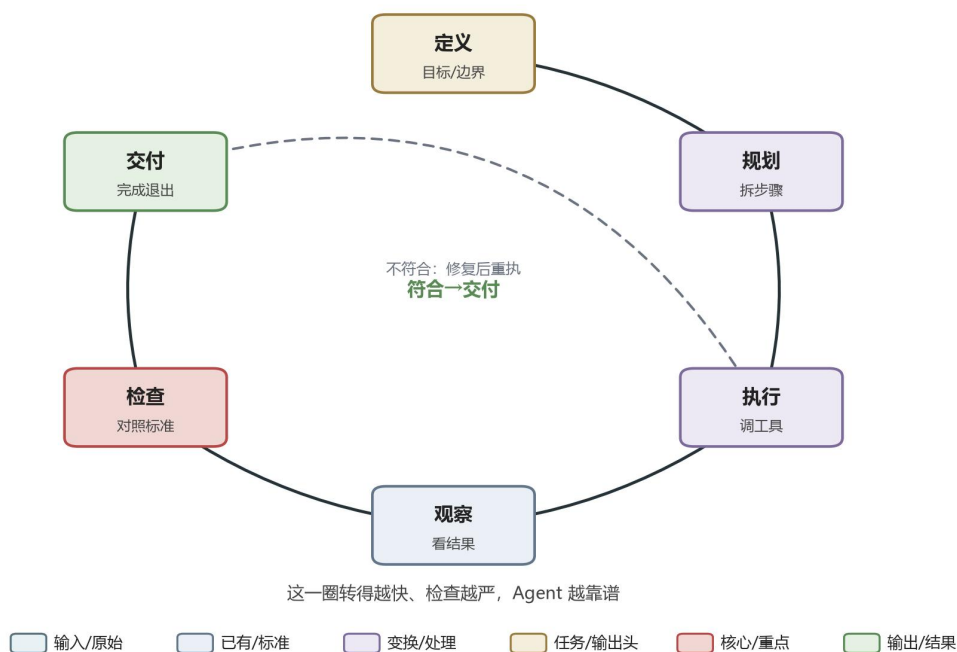


图 29 核心执行循环

F25 Planner - Executor - Evaluator 三角架构

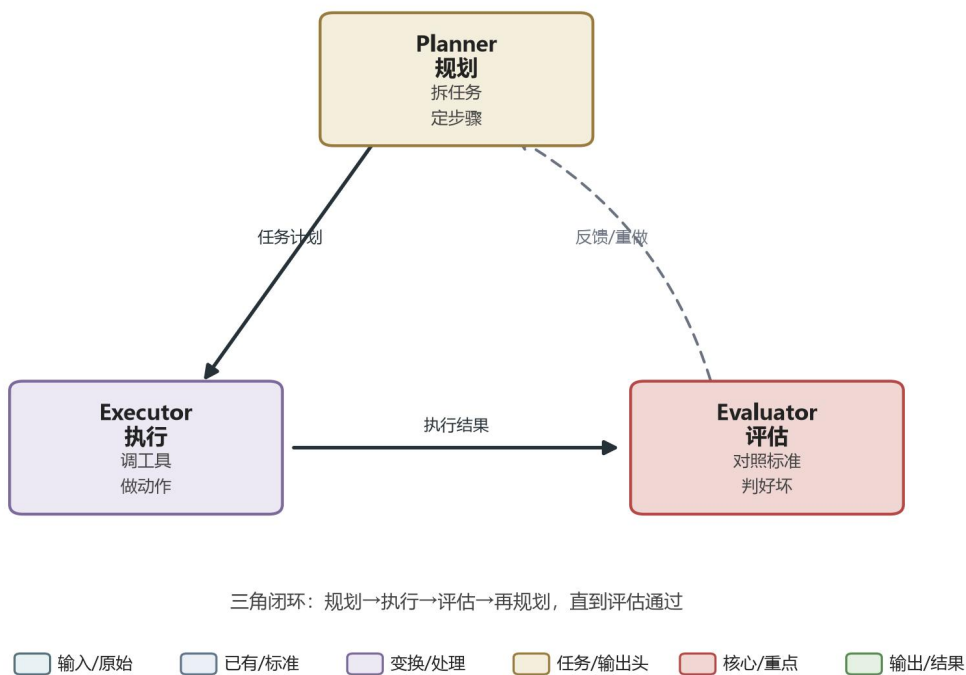


图 30 Planner/Executor/Evaluator 架构

13. **Planner**（规划器）。把一个大任务拆成一步步的小任务。强推理的模型适合干这个——它负责想“先干嘛后干嘛”。
14. **Executor**（执行器）。真正去干每一小步：搜、读、写、算。它可以用快一点、便宜一点的模型，因为它干的是执行而不是思考。
15. **Evaluator**（评估器）。干完一步检查一步：结果合不合格、要不要重来。用一个独立的“挑刺角色”，往往比让干活的人自己评价自己更可靠。这正是全书 PART 4 讲的“独立审查”。
16. **Sub-agent**（子智能体）。主 Agent 忙不过来时，派一个分身专门处理一个子任务，干完把结果交回来。好处是每个分身只看自己那摊事，不会被主任务的长上下文拖垮；代价是你要设计好“派出去什么、拿回来什么”。

五、把人和安全接进来：Human-in-the-loop、Hooks、Sandbox、Security

17. **Human-in-the-loop**（人在回路）。就是 PART 9 反复讲的：关键节点停下来等人确认。设计自己的 Agent 时，第一件事就是标好哪些节点必须等人。
18. **Hooks**（钩子）。钩子是“在某件事发生时自动触发一段你指定的动作”——任务开始前先检查环境、每次写文件后自动跑一遍格式检查、出错时自动通知你。它是把全书的“验收纪律”变成自动执行的手段。
19. **Sandbox**（沙箱）。沙箱是一个隔离的、安全的小房间：Agent 可以在里面跑代码、装东西、折腾，但出了事也只关在小房间里，碰不到你真正的文件。凡是让 Agent 跑代码，都尽量先在沙箱里跑。
20. **Security**（安全）。做自己的 Agent，安全是底线不是加分项：密钥不写进提示词、不提交公开仓库；给外部工具的权限最小够用；装别人的 Skill/MCP 之前按那十二问查一遍；对外动作一律走人工审批。前面 PART 7 的 L1–L4 风险分级，搬到这里就是你自己 Agent 的权限设计。

六、让它可看、可测、可上线：Observability、Evaluation、Deployment

21. **Observability**（可观测性）。你的 Agent 跑起来之后，你得能看见它每一轮干了什么：调了哪个工具、给了什么参数、拿到什么结果、为什么这一步这么决定。没有这个，它出错了你就是瞎猜。让它把每一步记成日志，是可观测性的起点。

22. Evaluation（评估）。怎么知道你这个 Agent 做得好不好？不能靠感觉。准备一组固定的测试用例（十个二十个典型任务），每次改完它都跑一遍，看哪些过了、哪些退化了。这就是前面 LAB F 的 Test 和 Regression Test，只不过对象换成了你自己的 Agent。

23. Deployment（部署）。把你做好的 Agent 从“你电脑上能跑”变成“你（和别人）真的能日常用”。对零基础起步，先用最现成的方式跑起来，别一上来就学服务器运维；能用稳定，比架构炫重要。

零基础怎么迈出第一步



图 31 贯穿案例成长路线

把这二十三个零件列出来，不是要你全部自己造一遍。对刚毕业的人，正确的顺序是反过来：先挑一个现成的 Agent 平台或开源框架，把上面能直接用的零件用起来——别人已经做好的运行时、工具调用、记忆管理，直接拿来；你真正要自己花心思的，只有三样：写清楚你的 System Prompt（这个 Agent 到底服务什么、怎么干）、接好它要读的资料和要调的工具、以及设计好“怎么算干得好”的测试。框架会换、模型会升级，这三样不会变。

不要一上来就想“我要从零写一个 Agent Runtime”。那是几十人团队几年的活，不是你毕业项目的目标。你的目标是“用现成零件，拼出一个真正替你干活的东西”。判断你有没有学会，标准也很朴素：一个你不懂技术的同学，拿着你写的说明书，能不能照着装好、跑起来、出一次合格结果；出了错，你能不能靠 Observability 留下的日志，说出它错在哪一步。能做到这两点，这个 Agent 就不只是一次作业，而是你以后真能用的工具。

配套项目：examples/my-agent（本机实测可跑）

为了让上面二十三个零件不再是抽象名词，教程在 `examples/my-agent/` 放了一个最小但真实可运行的 Agent。它用纯 Python 标准库写，不需要 API Key、不联网、不需要 pip 装任何东西，就能跑起来。零基础说明书见该目录的 `README.md`。

它把六个核心零件各落了一处代码：System Prompt 在 `system_prompt.txt`（可改），工具在 `tools.py`（读文件 / 安全计算器 / 查时间），Context 是对话历史列表，State 落到 `state.json`，遇到“删除 / 发送”类危险动作会停下来走人工审批，每一步写进 `agent.log`。没有 API Key 时它用一个确定性规则引擎跑——这正好对应前面说的“先把零件结构看懂，以后接大模型只换大脑，不换骨架”。

三步跑起来（在 `examples/my-agent/` 目录下）：

```
python test_agent.py    # 冒烟测试，应全部 PASS
python eval.py          # 评测脚本，输出 X/X 通过
python agent.py         # 交互式和它对话
```

本机实测（2026-09-18，Python 3.14）：`test_agent.py` 十项断言全 PASS；`eval.py` 七个用例 7/7 通过；交互模式下查时间、做计算、读文件、危险动作请求人工批准均按预期工作。它的诚实边界也写在 `README` 里：它是规则引擎不是大模型，只会识别固定句式，不联网、不接外部 API。这些边界正是你以后判断“哪个零件要换成现成框架”的起点。

毕业项目

F31 学习路径地图：从「会聊天」到「构建自己的 Agent」

能力进阶（每一级都在前一级可用的前提下才成立）

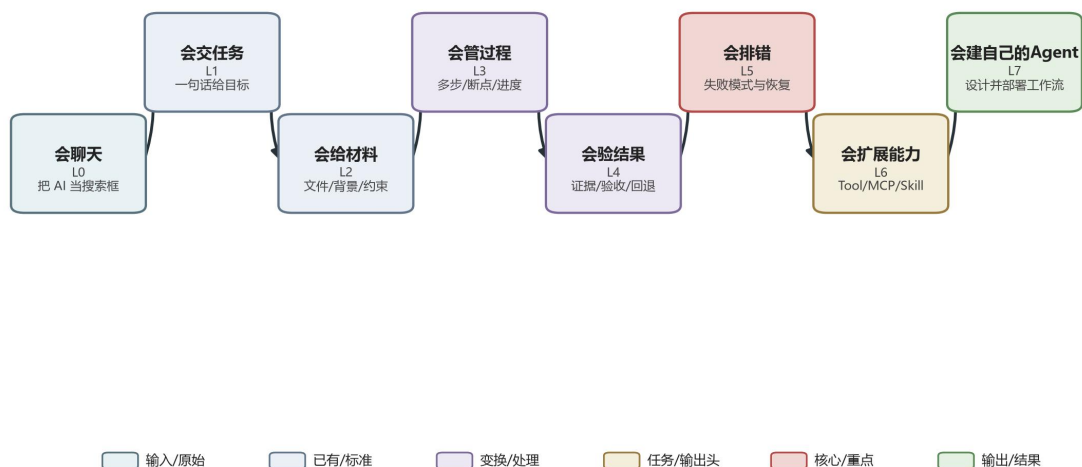


图 32 学习路径地图

走到这里，你的毕业项目是一件事：**设计、实现、测试并交付一个面向你自己专业的 Agent。**

它不需要技术多高深，需要的是把前面学的东西真的用一遍。建议这样做：

第一步，选定一个你本专业里每周都要做、而且做得烦的活。它可能是：文献研究、法律案例梳理、历史史料整理、宏观数据跟踪、市场调研、财务分析、软件维护、课程学习——任何一个你自己真用得上的。别选一个“听起来很酷但你自己不会用”的题目。

第二步，按 8 要素把这个 Agent 的岗位书写清楚：它服务谁、解决什么、材料从哪来、能用哪些工具、哪些动作必须人工批、怎么算干得好。这份说明书就是你的 **System Prompt**。

第三步，配好它需要的工具和资料：要读文件就接文件能力，要查网络就接搜索，要读你的资料库就配一个 RAG。

第四步，按 **Planner-Executor-Evaluator** 的思路把流程串起来：它先规划、再执行、自己先查一遍。

第五步，准备十到二十个测试用例，把它跑一遍，记录哪些过、哪些漏、哪些胡说。针对失败点改，改完用同一套用例再跑一遍——这就是它的回归测试。

第六步，交付。交付物不只是“能跑”，还包括：一份说明书（别人拿过去知道怎么用）、一套测试用例和结果记录、一份“它做不到什么、什么情况必须你来”的诚实说明。

最后，记住全书的结论：**Agent** 的价值不在于它会说多少，而在于它在明确的目标、合适的工具、可控的权限和可靠的验证下，持续完成真实工作。你造出的这个 **Agent**，能不能让你下周还愿意用它、敢把真事交给它，就是毕业与否的唯一标准。

附录 A 术语表

英文	中文译法	一句话解释
Agent	智能体 / 代理	能自主规划并调用工具完成任务的 AI 程序
Chatbot	聊天机器人	只能对话、不能操作外部工具的 AI
Copilot	副驾驶	辅助人完成工作但不自主决策的 AI
Automation	自动化	按预设规则自动执行重复任务
Autonomous Workflow	自主工作流	Agent 自主决定步骤并执行的流程
Prompt	提示词 / 提示	给 AI 的指令文本
Context	上下文	Agent 当前能看到的所有信息
Context Window	上下文窗口	一次对话能容纳的最大信息量
Memory	记忆	Agent 跨对话保留信息的能力
Tool	工具	Agent 可以调用的外部能力
Tool Calling	工具调用	Agent 决定调用某个工具的动作
Search	搜索	Agent 联网查找信息
Browser	浏览器	Agent 像人一样操作网页
Computer Use	电脑操作	Agent 直接操控鼠标键盘操作电脑
File System	文件系统	计算机的文件夹和文件管理
Terminal / Shell	终端 / 命令行	用文字命令操作计算机的界面
Code Execution	代码执行	Agent 运行程序代码来完成计算
API	应用程序接口	软件之间互相调用功能的标准化接口
Connector	连接器	连接外部应用（如邮箱、日历）的组件
Plugin	插件	给 ChatGPT 等平台扩展功能的模块
MCP	模型上下文协议	Model Context Protocol，让 Agent 连接外部工具和数据的标准协议
Skill	技能	一组可复用的指令+模板+脚本，教 Agent 完成某类任务

英文	中文译法	一句话解释
SKILL.md	技能说明文件	描述一个 Skill 的名称、用途、用法的说明文件
RAG	检索增强生成	先从知识库检索相关资料，再让 AI 基于资料回答
Workflow	工作流	按预设步骤串联的自动化流程
Sub-agent	子智能体	主 Agent 派生出来处理子任务的独立 Agent
Goal	目标	最终要得到什么
Context (工作框架)	上下文	Agent 为完成任务需要知道的背景
Resources	资源	文件、网页、数据、代码等可使用的材料
Capabilities	能力	Agent 可调用的工具和功能
Verification	验证	证明工作真的完成且结果可信
Reuse	沉淀/复用	把成熟流程保存为模板、Skill 或自动化
Milestone	里程碑	长任务中的关键完成节点
Checkpoint	检查点	中途暂停让用户确认的节点
Hallucination	幻觉	AI 编造不存在的事实
Git	版本控制工具	记录文件修改历史、可以回退的工具
GitHub	代码托管平台	存放和分享代码的网站
Repository / Repo	代码仓库	GitHub 上的一个项目文件夹
Clone	克隆	把 GitHub 项目下载到本地
Commit	提交	保存一次代码修改记录
Branch	分支	独立修改代码的平行工作线
Push	推送	把本地修改上传到 GitHub
Pull Request	合并请求	请求把分支修改合并到主分支
Dependency	依赖	项目运行需要的其他软件包
OAuth	开放授权	一种让第三方应用安全登录你的账号的协议
API Key	应用程序密钥	调用 API 时证明身份的一串密码
Environment Variable	环境变量	存储配置信息的系统级变量
Sandbox	沙箱	隔离的安全运行环境
RPA	机器人流程自动化	传统的模拟人工操作软件的自动化技术

英文	中文译法	一句话解释
Webhook	网络钩子	事件发生时自动通知某个网址的机制
Planner	规划器	把大任务分解成小步骤的组件
Executor	执行器	实际执行步骤的组件
Evaluator	评估器	检查结果是否合格的组件
Human-in-the-loop	人在回路	关键步骤需要人确认后才继续
Prompt Engineering	提示工程	设计有效提示词的方法
AGENTS.md	Agent 规则文件	项目根目录下给 Agent 读的说明文件

附录 B 资源地图精选

以下资源按 Tier 分级（A=官方基础、B=社区发现平台、C=专业工具集成、D=社区实验），

优先选取官方文档、高价值发现平台和常用工具集成。筛选规则：Tier A 全部保留；Tier B 每个类别选 2-3 个最常用的；

Tier C 选与教程章节直接相关的；Tier D 仅选有明确学习价值的。完整 98 条索引见 01_调研/02_Agent 资源索引.csv。

所有资源核验日期：2026-09-18。

资源名称	类型	来源	用途	等级
Anthropic Agent Skills Standard	Standard	Official	SKILL.md open format spec (name/description frontm	A
Anthropic Skills Engineering B	Standard	Official	Official blog post announcing Agent Skills open st	A
MCP Official Specification	Standard	Official	MCP protocol spec (JSON-RPC 2.0, Host/Client/Serve	A
MCP Official Registry	Registry	Official	Official MCP server metadata registry with DNS-ver	A
MCP Blog / Roadmap	Standard	Official	MCP roadmap and release notes	A
Claude Code Docs (Plugins)	Docs	Official	Claude Code plugin/skill/hook package format docs	A
Claude Code Plugin Marketplace	Docs	Official	How to create/distribute a plugin marketplace JSON	A
OpenAI Codex (github.com/opena	Coding Agent	Official	Open-source terminal coding agent; AGENTS.md conve	A
OpenAI Agents API (Codex harne	API	Official	Cloud agents with Codex harness, fully managed	A
Gemini CLI Docs	Docs	Official	Gemini CLI extensions, skills, GEMINI.md, subagent	A
GitHub Copilot Agent Skills Do	Docs	Official	VS Code / Visual Studio / Copilot CLI Agent Skills	A
GitHub Copilot CLI Changelog	Docs	Official	Copilot CLI releases: plugins marketplace, MCP, ho	A
Microsoft Agent Framework	SDK	Official	Unified SDK merging AutoGen + Semantic Kernel	A

资源名称	类型	来源	用途	等级
OpenAI Plugin Directory	Directory	Official	Replaces App Directory (2026-07-09); packages skil	A
GitHub MCP Server	MCP	Official	Official GitHub MCP server; secret scanning GA	A
anthropics/skills	Skill Repo	Official	Official Anthropic skills: doc skills + examples (	A
openai/plugins (legacy)	Plugin Repo	Official	Codex plugin examples repo (replaces deprecated op	A
huggingface/skills	Skill Repo	Official	Official HF skills: hf-cli, model-trainer, best-mo	A
microsoft/skills	Skill Repo	Official	Microsoft official skills incl. Skills for Fabric	A
NVIDIA/skills	Skill Repo	Official	NVIDIA official skills	A
vercel-labs/skills	Skill Repo	Official	Source repo for skills.sh; 900K+ weekly installs	A
github/copilot-plugins	Plugin Repo	Official	Official Copilot plugins marketplace incl. advance	A
iflytek/skillhub (讯飞)	Skill Registry	Official	Enterprise self-hosted skill registry; RBAC, audit	A
Astron SkillHub (在线实例)	Skill Registry	Official	Hosted instance of iflytek SkillHub	A
ModelScope MCP Marketplace	MCP Directory	Official	中文 MCP 市场 (~5.4K servers)	A
Docker MCP Catalog	MCP Catalog	Official	Containerized, security-scanned MCP server catalog	A
Work IQ MCP (Microsoft)	MCP	Official	Microsoft 365 work context via MCP	A
Microsoft Power Automate	Automation Plat	Official	RPA + cloud flows; desktop automation; Microsoft 3	A
Claude Code	Coding Agent	Official	Terminal coding agent; plugins/skills/hooks/MCP	A
OpenAI Codex	Coding Agent	Official	Terminal + cloud coding agent; AGENTS.md	A
Gemini CLI	Coding Agent	Official	Open-source terminal agent; extensions/skills	A
GitHub Copilot CLI	Coding Agent	Official	Terminal coding agent; plugins/MCP/hooks	A
Antigravity	Coding Agent	Official	Google AI IDE; listed by skills.sh	A
Kiro CLI	Coding Agent	Official	AWS terminal coding agent; listed by skills.sh	A
OpenAI Agents SDK	SDK	Official	Multi-agent workflow SDK; handoffs, guardrails, tr	A

资源名称	类型	来源	用途	等级
Google ADK	SDK	Official	Google Agent Development Kit; GCP-native	A
SWE-bench	Benchmark	Academic	Solve real GitHub issues; Verified/Lite/Pro varian	A
tau2-bench	Benchmark	Academic	Multi-turn tool-use; retail/airline/telecom; pass^	A
Berkeley Function-Calling Lead	Leaderboard	Academic	BFCL V4 function calling AST evaluation	A
OSWorld	Benchmark	Academic	Real desktop GUI automation (Ubuntu/Win/macOS)	A
WebArena	Benchmark	Academic	Sandboxed web agent; 812 templated tasks	A
GAIA	Benchmark	Academic	General AI assistant; Levels 1/2/3	A
Terminal-Bench	Benchmark	Academic	Terminal coding benchmark	A
SkillsMP	Directory	Communit	Aggregates 2.39M+ public SKILL.md files for search	B
SkillHub.club	Directory	Communit	AI-evaluated skill marketplace; 115.3K skills, 65	B
AgentSkillsHub (agentskillshub)	Directory	Communit	Security-graded MCP/skill directory; 156K+ repos;	B
skills.sh (Vercel)	Directory+CLI	Communit	Agent Skills directory + leaderboard; npx skills a	B
agentskills.codes	Directory+CLI	Communit	Open registry of 19,296 installable skills; daily	B
SkillMD.ai	Docs+Directory	Communit	SKILL.md authoring guides and resource-scout meta-	B
agentskill.sh	Plugin Marketpl	Communit	Claude Code plugin marketplace; /learn self-improv	B
sickn33/antigravity-awesome-sk	Awesome List	Communit	1400+ skills registry-style; CLI; renamed from ant	B
Glama	MCP Directory	Communit	MCP server registry superset: 88K servers, maintai	B
PulseMCP	MCP Directory	Communit	Daily hand-curated MCP server directory; ~12K serv	B
mcp.so	MCP Directory	Communit	Community third-party MCP server collection	B
Smithery	MCP Platform	Communit	Discover/deploy/manage MCP servers; CLI, hosted re	B

资源名称	类型	来源	用途	等级
Pipedream MCP	MCP Platform	Communit	Exposes thousands of SaaS app actions as MCP tools	C
Composio	Integration Pla	Communit	AI-native agent integration; 850+ connectors; MCP	C
Composio Connect	MCP Gateway	Communit	Single MCP connection to 1000+ apps via 7 meta-too	C
Zapier	Automation Plat	Commerci	9000+ app automation; Zapier MCP exposes actions t	C
getAbstract MCP	MCP	Commerci	Semantic search/Q&A over getAbstract book summarie	C
n8n	Automation Plat	Communit	Workflow automation + AI agents; 500+ integrations	C
Make	Automation Plat	Commerci	Visual scenario builder; built-in AI Agents	C
Dify	Agent Builder	Communit	Open-source LLM app/agent platform; visual workflow	C
Coze (扣子)	Agent Builder	Official	ByteDance bot/agent builder; visual orchestration,	C
Langflow	Agent Builder	Communit	Low-code visual agent/MCP builder; LangChain ecosy	C
openai/skills (DEPRECATED)	Skill Repo	Official	DEPRECATED 2026-06-22; moved to Plugin Directory	D
travisvn/awesome-claude-skills	Awesome List	Communit	Curated Claude Code skills list by category	D
ComposioHQ/awesome-claude-skil	Awesome List	Communit	Large community skills list (~66K stars)	D
hesreallyhim/awesome-claude-co	Awesome List	Communit	Broad Claude Code eco: skills, hooks, slash comman	D
VoltAgent/awesome-agent-skills	Awesome List	Communit	1000+ cross-platform skills (Claude Code, Codex, G	D

附录 C 8 要素任务法模板

每次给 Agent 下任务前，按以下 8 要素逐项写清楚。复制此模板，填空即可。

要素	英文	要回答的问题	填写示例
目标	Goal	最终要得到什么？（有数量、有范围、有形式）	周末前把第三章整理成知识结构图，覆盖全部小节
材料	Materials	可以用什么？（材料在哪、哪些必读）	材料在 D:\课程\论文\文件夹，01.pdf 必读
环境	Environment	在哪里工作？（圈定活动范围）	只在 D:\课程\论文\文件夹内工作
权限	Permissions	可以做什么？（读/写/删/对外发送）	可读可写，不可删原始文件，不可对外发送
约束	Constraints	什么不能做？（红线）	不得编造文献；不得覆盖原始文件；中文写作
过程	Process	按什么步骤做？	先盘点文件→提取要点→填表→自检→交付
验收	Acceptance	怎么证明做完了？	知识结构图覆盖全部小节，每小节一句话
交付	Deliverables	最后交出什么？	一份 .md 文件 + 一张结构图 .png

使用方法：把这个模板复制到你的任务说明开头，逐项填写。8 项写齐了再发给 Agent，不要只写“帮我做 XX”。